

Dissociating Artificial Intelligence from Artificial Consciousness

Graham Findlay^{a,1}, William Marshall^{c,1}, Larissa Albantakis^a, Isaac David^{a,b}, William GP Mayner^a, Christof Koch^d, and Giulio Tononi^{a,2}

^aDepartment of Psychiatry, University of Wisconsin, Madison, Wisconsin, United States of America

^bNeuroscience Training Program, University of Wisconsin, Madison, Wisconsin, United States of America

^cDepartment of Mathematics and Statistics, Brock University, St. Catharines, Ontario, Canada

^dAllen Institute, Seattle, WA, United States of America

¹G.F. (Graham Findlay) and W.M. (William Marshall) contributed equally to this work.

²Corresponding author: ghtononi@wisc.edu

Developments in machine learning and computing power suggest that artificial general intelligence is within reach. This raises the question of artificial consciousness: if a computer were to be functionally equivalent to a human, being able to do all we do, would it experience sights, sounds, and thoughts, as we do when we are conscious? Answering this question in a principled manner can only be done on the basis of a theory of consciousness that is grounded in phenomenology and that states the necessary and sufficient conditions for any system, evolved or engineered, to support subjective experience. Here we employ Integrated Information Theory (IIT), which provides principled tools to determine whether a system is conscious, to what degree, and the content of its experience. We consider pairs of systems constituted of simple Boolean units, one of which—a basic stored-program computer—simulates the other with full functional equivalence. By applying the principles of IIT, we demonstrate that (i) two systems can be functionally equivalent without being phenomenally equivalent, and (ii) that this conclusion is not dependent on the simulated system’s function. We further demonstrate that, according to IIT, it is possible for a digital computer to simulate our behavior, possibly even by simulating the neurons in our brain, without replicating our experience. This contrasts sharply with computational functionalism, the thesis that performing computations of the right kind is necessary and sufficient for consciousness.

Artificial intelligence (AI) is progressing rapidly, fueled by theoretical and hardware developments in machine learning [1–3]. Though large language models such as ChatGPT and vision-language-action models (implementing robotically embodied AI) still lack several human capabilities, they perform spectacularly on a wide array of tasks, demonstrating abilities that were once the stuff of science fiction [4–6]. Consequently, a majority of AI researchers expect that artificial general intelligence is within reach, leading to systems that perform at least as well as humans in most and, eventually, in all cognitive domains [7]. Such prospects raise the following question: would machines with human-like intelligence also have human-like consciousness? More precisely, would they have our subjective experiences—“what it is like” to perceive a scene, endure a pain, or entertain a thought in one’s mind [8]? Because of the torrential growth of the size and power of AI systems, these questions are shifting from purely hypothetical to practically significant: how we interact with AI, and what rights we accord to them, will hinge on whether we consider them as conscious entities who share our subjective experiences, including pleasure and pain [9–12].

When Alan Turing proposed what is now known as the Turing test in 1950, he emphasized that his “imitation game” was a test of intelligence, not of consciousness [13]. Nevertheless, many have since embraced computational functionalism: “the thesis that performing computations of the right kind is

necessary and sufficient for consciousness” [14, 15]. So far, there is little agreement on what constitutes a computation “of the right kind.” Most functionalists seem to agree that if computers were to replicate our cognitive functions in every respect, they would also enjoy our subjective experiences. Others propose stricter definitions of functional equivalence: to be conscious like us, computers should simulate not only our cognitive functions but the detailed causal interactions that occur in our brain—say at the level of individual neurons [16].

But is functional equivalence really sufficient for phenomenal equivalence? To address this question, it is critical to rely on a general theory of consciousness, one that states the necessary and sufficient conditions for any system, evolved or engineered, to support subjective experience [17]. In this paper, we address the issue of computer consciousness on the basis of Integrated Information Theory (IIT) [18–20], which offers a principled explanation and a mathematical framework to determine whether and how a system is conscious.

Crucially, IIT does not start from neural correlates of consciousness, such as neural activity patterns [21–23], nor from cognitive functions often associated with consciousness, such as broadcasting or monitoring information, or adjusting an inferential model of the world and our actions [24–28]. Instead, IIT starts with the essential properties of consciousness itself—those that are irrefutably true of every conceivable ex-

perience. These are IIT’s five *axioms of phenomenal existence*: every experience is intrinsic (for itself), specific (this one), unitary (a whole, irreducible to separate experiences), definite (this whole, containing all it contains, neither less nor more), and structured (composed of distinctions bound by relations that make it feel the way it feels).

The theory then formulates these five essential phenomenal properties in terms of corresponding causal properties—IIT’s five *postulates of physical existence*—which a substrate must have to support consciousness. These five postulates—intrinsicity, information, integration, exclusion, and composition—can be formulated mathematically and assessed algorithmically for any substrate, given the system’s state and complete causal model (see [Causal models](#) below) [18, 20]. The analysis identifies systems that can support consciousness, called *complexes*. The causal powers of a complex are then fully *unfolded*, yielding a *cause-effect structure* [18, 20]. Finally, IIT claims that the quality of an experience—“what it is like to be” in a specific phenomenal state—is fully accounted for, with no additional ingredients, by that complex’s cause-effect structure [18, 20].

IIT is supported by growing scientific evidence about our own consciousness. For example, IIT provides a self-consistent account of why the cerebral cortex is important for consciousness but the cerebellum is not, despite having four times more neurons; why consciousness fades in deep slow-wave sleep, even though cortical neurons remain active; and why consciousness might have evolved [19]. IIT also makes several experimental predictions that have been validated in humans [19, 29, 30]. Measures reflecting integrated information break down when consciousness vanishes in deep sleep, under general anesthesia, or after widespread brain damage, and recover during dreaming sleep, dreaming under ketamine anesthesia, or after recovery from brain injury [31]. Indeed, a crude-but-practical proxy for IIT’s integrated information measure, explicitly designed to test the theory’s core tenets, has been successfully employed to detect covert consciousness in behaviorally unresponsive patients [32, 33].

To demonstrate that functional equivalence does not imply equivalence of cause-effect structure, and therefore does not imply equivalence of phenomenology, we first apply IIT’s mathematical framework to a simple target system (the *simulandum*) and to a computer that simulates it (the *simulans*). We show that even though the computer is functionally equivalent to the target system, it is not a complex and fails to replicate the target system’s cause-effect structure. Furthermore, we show that treating the computer’s units and states at *macro grains* [34] does not change this conclusion. We extend these results to non-trivial cases, including a Turing-complete version of the computer that can simulate arbitrary systems with arbitrary functions. It follows that such a computer could, for example, simulate a human brain in micro-physical detail, without replicating its experiences. Our results challenge the default, dominant assumption of *computational functionalism*: that traditional computer systems replicating our cognitive functions or simulating the causal interactions occurring within our brain, will necessarily replicate our experiences, or have substantial experience of any kind.

Theory

This section provides a high-level overview of IIT and highlights aspects of the theory that are relevant to the results presented here. For a more detailed and complete exposition of IIT and its justification, see [18, 20].

Causal models. The starting point of IIT’s analysis is a causal model of a physical system that captures all the interactions within it. Here, “physical” is meant operationally, indicating that the individual units can be manipulated and observed. The causal model is constituted of *micro units*, each having two internal states as well as inputs and outputs. The units are micro units in the sense that no finer detail about them or their function is included in the model. Given a causal model, any set of micro units can be considered as a *candidate system*. Any units not included in a candidate system are referred to as that system’s *background conditions*. The causal model of a candidate system is fully characterized by its *transition probability matrix (TPM)*, causally conditioned over its background conditions [34].

Here we consider models with synchronously updating units implementing Boolean logic, which can be thought of as standing for transistors driven by a system-wide clock. While our idealized units ignore most physical details, they are sufficient to make our points about functional and phenomenal equivalence. Practically, the spatiotemporal grain of transistors and their interconnections is unambiguous, well-demarcated from other grain sizes, and is thought to capture the fundamental causal interactions within engineered silicon circuits [35, 36].

Identifying complexes. According to IIT, a substrate can support consciousness—it is a complex—if and only if it fulfills the five postulates [18–20]. To determine whether a system of units is a complex, one evaluates its *system integrated information* (φ_s)—the extent to which the system’s intrinsic information (capturing the postulates of intrinsicity and information) [37] is affected by its minimum partition into causally independent parts (capturing the integration postulate). To satisfy the exclusion postulate, a candidate system must specify a maximum of integrated information (φ_s^*) compared to all competing candidate systems with overlapping units and grains. This can be determined by evaluating φ_s for every possible set of units within the causal model. Moreover, every system is evaluated at multiple grains, by exhaustively grouping subsets of its micro units into *macro units*, and mapping states of the constituent micro units to states of the resulting macro units. This procedure, called *macroing*, is done because a system’s intrinsic causal powers (φ_s) may be higher at a coarser than at a finer grain, depending on its internal organization [34, 38, 39].

Once the maximum of integrated information within a causal model has been identified, all units of that complex are excluded from participating in other complexes. The search for complexes is then repeated recursively over the remaining units of the system until all non-overlapping complexes are identified.

Unfolding the cause–effect structure of a complex. Finally, composition is evaluated by assessing how a complex’s causal power is structured. Briefly, *distinctions* capture the causal powers of subsets of units, while *relations* between distinctions characterize how their powers overlap. Distinctions and relations are both associated with measures of their irreducibility (φ_d and φ_r , respectively). Together, the totality of a system’s distinctions and relations composes its *cause–effect structure* (also called a Φ -structure). The cause–effect structure fully characterizes what a complex in a state specifies about itself through the congruent causes and effects of its various subsets. The process of identifying all of a complex’s distinctions and relations, and thereby obtaining its cause–effect structure, is referred to as *unfolding*. According to IIT, the cause–effect structure of a complex in a state accounts for the quality of its experience in full, with no additional ingredients needed (“quality is structure,” [18, 20, 40, 41]). The quantity of consciousness associated with a cause–effect structure is measured by its *structure integrated information* (Φ)—the total irreducibility of its distinctions and relations ($\sum \varphi_d + \sum \varphi_r$). If the state of a complex changes, its cause–effect structure may change as well, and therefore its quality of consciousness.

Results

Below, we apply IIT’s causal powers analysis to a target system constituted of four micro units (PQRS) and to a functionally-equivalent four-bit computer, constituted of 117 micro units, able to simulate PQRS indefinitely. We first demonstrate that functional equivalence does not imply equivalence of cause–effect structures at the grain of micro units. We then demonstrate that there is no function-relevant macroing of the computer compliant with IIT’s postulates that replicates the cause–effect structure of the target system. Finally, we extend the four-bit computer to be Turing-complete and demonstrate that the previous results do not depend on the complexity of the function being implemented.

A target system (PQRS) and the cause–effect structure it specifies. Fig. 1A shows PQRS, the target system to be simulated, comprising a set of four binary units whose dynamics are defined by a truth table or, more generally, a transition probability matrix (Fig. 1B–C). For simplicity, we consider PQRS’s causal model in isolation, without background conditions. The system PQRS in state 0101 (also written as $pQrS$) satisfies intrinsicity, information, integration, and exclusion, and is thus a complex, with $\varphi_s = 1.51$ *intrinsic bits* (Fig. 1D), also called *ibits* [37]. Its cause–effect structure, unfolded according to [18, 42], is composed of 13 distinctions and 8184 relations and has $\Phi = 391.25$ ibits (Fig. 1E–F). The system will transition through a repeating cycle of nine states, with varying complexes, cause–effect structures, and Φ values (Supplementary Fig. 9). This minimal system, though it contains $> 8,000$ relations, was chosen to reduce the computational requirements of analyzing the cause–effect structures specified by the computer simulating it, without loss of generality, as shown below.

A computer constituted of 117 units that can simulate PQRS indefinitely. Fig. 2A shows a basic four-bit computer capable of simulating PQRS. This simple digital computer with a Harvard-like, stored-program architecture has recognizable components: a clock, frequency dividers, program memory, an instruction register, data registers, and a multiplexer that combines instructions with data to serve as a minimal processor. The data registers store the state of the system until instructed to update; the program memory stores the function of the units the computer is simulating; the multiplexer’s processing units combine this information about state and function to compute the next state of the target system; the clock synchronizes these processes and tells the registers when to update their state. The clock and program are read-only components of the computer; they output to the data registers and multiplexer but receive no feedback connections. For a detailed explanation of the computer’s operation, and for a comparison to common computer architectures, see the “Step-by-step guide to the computer’s operation” supplement.

When initialized appropriately, this computer simulates the behavior of PQRS. The initialization procedure is simple: First, the states of the program units are set in such a way that they encode PQRS’s state transition rules (Fig. 2B). Next, the states of the units P’, Q’, R’, and S’ are set to reflect the initial state of PQRS (Fig. 2C). Finally, the state of the clock is set as shown in Fig. 2A. The processing units may be initialized to any state. After eight updates, the states of P’, Q’, R’, and S’ reflect a single state transition of P, Q, R, and S (Fig. 2D), and the two systems will continue to produce homomorphic sequences of states indefinitely. This is true regardless of the initial states of PQRS and P’Q’R’S’, hence the two systems—the 4-unit simulandum and the 117-unit simulans—are functionally equivalent (*modulo* eight updates).

The computer fragments into multiple complexes, none of which specifies a cause–effect structure identical to that of PQRS. Does the computer, when programmed to be functionally equivalent to $pQrS$, also replicate its cause–effect structure? By applying IIT’s causal powers analysis to the computer as a whole, we find that, unlike $pQrS$, the computer has $\varphi_s = 0$ ibits (Fig. 3A, grey). This is because the computer contains modules that are connected to the rest of the system in a purely feedforward manner. This violates the requirement for integration, which can be assessed by partitioning the set of units that constitute a system into separate parts. In an integrated system ($\varphi_s > 0$ ibits), every part will both make a difference to— and take a difference from the rest of the system [43].

Most subsets of units also have $\varphi_s = 0$ ibits. In fact, the complete IIT analysis reveals that the computer contains 24 individual complexes, each one constituted of between one and four units, each with $\Phi \leq 6$ ibits (Fig. 3A, blue; “Micro-grain analysis of the weakly connected computer” supplement). Thus, the entire computer is never a single, large complex, no matter the state, but merely an aggregate of smaller complexes, each of which has its own tiny cause–effect structure. Importantly, each of the 24 complexes specify cause–effect structures that differ from the one specified by $pQrS$.

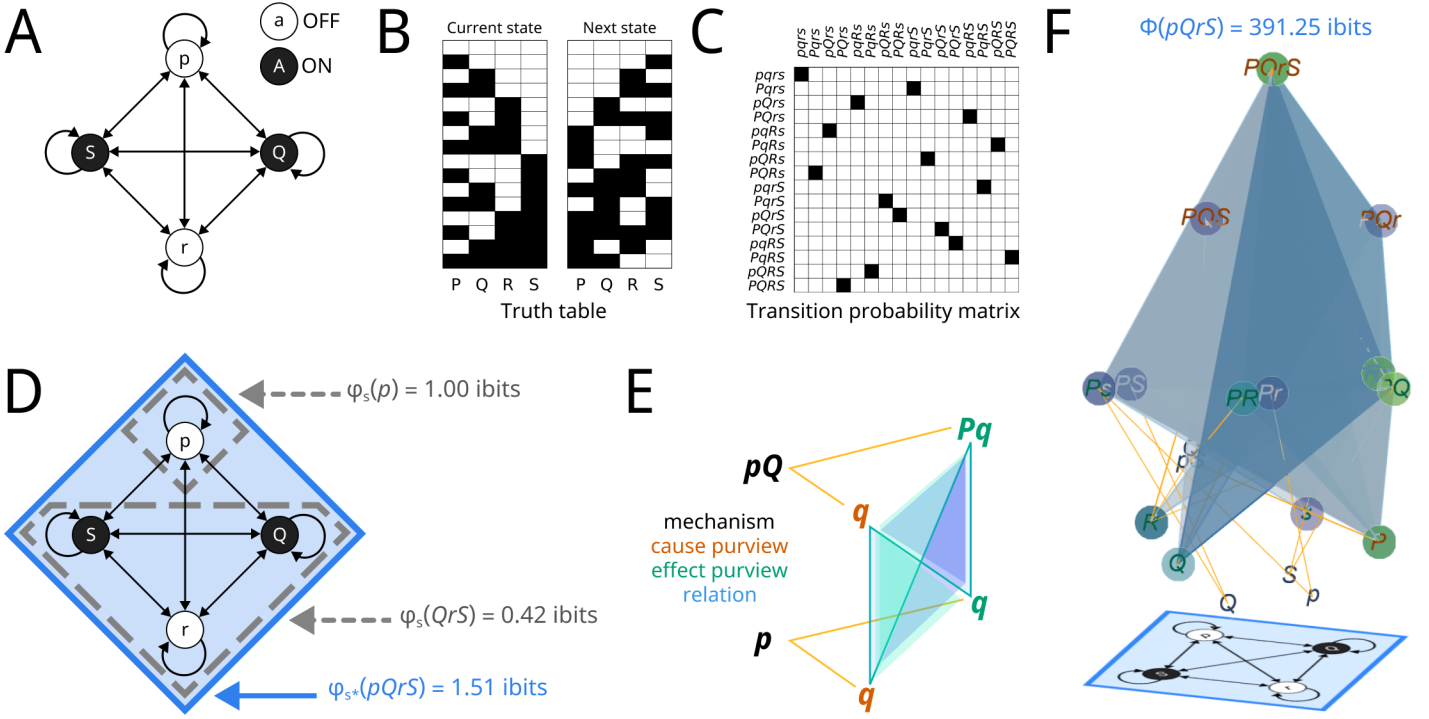


Figure 1: A target system for simulation. (A) PQRs is a substrate of four units with binary states that update synchronously at discrete intervals. The system is fully connected: each unit receives input from every unit, including itself. Here, the system is shown in state 0101, also written as $pQrS$. (B) The units P, Q, R, and S implement custom boolean functions of four inputs which, despite not resembling familiar logic gates, are described by a truth table. The truth table yields, for any possible state of PQRs (left), what its subsequent state would be (right). (C) Since all of PQRs' state transitions are deterministic, this truth table is also equivalent to PQRs' Transition Probability Matrix (TPM; right). (D) Applying IIT's postulates to $pQrS$ and its TPM reveals a single, maximally irreducible complex ($\varphi_s(pQrS) = 1.51$ ibits). This complex is, by definition, more irreducible than all other candidate systems over the same substrate, including QrS ($\varphi_s(QrS) = 0.42$ ibits; grey dashes) and p ($\varphi_s(p) = 1.00$ ibits; grey dashes). All other candidate systems (e.g., pQr , pQS , prS , pQ , pr , etc.; not shown) are fully reducible, with $\varphi_s = 0$. (E-F) The unfolded cause-effect structure of $pQrS$ consists of 13 distinctions (p , q , S , pQ , pS , Qr , QS , rS , pQr , pQS , prS , QrS , $pQrS$) and 8184 relations among these 13 distinctions. Relations bind state-congruent overlaps among the causes and effects of distinctions. Only 2-relations (edges) and 3-relations (triangles) are shown here. For example (D), mechanism pQ specifies a maximally irreducible cause P and a maximally irreducible effect Rs , yielding a distinction. Distinctions pQ and pS jointly specify overlapping and congruent causes or effects (in this case, a cause P), yielding a relation. According to IIT, the cause-effect structure (E) corresponds to the content, or quality, of the experience supported by $pQrS$. The quantity of consciousness is measured by Φ (here equal to 391.25 ibits), a non-negative number equal to the sum of the irreducibility values (φ) of all distinctions and relations in its cause-effect structure [18].

(Fig. 3B). The number of distinctions are smaller (1–4 distinctions, whereas $pQrS$ has 13). The computer has only first-order distinctions (specified by individual units), whereas $pQrS$ also has higher-order distinctions (specified by two or more units). The number of relations is also vastly different (at most four relations for the computer, whereas $pQrS$ has 8184), as are relation order and degree [18]. Moreover, these overall disparities hold true for other states of PQRs (Supplementary Fig. 9) and all other states the computer can support (see “Micro-grain analysis of the weakly connected computer” supplement). Thus, despite being functionally equivalent, the computer and PQRs do not support equivalent cause-effect structures.

The computer's lack of integration is not merely due to lack of feedback. It is possible to modify the computer by introducing feedback connections that alter its TPM, but do not impair its ability to simulate $pQrS$ (Fig. 3C). This version of the computer is strongly connected in the graph-theoretical sense, yet as a whole remains reducible, with $\varphi_s = 0$ ibits. In fact, the same subsystems that were maximally irreducible without feedback connections (Fig. 3A–B) remain maximal even with such feedback in place (see “Micro-grain analysis of the strongly connected computer” supplement). This is because, despite the presence of feedback connections, the computer has many *fault lines*, such as sparse connectivity

and bottlenecks, along which it segregates into smaller, non-overlapping and maximally integrated pieces [43]. In other words, the computer with or without feedback is an aggregate of about two dozen tiny complexes, rather than a single, larger complex in its own right.

The computer's failure to replicate the cause-effect structure of PQRs cannot be rescued by treating its units or states at macro grains. In IIT, the units that constitute a complex, called *intrinsic units*, [34] are those that maximize the complex's existence, as measured by φ_s . In principle, a complex's intrinsic units are established by evaluating the system at all possible grains, exhaustively grouping subsets of its micro units into macro units, and mapping states of the constituent micro units to states of the resulting macro units (Fig. 4A) [34, 38, 39, 44]. This procedure, called macroing, is done because a system's intrinsic causal powers may be higher at a coarser than at a finer grain, depending on its internal organization (Fig. 4B). In such cases, a system's intrinsic units are macro units, rather than micro units.

A complex and its intrinsic units must comply with IIT's postulates of physical existence [18]. Thus, like complexes, macro units can only qualify as units if their cause-effect power is irreducible to that of independent subsets (satisfying integration). Otherwise, one could literally build something (a macro unit) out of nothing (non-interacting micro units;

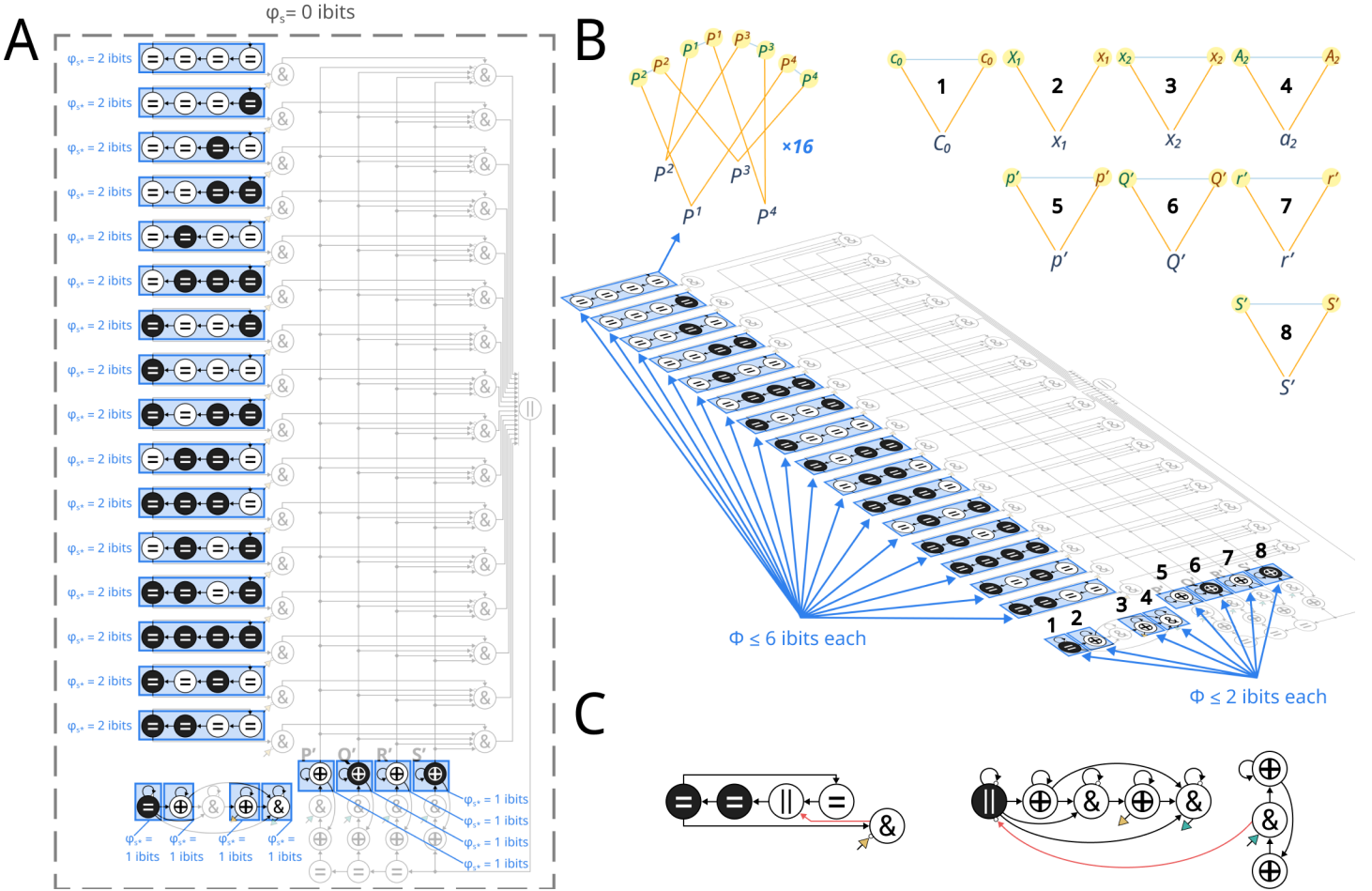


Figure 3: Identifying the computer's complexes and unfolding their cause-effect structures. (A) The computer as a whole is not integrated, regardless of its state ($\varphi_s = 0$ ibits), due in part to the presence of purely feedforward modules. Therefore, the computer is not a complex and has no cause-effect structure. Instead, it fragments into twenty-four disjoint small complexes, each one of which is maximally irreducible (shown in blue). None of these complexes specify a cause-effect structure equivalent to that specified by $pQrS$. (B) Shown here for illustrative purposes are the cause-effect structures of nine complexes depicted in panel A. Due to lack of space, the cause-effect structures of fifteen complexes in the program memory are not shown, but they are essentially identical to the one shown (top left). The cause-effect structures of the eight complexes in the clock and data registers are labeled (1-8). All complexes specify cause-effect structures that are simple, with only first-order relations, and not as rich as that of $pQrS$ (see Fig. 1F). (C) Feedback connections can be added to the computer without disrupting its function; the computer's ability to indefinitely simulate any four-unit Boolean system is unimpaired. Each line of the program has a single COPY unit replaced by an OR unit, to which a feedback connection from the instruction register is wired (red; left). Similarly, the clock has a single COPY unit replaced by an OR unit, to which a negated feedback connection from each of the data registers is wired (red; right).

Fig. 4C). And, just as complexes must be definite, which translates into maximally irreducible cause-effect power (satisfying exclusion), so must macro units. Otherwise, one could build something (a macro unit) out of nearly nothing (weakly interacting sets of micro units; Fig. 4D). Several other constraints on what constitutes a valid macro unit follow from the postulates. For example, intrinsic units should not overlap (satisfying exclusion) [34].

From the extrinsic perspective of a computer engineer, certain macroing schemes may provide a natural way to construct a high-level characterization of the computer. For example, it is useful to abstract away low-level digital logic into high-level components with precise input-output behavior, such as multiplexers, adders, bit shifters, and registers. It is also helpful to chunk together multiple updates of the state of such components into interpretable computational steps.

Fig. 5 shows one of many ways in which one might consider macroing the computer from an extrinsic, computational-functionalist perspective. Each line of the program, together with its corresponding unit in the instruction register, is grouped into a macro unit (α through π). The state of each macro unit might be defined as the state of its constituent

instruction register unit, abstracting away all the sub-macro interactions within. One might also group the multiplexer units together (ω) and define the macro state as the state of the multiplexer's output. Finally, one might consider each data register as a macro unit ($\rho, \sigma, \varsigma, \nu$) whose state is defined by the state of its output (P', Q', R', S'). This way of macroing results in a clear mapping between macro units $\rho\sigma\varsigma\nu$ and PQRS that captures the functional equivalence of the two systems.

However, the macroing shown in Fig. 5 is inconsistent with IIT's postulates of physical existence. For example, the requirement that macro units be integrated (Fig. 4C–D) means that ω is not a valid unit, as it is fully reducible (for further examples, see “Applying IIT’s postulates to candidate macro units within the computer” supplement). In fact, as shown in the “Macro grain analysis of the strongly connected computer” supplement, there is no macroing of the computer reflecting its function as a simulans that can replicate the cause-effect structure of PQRS. On the basis of IIT, this means that functional equivalence is not accompanied by phenomenal equivalence.

While one example is enough to make the case, we con-

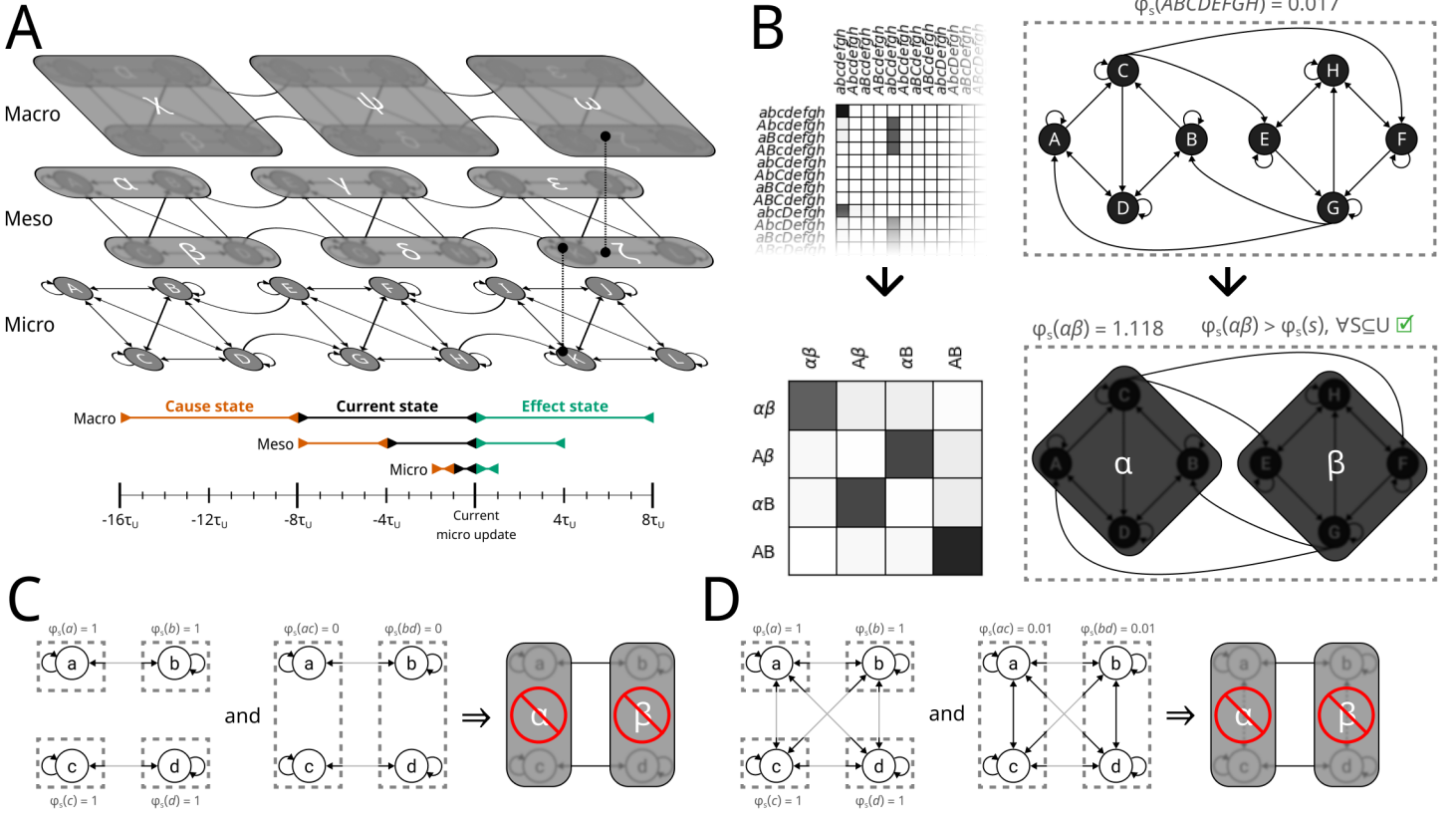


Figure 4: Identifying a system's intrinsic units based on maximally irreducible cause-effect power. (A) Identifying a system's intrinsic units by macroing over units and updates. Macro states can be defined not only over sets of units (top), but also over sets of updates (bottom). Macroing can be performed recursively across "meso" units and updates. (B) An example of a system whose intrinsic cause-effect power is maximal at a macro grain. There are combinatorially many ways to macro the eight micro units $ABCDEFGH$. Shown here is one possible macroing into two macro units $\alpha\beta$ over two updates. The corresponding macro TPM captures the intrinsic causal powers of $\alpha\beta$ at this grain, and is analyzed using the same formalism for micro systems. $\alpha\beta$ in state 11 has system integrated information $\varphi_s = 1.118$ ibits, greater than all of the micro-grain candidate systems (max $\varphi_s = 0.135$ ibits, not shown), including the system $ABCDEFGH$ ($\varphi_s = 0.017$). (C) A complex and its intrinsic units must comply with IIT's postulates of physical existence. Consider a system of four units $abcd$, in which there are no connections between vertical neighbors. Each micro unit has $\varphi_s = 1$ ibits on its own (left), while each pair of vertical neighbors has $\varphi_s = 0$ ibits (middle). Because each pair of vertical neighbors is reducible, they do not satisfy the integration postulate, and are not valid macro units (right). (D) Macro units must also be definite, satisfying the exclusion postulate. This translates into the requirement that a macro unit must be "maximally irreducible within" (have greater φ_s than any subset of its constituents) [34]. Consider $abcd$ when weak connections are introduced between vertical neighbors (left). Although each pair of vertical neighbors is now weakly integrated with $\varphi_s = 0.01$ ibits (middle), they are not maximally irreducible within (e.g., $\varphi_s(ac) < \varphi_s(a)$). Thus, the vertical neighbors are not valid macro elements (right). IIT's postulates of physical existence do not allow for something (a unit) to be built out of nothing (panel C) or nearly nothing (panel D). Adapted from [34].

ture that this result will generalize to many standard computer architectures. The underlying reason is that IIT is concerned with the complete set of causal powers intrinsic to a system, no more (i.e., no powers extrinsic to a mechanism or system may be borrowed, and no intrinsic powers may be multiplied) and no less (i.e., one cannot selectively pick the units or interactions within the system that are of interest to an external observer, and ignore the rest). For the very general computer architecture explored below, macroing of micro units into functionally relevant macro units necessarily involves either omission of, or causal overlap over, a set of shared constituents essential to each unit's function. These two facts together make it extremely unlikely that any similar computer could precisely replicate a target system's cause-effect structure (see "How the computer compares to modern computer architectures" and "Why the computer is unlikely to support a complex with high structure integrated information (Φ)"). Regardless, the point is that, in general, functional equivalence does not imply cause-effect structure equivalence, and therefore does not imply phenomenal equivalence on the basis of IIT.

Cause-effect structures specified by the computer can be dissociated from those of target systems it is simulating.

The above example shows that while a simple computer can be functionally equivalent to a system with a radically different substrate—the complex PQRS—it cannot specify an equivalent cause-effect structure. Owing to its architecture, the computer fragments into many small complexes, each of which specifies a trivial cause-effect structure, rather than the single, much larger cause-effect structure specified by PQRS. We now show that the computer fragments into the same small complexes that specify trivial cause-effect structures regardless of the target system it is simulating.

Consider another four-unit system, $WXYZ$, an elementary cellular automaton that happens to implement Wolfram's Rule 110 (Supplementary Fig. 11) [45]. Cellular automata of any size can be made to implement Rule 110, which completely specifies that system's causal model and TPM. Furthermore, systems of arbitrary, finite size that implement Rule 110 are Turing complete [45, 46]. If placed into state 1101, $WXYZ$ fragments into two complexes, whose cause-effect structures are markedly different from that specified by $pQrS$ (Fig. 6, upper quadrants). The general-purpose four-bit computer described here can simulate $WXYZ$ just as

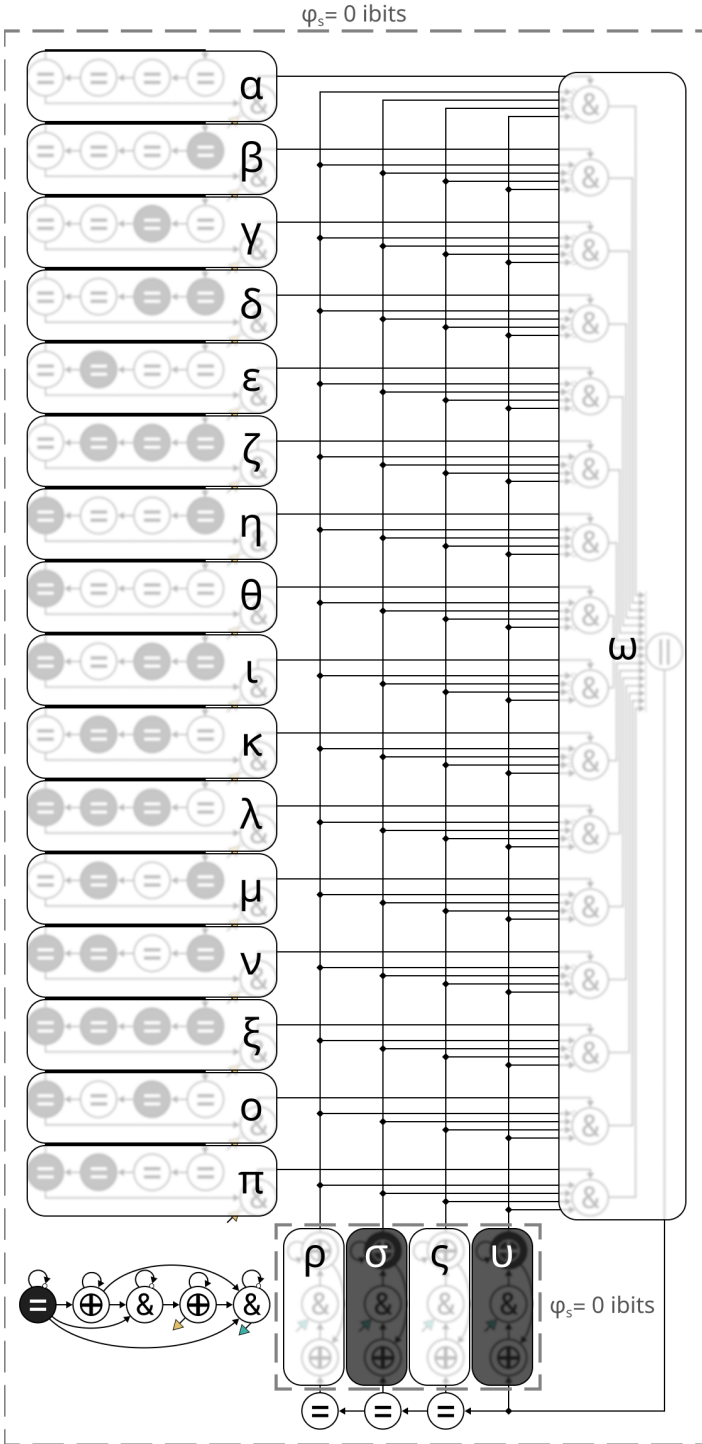


Figure 5: The computer does not replicate the target's cause-effect structure at any macro grain. One might consider macroing the computer as shown: Each line of the stored program and its corresponding instruction register unit is grouped into its own unit (α - π), the multiplexer units are grouped together (ω), and each data register is grouped into its own unit (ρ , σ , ζ , υ). However, not all of these units qualify as intrinsic units (see text). Moreover, integration occurring in the background conditions, extrinsic to a candidate complex, does not contribute to its irreducibility [34]. The functional integration among the units constituting the candidate complex $\rho\sigma\zeta\upsilon$ is entirely mediated by the multiplexer, and must be discounted (see text).

well as $pQrS$. Moreover, analysis of the computer at the micro grain reveals that it again fragments into 24 complexes, specifying similarly trivial cause-effect structures, no matter whether it is simulating $pQrS$, $WXyZ$ (Fig. 6, lower quadrants), or any other four-unit system. This shows that the fragmented cause-effect structures supported by the com-

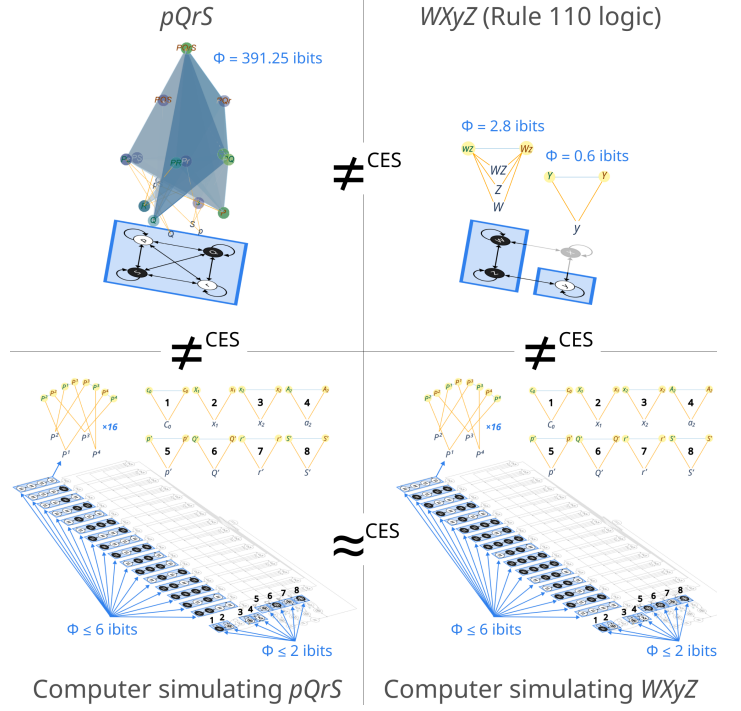


Figure 6: Dissociation between function and cause-effect structure. The single, large cause-effect structure specified by $pQrS$ (top left) and the two, smaller cause-effect structures specified $WXyZ$ (top right) bear little resemblance to each other. However, the fragmented cause-effect structures specified by the computer at the micro grain are insensitive to this difference. In fact, the cause-effect structures supported by the computer as it simulates $pQrS$ (bottom left; see Fig. 3B) are nearly identical to those supported by the computer as it simulates $WXyZ$ (bottom right; see “Analysis of the ring motif” supplement). Thus, the cause-effect structures of the computer and those of the target systems it simulates are not just different, but dissociated.

puter are insensitive to major changes in the cause-effect structure supported by different target systems.

The dissociation between input-output functions and cause-effect structures can be extended by induction to arbitrarily large computers simulating arbitrarily complex behaviors. The computer shown in Fig. 2 is only capable of simulating four-bit/four-unit systems, but none of the qualitative findings above depend on the computer's size—only on its architecture. So long as this architecture is preserved, similar results will hold for arbitrarily large computers.

To increase the capacity of the computer to $n = 2^k$ bits ($k \in \mathbb{N}$) requires increasing the number of program instructions to 2^n , each with length n , appending a frequency divider to the existing clock chain (for a total of $2k + 1$ timekeeping units), increasing the number of data registers to n , extending the multiplexer to have 2^n inputs (one corresponding to each possible state of the system), and extending the buffer (chain of copy units extending from the processor) to $2n - 5$ units. In this way, the expansion process can be continued to create an arbitrarily large computer; an n -bit version of the computer would require $\Theta(n2^n)$ units and $2n$ updates to complete one simulation iteration (Fig. 7A, left).

Like the computer shown in Fig. 2, such a system will have $2^n + n + k + 1$ complexes: one n -unit complex for each line of the program, and one single-unit complex for each unit with a self-loop in the clock chain and data registers. Each complex will continue to specify the same simple cause-effect structures, with the largest having $\Phi \leq \frac{3n}{2}$ ibits (“Analysis

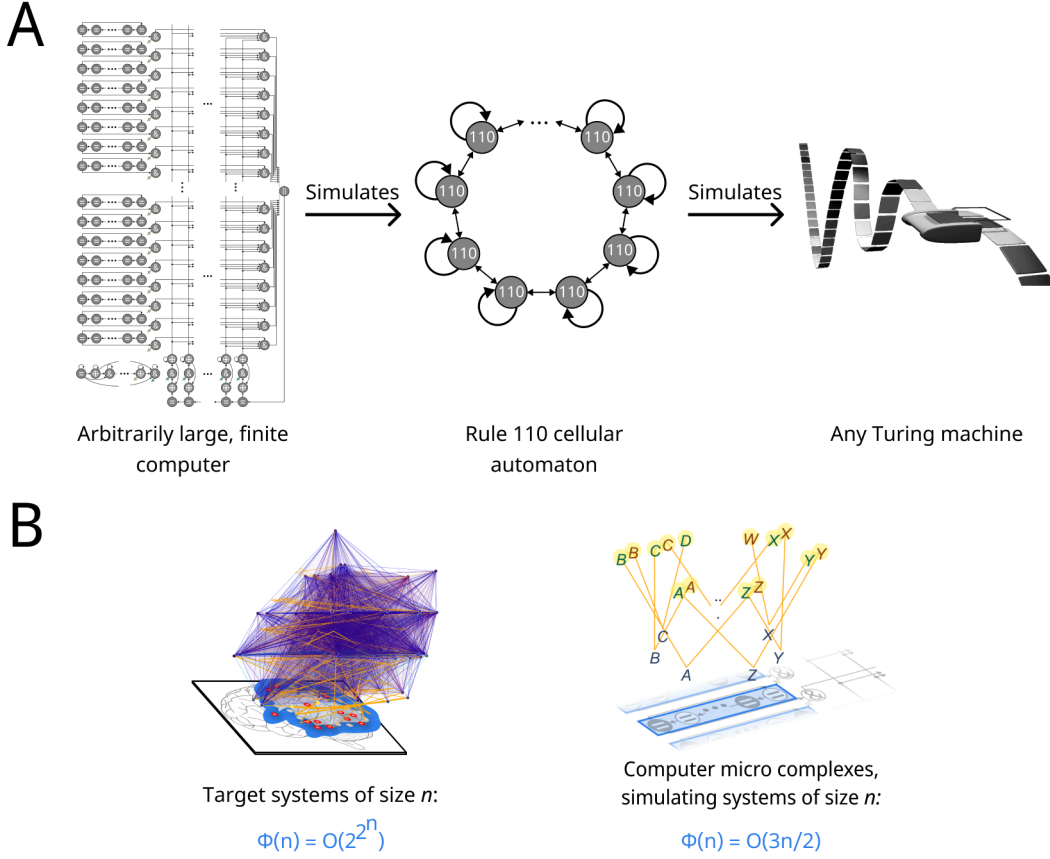


Figure 7: Inductive extension to large computers simulating arbitrarily complex systems. (A) The computer can be incrementally extended to simulate systems of arbitrary size (left), including biological brains or the Rule 110 cellular automaton (middle), which is Turing-complete (right). By transitivity, the programmable computer is Turing-complete too, if extended to arbitrary sizes. (B) By induction on the size of the simulated system, it can be shown that the cause-effect structures specified by subsystems within the computer will not change qualitatively at the micro grain and can only grow linearly with system size (see “Analysis of the ring motif” supplement), whereas the cause-effect structures of the simulated target systems may grow double-exponentially in size and richness, resulting in an increasing phenomenal dissociation between the computer and the systems it simulates.

of the ring motif” supplement). As before, adding feedback connections will not change these results. Again, macroings of the system following from its function as simulans should not be expected to replicate the cause-effect structure(s) of its simulanda, and there will always be a class of systems for which replication is provably impossible.

Such an arbitrarily large computer is capable of simulating arbitrarily complex behaviors. For example, an n -bit version of a digital computer can simulate any n -variable logical system, including all cellular automata. An arbitrarily large elementary cellular automaton implementing Rule 110 is Turing-complete [46], and since the computer is at least as powerful as this cellular automaton, the computer is itself Turing-complete. By the Church–Turing thesis [47], then, our simple computer could perform any computable function.

The dissociation shown above (Fig. 6) between the cause-effect structures specified by the computer at the micro grain and those specified by the target systems it is simulating can be exacerbated if the targets are large and have high Φ . In fact, the magnitude of this dissociation can be a double-exponential function of the size of the simulated system [48]. A system of n units has $2^n - 1$ possible distinctions with $\sum \varphi_d = O(2^n)$ ibits [49]. Only n of these distinctions capture causal constraints exerted by first-order mechanisms; the vast majority of possible distinctions will be higher-order, corresponding to causal constraints exerted by combinations of

units. In turn, the number of possible relations is given by the powerset of distinctions, and is therefore double-exponential as a function of n for rich systems, with $\sum \varphi_r = O(2^{2^n})$ ibits. In other words, the structure integrated information of a rich system can be as high as $\Phi = \sum \varphi_d + \sum \varphi_r = O(2^{2^n})$ ibits [49]. There is no reason to expect that the computer, no matter how large its size and how complex its program, will specify anything beyond small cause-effect structures, each with just first-order mechanisms and linearly increasing Φ (“Why the computer is unlikely to support a complex with high structure integrated information (Φ)”).

Discussion

IIT starts from the essential properties of experience and formulates them in causal, operational terms. This leads to the conclusion that to support consciousness, a system must have cause-effect power that is intrinsic, specific, irreducible, definite (maximally irreducible), and structured. The particular cause-effect structure of distinctions and relations specified by the system accounts for the quality of a particular experience—“what it is like to be” that system in its current state [8]. The amount of structure integrated information (Φ) of the cause-effect structure corresponds to the quantity of consciousness.

Here we applied the mathematical framework of IIT to the causal model of a simple target system, constituted of four units implementing Boolean functions, as well as to an explicit model of a simple, stored-program, four-bit computer, constituted of 117 units and programmed to be functionally equivalent to the target. Although the computer replicated the input-output functions of the target, neither the computer as a whole, nor subsets of its units, could replicate the cause-effect structure of the target. Furthermore, the latter could not be replicated by macroing the units of the computer according to the postulates of IIT. Finally, these results held for a Turing-complete version of the computer which, in principle, can simulate arbitrarily large systems.

The target system employed here was extremely small and specified a single cause-effect structure that was comparatively simple (for the state shown in Fig. 1, composed of 13 distinctions and 8184 relations, with $\Phi = 391.25$ ibits). The functionally equivalent computer, considered at the micro grain, fragmented into 24 small complexes, each of which specified trivial cause-effect structures (with a maximum of 4 distinctions and 4 relations). The substrate of consciousness in the brain can be assumed to be a complex constituted of a much larger number of units that specifies a cause-effect structure of extraordinary richness and high Φ [40, 49]. Based on the principles of IIT and the results obtained here, we conjecture that computers with architectures sufficiently similar to the one shown here will fragment into many small complexes that specify trivial cause-effect structures (see “How the computer compares to modern computer architectures” and “Why the computer is unlikely to support a complex with high structure integrated information (Φ)”). If they were to support some non-trivial cause-effect structures, it would *not* be in virtue of their computational power, functional sophistication, or which simulations they perform. We argue that if standard computers implementing general AI were to replicate our behaviors and cognitive functions—even if they did so by simulating, neuron by neuron, the functioning of our brain—they would generally have negligible consciousness and would not replicate our experiences.

The issue of computer consciousness has been considered before, but not on the basis of a comprehensive, quantitative theory of consciousness. Some have argued that computers are unlikely to be conscious, motivated by various intuitions. For example, consciousness might depend on some critical biological attribute, such as homeostasis, emotion, or embodiment [50–54]. However, it is not clear why such attributes would be critical for consciousness, nor why they would be beyond the reach of computers [55, 56]. Other have suggested that computers may lack some critical “physical-chemical” ingredient, though it is not clear which or why [57, 58].

On the other side of the fence, a common assumption is that consciousness should be identified with some function or computation, whether language, attention, self-monitoring, “global broadcasting” of information, “recurrent processing,” or “predictive processing” [24, 25, 28, 59–61]. For some, consciousness would also require embodiment and interaction with the physical world, including sensing, acting, and autonomy [62, 63], or the replication of some neuronal micro-functions [16]. Functions are multiply realizable, and mental states, if seen as intermediate computational states between

inputs and outputs, can in principle be implemented by both brains and computers. However, there is strong evidence that consciousness can be dissociated from a variety of cognitive functions [22]. We can be conscious even when we are not doing, thinking, or saying anything, as in “pure presence” meditation [64, 65]; in dreaming sleep, when we are not interacting with the environment [66]; under certain psychedelics [67], or under ketamine anesthesia [68].

Moreover, some have argued that computation is an implausible foundation for consciousness precisely because it is substrate-independent and observer-dependent [69–71]. Turing completeness can be achieved using radically different substrates [72–75], whether simple [46, 76] or complex, feedforward [77] or recurrent [78, 79]. Also, any program could in principle be mapped by an observer onto almost any thermodynamically open substrate through an appropriate encoding of inputs and outputs [69, 70] (but see [80, 81]). Why would one assume that a simulation of what something *does* should be equivalent to what something *is*? After all, we do not expect that running a simulation of a rainstorm will make a computer wet [57, 82] or that a simulation of a black hole will bend space-time around the computer. Above all, why should certain functions or computations (but not others) be associated with experience, and why would they feel the way they do (and not some other way)?

On the basis of IIT and in sharp contrast to computational functionalism, we argue that what matters for consciousness, its presence, quality, and quantity, is a system’s intrinsic causal structure—what the system *is*—rather than its extrinsic functions—what the system *does* [18, 20, 83–85]. Because internal organization cannot generally be inferred from overall system dynamics or input-output behavior [18, 86–88], functions are not reliable proxies for the properties of consciousness. In short, consciousness is about being, not doing.

Of course, the conclusions derived in this paper can only be considered valid to the extent that IIT itself can be considered valid. Future experiments may show that, contrary to the predictions of IIT, our own consciousness is not associated with a maximum of intrinsic cause-effect power, or that the quality of specific experiences (say, the experience of visual space) is not consistent with the properties of the associated cause-effect structures, [18, 19, 40, 89]. In that case IIT would be invalidated, and with it the present results concerning machine consciousness. So far, however, IIT provides a rational basis for making inferences about the presence of consciousness in biological or artificial systems other than ourselves. IIT is grounded in the essential properties of our own consciousness—the only case in which we have immediate, irrefutable proof that subjective experience exists. Its predictions can be assessed on our own brain, so far with consistent results [19]. It can explain disparate facts in a coherent manner—for example, why certain brain substrates are important for consciousness while others are not [19, 22], why consciousness vanishes during sleep despite ongoing neural activity [90], whether a stimulus will be perceived or remain unconscious during psychophysical manipulations [30], why the stream of consciousness may split under some concurrent tasks [29], why specific contents of consciousness such as the visual field, temporal flow or mental objects may feel the way they do [40, 41, 91–93], and why consciousness may

have evolved due to selective pressure for information integration [94–96]. Finally, IIT provides a precise mathematical formulation of the requirements for consciousness. Crucial for the present purposes, this formulation can be applied, in principle, whenever an adequate causal model of a system is available, irrespective of what it is made of—neurons or transistors—and irrespective of its functions.

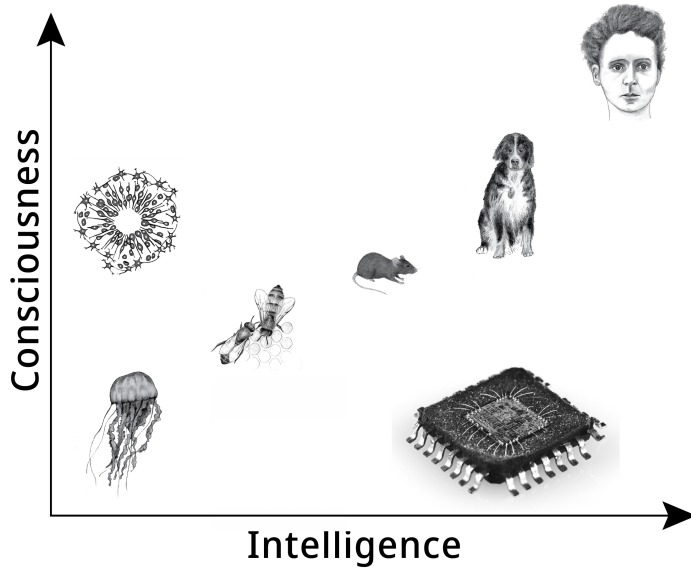


Figure 8: A double dissociation between consciousness and intelligence. In the biological systems to which we are accustomed, consciousness and intelligence are often assumed to go hand in hand. IIT offers a possible explanation for this co-evolution [94]. However, although our instinctive attribution of consciousness based on intelligent behavior may be justified in some cases, it is unlikely to be justified in artificially designed systems—either silicon or organic. We have shown here that IIT implies the distinct possibility of artificial intelligence without consciousness (bottom right). On the other hand, our analysis does not rule out artificial consciousness altogether. An intriguing question is whether truly neuromorphic computers that mimic the physical organization of our brain and do not rely on a CPU could be designed to achieve both functional and phenomenal equivalence to human brains. Similarly, it is an open question whether artificial organic systems, such as cerebral organoids (top left), may lead to forms of consciousness without intelligence. Modified from [97].

An important issue is the extent to which these results can be generalized. The stored-program computer model introduced here captures certain fundamental aspects of modern computer architectures, such as a central processing unit (CPU), registers, a clock, and separation between computation and memory, quite unlike nervous systems. By applying IIT’s postulates to this particular computer-like system, we demonstrated that functional equivalence with the system it is simulating is not accompanied by equivalence in cause–effect structure, and thus by phenomenal equivalence. In principle, one could apply a similar approach to a computer architecture of choice [98]. However, the results obtained here depend primarily on the presence of computational bottlenecks, such as the processor/multiplexer, which force simulated constituents of the target system (say, virtual neurons) to overlap over the same physical substrate (say, a real register). Hence, our findings should extend to a broad class of machines that rely on CPUs or even graphical processing units (GPUs) (see “How the computer compares to modern computer architectures” supplement). They would also extend to brain–machine interfaces that were to replace the connections among neurons by multiplexing the output–input traffic through a CPU; or to replace the input–output transfer function of neurons, again multiplexed through a CPU, while leaving neuronal connec-

tions in place (see “Replication vs. Simulation” supplement). On the other hand, our analysis does not rule out artificial consciousness in other contexts. For example, an intriguing question is whether neuromorphic computers that mimic the physical organization of our brain [99–103] could be designed to achieve both functional and phenomenal equivalence. This would imply that similar experiences can be supported by different kinds of substrates [104]. A related question is whether quantum computers, with entangled qubits, might be engineered to possess the required cause–effect structures and high integrated information [105–107].

The identification of subjective experience with maxima of intrinsic cause–effect power also leads to a reconsideration of the relationship between intelligence and consciousness (Fig. 8) [108]. Historically, we have been accustomed to intelligence and consciousness going hand in hand: to behave intelligently, we typically need to be conscious—in fact, we can hardly do anything when unconscious. Humans have an innate tendency to attribute agency and consciousness to other agents that behave intelligently. If general AI came about, it would probably feel unnatural, not to say narrow-mindedly bio-centric, to deny consciousness to machines that act, speak, and reason like us. We showed previously that IIT offers an explanation for why intelligence and consciousness may tend to co-evolve in biological organisms: selective pressure for intelligence, acting on resource-limited substrates such as brains, typically leads to an increase in integration among the parts of a system [94, 95]. This is seldom the case for engineered systems, for which simplicity and modularity are favored [109]. Thus, while our instinctive attribution of consciousness to biological organisms behaving intelligently may often be justified, in other cases IIT leads to inferences that may contrast with our intuitions and projections. For example, cerebral organoids sharing key anatomical features of brain regions that support consciousness, if kept in a dream-like state, might be vividly conscious despite lacking overt intelligence [110, 111]. Conversely, as shown here, a traditional computer might exhibit human-like behavior and extraordinary intelligence yet experience nothing—it would not see, hear, feel, think, or understand anything. It would only act *as if* it did—it would do everything and be nothing.

ACKNOWLEDGEMENTS. This project was made possible through the support of a grant from Templeton World Charity Foundation (TWCF0216, G.T.). In addition, this research was supported by the David P White Chair in Sleep Medicine at the University of Wisconsin-Madison (G.T.), by the Tiny Blue Dot Foundation (UW 133AAG3451; G.T.), and by the Natural Science and Engineering Research Council of Canada (NSERC; RGPIN-2019-05418; W.M.). L.A. acknowledges the support of a grant from the Templeton World Charity Foundation (TWCF-2020-20526). The funders had no role in study design, data collection and analysis, decision to publish, or preparation of the manuscript.

AUTHOR CONTRIBUTIONS. **Conceptualization:** GF, WM, LA, CK, GT; **Data curation:** ; **Formal analysis:** GF, WM, ID; **Funding acquisition:** GT; **Investigation:** GF, WM, ID; **Methodology:** GF, WM, LA, GT; **Project administration:** GT; **Resources:** GT; **Software:** GF, WM, ID, WGPM; **Supervision:** WM, LA, GT; **Validation:** ; **Visualization:** GF, CK, GT; **Writing – original draft:** GF, WM, GT; **Writing – review & editing:** GF, WM, LA, ID, CK, GT

References

- [1] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. "Deep Learning". In: *Nature* 521.7553 (May 2015), pp. 436–444. ISSN: 1476-4687. DOI: [10.1038/nature14539](#).
- [2] Juergen Schmidhuber. *Annotated History of Modern AI and Deep Learning*. Dec. 2022. DOI: [10.48550/arXiv.2212.11279](#). arXiv: [2212.11279](#).
- [3] Yoshua Bengio et al. "Managing Extreme AI Risks amid Rapid Progress". In: *Science* 384.6698 (May 2024), pp. 842–845. DOI: [10.1126/science.adn0117](#).
- [4] Anthony Brohan et al. *RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control*. July 2023. DOI: [10.48550/arXiv.2307.15818](#). arXiv: [2307.15818](#).
- [5] Aarohi Srivastava et al. *Beyond the Imitation Game: Quantifying and Extrapolating the Capabilities of Language Models*. June 2023. DOI: [10.48550/arXiv.2206.04615](#). arXiv: [2206.04615](#).
- [6] Sébastien Bubeck et al. *Sparks of Artificial General Intelligence: Early Experiments with GPT-4*. Apr. 2023. DOI: [10.48550/arXiv.2303.12712](#). arXiv: [2303.12712](#) [cs].
- [7] Katja Grace et al. *Thousands of AI Authors on the Future of AI*. Apr. 2024. DOI: [10.48550/arXiv.2401.02843](#). arXiv: [2401.02843](#).
- [8] Thomas Nagel. "What Is It Like to Be a Bat?" In: *The Philosophical Review* 83.4 (Oct. 1974), p. 435. ISSN: 00318108. DOI: [10.2307/2183914](#). JSTOR: [2183914](#).
- [9] Robert Long et al. *Taking AI Welfare Seriously*. Nov. 2024. DOI: [10.48550/arXiv.2411.00986](#). arXiv: [2411.00986](#).
- [10] Thomas Metzinger. "Artificial Suffering: An Argument for a Global Moratorium on Synthetic Phenomenology". In: *Journal of Artificial Intelligence and Consciousness* 08.01 (Mar. 2021), pp. 43–66. ISSN: 2705-0785, 2705-0793. DOI: [10.1142/S270507852150003X](#).
- [11] David John Chalmers. *Reality+: Virtual Worlds and the Problems of Philosophy*. First edition. New York, NY: W. W. Norton & Company, 2022. ISBN: 978-0-393-63580-5.
- [12] Henry Shevlin. "Consciousness, Machines, and Moral Status". 2024.
- [13] Alan M. Turing. "Computing Machinery and Intelligence". In: *Mind* 59.October (1950), pp. 433–60.
- [14] Hilary Putnam. "Psychological Predicates". In: *Art, Mind, and Religion*. Ed. by W. H. Capitan and D. D. Merrill. University of Pittsburgh Press, 1967, pp. 37–48.
- [15] Patrick Butlin et al. *Consciousness in Artificial Intelligence: Insights from the Science of Consciousness*. Aug. 2023. DOI: [10.48550/arXiv.2308.08708](#). arXiv: [2308.08708](#) [cs, q-bio].
- [16] Andy Clark. *Microcognition: Philosophy, Cognitive Science, and Parallel Distributed Processing*. Explorations in Cognitive Science 6. Cambridge, Mass: MIT Press, 1989. ISBN: 978-0-262-03148-6.
- [17] Francesco Ellia et al. "Consciousness and the Fallacy of Misplaced Objectivity". In: *Neuroscience of Consciousness* 2021.2 (Dec. 2021), niab032. ISSN: 2057-2107. DOI: [10.1093/nc/niab032](#).
- [18] Larissa Albantakis et al. "Integrated Information Theory (IIT) 4.0: Formulating the Properties of Phenomenal Existence in Physical Terms". In: *PLOS Computational Biology* 19.10 (Oct. 2023), e1011465. ISSN: 1553-7358. DOI: [10.1371/journal.pcbi.1011465](#).
- [19] Giulio Tononi et al. "Integrated Information Theory: From Consciousness to Its Physical Substrate". In: *Nature Reviews Neuroscience* 17.7 (July 2016), pp. 450–461. ISSN: 1471-0048. DOI: [10.1038/nrn.2016.44](#).
- [20] Jeremiah Hendren et al. *Integrated Information Theory Wiki*. <https://www.iit.wiki>. June 2024.
- [21] Victor A. F. Lamme. "How Neuroscience Will Change Our View on Consciousness". In: *Cognitive Neuroscience* 1.3 (Aug. 2010), pp. 204–220. ISSN: 1758-8928. DOI: [10.1080/17588921003731586](#).
- [22] Christof Koch et al. "Neural Correlates of Consciousness: Progress and Problems". In: *Nature Reviews Neuroscience* 17.5 (May 2016), pp. 307–321. ISSN: 1471-0048. DOI: [10.1038/nrn.2016.22](#).
- [23] Francis Crick and Christof Koch. "Towards a Neurobiological Theory of Consciousness". In: *Seminars in the Neurosciences* 2 (1990), pp. 263–275.
- [24] Stanislas Dehaene, Hakwan Lau, and Sid Kouider. "Author Response to: What Is Consciousness, and Could Machines Have It?" In: *Science* 359.6374 (Jan. 2018), pp. 400–402. ISSN: 0036-8075, 1095-9203. DOI: [10.1126/science.aar8639](#).
- [25] Michael S. A. Graziano and Taylor W. Webb. "The Attention Schema Theory: A Mechanistic Account of Subjective Awareness". In: *Frontiers in Psychology* 6 (2015). ISSN: 1664-1078.
- [26] David M. Rosenthal. "Consciousness and Its Function". In: *Neuropsychologia*. Consciousness and Perception: Insights and Hindsight 46.3 (Jan. 2008), pp. 829–840. ISSN: 0028-3932. DOI: [10.1016/j.neuropsychologia.2007.11.012](#).
- [27] Stephen M Fleming. "Awareness as Inference in a Higher-Order State Space". In: *Neuroscience of Consciousness* 2020.1 (Jan. 2020), niz020. ISSN: 2057-2107. DOI: [10.1093/nc/niz020](#).
- [28] Wanja Wiese and Karl J. Friston. "The Neural Correlates of Consciousness under the Free Energy Principle: From Computational Correlates to Computational Explanation". In: *Philosophy and the Mind Sciences* 2 (July 2021). ISSN: 2699-0369. DOI: [10.33735/phimisci.2021.81](#).
- [29] Shuntaro Sasai et al. "Functional Split Brain in a Driving/Listening Paradigm". In: *Proceedings of the National Academy of Sciences* 113.50 (Dec. 2016), pp. 14444–14449. ISSN: 0027-8424, 1091-6490. DOI: [10.1073/pnas.1613200113](#).
- [30] Andrew M. Haun et al. "Conscious Perception as Integrated Information Patterns in Human Electroencephalography". In: *eNeuro* 4.5 (Sept. 2017). ISSN: 2373-2822. DOI: [10.1523/ENEURO.0085-17.2017](#).
- [31] Silvia Casarotto et al. "Stratification of Unresponsive Patients by an Independently Validated Index of Brain Complexity". In: *Annals of Neurology* 80.5 (Nov. 2016), pp. 718–729. ISSN: 1531-8249. DOI: [10.1002/ana.24779](#).
- [32] Brian L. Edlow et al. "Measuring Consciousness in the Intensive Care Unit". In: *Neurocritical Care* 38.3 (June 2023), pp. 584–590. ISSN: 1556-0961. DOI: [10.1007/s12028-023-01706-4](#).
- [33] Matteo Fecchio et al. "Electroencephalographic Responses to Transcranial Magnetic Stimulation Are Sensitive to Fluctuations in Level of Consciousness in Patients with Severe Brain Injuries". In: *Brain Stimulation: Basic, Translational, and Clinical Research in Neuromodulation* 16.1 (Jan. 2023), pp. 375–376. ISSN: 1935-861X, 1876-4754. DOI: [10.1016/j.brs.2023.01.742](#).
- [34] William Marshall et al. "Intrinsic Units: Identifying a System's Causal Grain". In: *bioRxiv* (Sept. 2024), p. 2024.04.12.589163. DOI: [10.1101/2024.04.12.589163](#).
- [35] Warren S. McCulloch and Walter Pitts. "A Logical Calculus of the Ideas Immanent in Nervous Activity". In: *The bulletin of mathematical biophysics* 5.4 (Dec. 1943), pp. 115–133. ISSN: 1522-9602. DOI: [10.1007/BF02478259](#).
- [36] John von Neumann. *The Computer and the Brain*. New Haven, CT: Yale University Press, 1958. ISBN: 978-0-300-08473-3 978-0-300-00793-0 978-0-300-02415-9.
- [37] Leonardo S. Barbosa et al. "A Measure for Intrinsic Information". In: *Scientific Reports* 10.1 (Nov. 2020), p. 18803. ISSN: 2045-2322. DOI: [10.1038/s41598-020-75943-4](#).
- [38] William Marshall, Larissa Albantakis, and Giulio Tononi. "Black-Boxing and Cause-Effect Power". In: *PLOS Computational Biology* 14.4 (Apr. 2018), e1006114. ISSN: 1553-7358. DOI: [10.1371/journal.pcbi.1006114](#).
- [39] Erik P. Hoel et al. "Can the Macro Beat the Micro? Integrated Information across Spatiotemporal Scales". In: *Neuroscience of Consciousness* 2016.1 (Jan. 2016). DOI: [10.1093/nc/niw012](#).
- [40] Andrew Haun and Giulio Tononi. "Why Does Space Feel the Way It Does? Towards a Principled Account of Spatial Experience". In: *Entropy* 21.12 (Dec. 2019), p. 1160. ISSN: 1099-4300. DOI: [10.3390/e21121160](#).
- [41] Renzo Comolatti, Matteo Grasso, and Giulio Tononi. *Why Does Time Feel the Way It Does? Towards a Principled Account of Temporal Experience*. Nov. 2024. DOI: [10.48550/arXiv.2412.13198](#). arXiv: [2412.13198](#) [q-bio].
- [42] William G. P. Mayner et al. "PyPhi: A Toolbox for Integrated Information Theory". In: *PLOS Computational Biology* 14.7 (July 2018), e1006343. ISSN: 1553-7358. DOI: [10.1371/journal.pcbi.1006343](#).
- [43] William Marshall et al. "System Integrated Information". In: *Entropy* 25.2 (Feb. 2023), p. 334. ISSN: 1099-4300. DOI: [10.3390/e25020334](#).
- [44] Renzo Comolatti and Erik Hoel. *Causal Emergence Is Widespread across Measures of Causation*. Feb. 2022. arXiv: [2202.01854](#) [physics].
- [45] Stephen Wolfram. "Statistical Mechanics of Cellular Automata". In: *Reviews of Modern Physics* 55.3 (July 1983), pp. 601–644. DOI: [10.1103/RevModPhys.55.601](#).
- [46] Matthew Cook. "Universality in Elementary Cellular Automata". In: *Complex Systems* (2004), p. 40.
- [47] B. Jack Copeland. "The Church-Turing Thesis". In: *The Stanford Encyclopedia of Philosophy*. Ed. by Edward N. Zalta. The Metaphysics Research Lab, Stanford University, 2008.
- [48] Larissa Albantakis and Giulio Tononi. "The Intrinsic Cause-Effect Power of Discrete Dynamical Systems—From Elementary Cellular Automata to Adapting Animats". In: *Entropy* 17.8 (Aug. 2015), pp. 5472–5502. ISSN: 1099-4300. DOI: [10.3390/e17085472](#).
- [49] Alireza Zaeemzadeh and Giulio Tononi. "Upper Bounds for Integrated Information". In: *PLOS Computational Biology* 20.8 (Aug. 2024), e1012323. ISSN: 1553-7358. DOI: [10.1371/journal.pcbi.1012323](#).
- [50] Jaan Aru, Matthew E. Larkum, and James M. Shine. "The Feasibility of Artificial Consciousness through the Lens of Neuroscience". In: *Trends in Neurosciences* 46.12 (Dec. 2023), pp. 1008–1017. ISSN: 0166-2236. DOI: [10.1016/j.tins.2023.09.009](#).
- [51] Antonio Damasio and Hanna Damasio. "Feelings Are the Source of Consciousness". In: *Neural Computation* 35.3 (Feb. 2023), pp. 277–286. ISSN: 0899-7667. DOI: [10.1162/neco_a_01521](#).

- [52] Hubert L. Dreyfus. *What Computers Still Can't Do: A Critique of Artificial Reason*. Cambridge, Mass: MIT Press, 1992. ISBN: 978-0-262-04134-8 978-0-262-54067-4.
- [53] Kenneth Sayre. "Three More Flaws in the Computational Model". In: *Proceedings and Addresses of the American Philosophical Association* 66.6 (1993), pp. 3–67. ISSN: 0065-972X. JSTOR: [3130671](#).
- [54] Anil Seth. *Conscious Artificial Intelligence and Biological Naturalism*. June 2024. DOI: [10.31234/osf.io/tz6an](#).
- [55] P. M. Churchland and P. S. Churchland. "Could a Machine Think?" In: *Scientific American* 262.1 (Jan. 1990), pp. 32–37. ISSN: 0036-8733. DOI: [10.1038/scientificamerican0190-32](#).
- [56] S. Pinker. *How the Mind Works (1997/2009)*. 2009th ed. New York, NY: W. W. Norton & Company, 2009.
- [57] John R. Searle. "Minds, Brains and Programs". In: *Behavioral and Brain Sciences* 3.3 (1980), pp. 417–57.
- [58] Roger Penrose. *The Emperor's New Mind: Concerning Computers, Minds, and the Laws of Physics*. Oxford ; New York: Oxford University Press, 1989. ISBN: 978-0-19-851973-7.
- [59] Jerry A. Fodor. *The Language of Thought*. The Language & Thought Series. Cambridge, Mass: Harvard Univ. Press, 1979. ISBN: 978-0-674-51030-2.
- [60] Lenore Blum and Manuel Blum. "A Theory of Consciousness from a Theoretical Computer Science Perspective: Insights from the Conscious Turing Machine". In: *Proceedings of the National Academy of Sciences* 119.21 (May 2022), e2115934119. DOI: [10.1073/pnas.2115934119](#).
- [61] John Brockman et al., eds. *What to Think about Machines That Think: Today's Leading Thinkers on the Age of Machine Intelligence*. First edition. Edge Question Series. New York: Harper Perennial, 2015. ISBN: 978-0-06-242565-2.
- [62] Jesse Prinz. "Is Consciousness Embodied". In: *The Cambridge Handbook of Situated Cognition*. Ed. by Murat Aydede and P. Robbins. Cambridge: Cambridge University Press, 2009, pp. 419–437.
- [63] Giovanni Pezzulo and Paul Cisek. "Navigating the Affordance Landscape: Feedback Control as a Process Model of Behavior and Cognition". In: *Trends in Cognitive Sciences* 20.6 (June 2016), pp. 414–424. ISSN: 1364-6613. DOI: [10.1016/j.tics.2016.03.013](#).
- [64] Robert K. C. Forman, ed. *The Problem of Pure Consciousness: Mysticism and Philosophy*. 1st published. New York: Oxford Univ. Press, 1997. ISBN: 978-0-19-510976-4.
- [65] Melanie Boly et al. "Neural Correlates of Pure Presence". In: *bioRxiv* (Jan. 2024), p. 2024.04.18.590081. DOI: [10.1101/2024.04.18.590081](#).
- [66] Francesca Siclari et al. "The Neural Correlates of Dreaming". In: *Nature Neuroscience* 20.6 (June 2017), pp. 872–878. ISSN: 1546-1726. DOI: [10.1038/nn.4545](#).
- [67] Christopher Timmermann et al. "Human Brain Effects of DMT Assessed via EEG-fMRI". In: *Proceedings of the National Academy of Sciences* 120.13 (Mar. 2023), e2218949120. DOI: [10.1073/pnas.2218949120](#).
- [68] Simone Sarasso et al. "Consciousness and Complexity during Unresponsiveness Induced by Propofol, Xenon, and Ketamine". In: *Current Biology* 25.23 (Dec. 2015), pp. 3099–3105. ISSN: 0960-9822. DOI: [10.1016/j.cub.2015.10.014](#).
- [69] James Blackmon. "Searle's Wall". In: *Erkenntnis* 78.1 (Feb. 2013), pp. 109–117. ISSN: 1572-8420. DOI: [10.1007/s10670-012-9405-4](#).
- [70] Hilary Putnam. *Representation and Reality*. MIT Press, 1987.
- [71] Romain Brette. "Is Coding a Relevant Metaphor for the Brain?" In: *Behavioral and Brain Sciences* 42 (Jan. 2019), e215. ISSN: 0140-525X, 1469-1825. DOI: [10.1017/S0140525X19000049](#).
- [72] A. M. Turing. "Computability and λ -Definability". In: *The Journal of Symbolic Logic* 2.4 (Dec. 1937), pp. 153–163. ISSN: 0022-4812, 1943-5886. DOI: [10.2307/2268280](#).
- [73] Edward F. Moore. "Gedanken-Experiments on Sequential Machines". In: *Automata Studies (AM-34)*. Ed. by C. E. Shannon and J. McCarthy. Annals of Mathematical Studies 34. Princeton University Press, 1956, pp. 129–153. ISBN: 978-1-4008-8261-8.
- [74] Noam Chomsky. "On Certain Formal Properties of Grammars". In: *Information and Control* 2.2 (June 1959), pp. 137–167. ISSN: 0019-9958. DOI: [10.1016/S0019-9958\(59\)90362-6](#).
- [75] M. O. Rabin and D. Scott. "Finite Automata and Their Decision Problems". In: *IBM Journal of Research and Development* 3.2 (Apr. 1959), pp. 114–125. ISSN: 0018-8646. DOI: [10.1147/rd.32.0114](#).
- [76] M. Schönfinkel. "Über die Bausteine der mathematischen Logik". In: *Mathematische Annalen* 92.3 (Sept. 1924), pp. 305–316. ISSN: 1432-1807. DOI: [10.1007/BF01448013](#).
- [77] Kurt Hornik. "Approximation Capabilities of Multilayer Feedforward Networks". In: *Neural Networks* 4.2 (Jan. 1991), pp. 251–257. ISSN: 0893-6080. DOI: [10.1016/0893-6080\(91\)90009-T](#).
- [78] H. T. Siegelmann and E. D. Sontag. "On the Computational Power of Neural Nets". In: *Journal of Computer and System Sciences* 50.1 (Feb. 1995), pp. 132–150. ISSN: 0022-0000. DOI: [10.1006/jcss.1995.1013](#).
- [79] Kenneth Krohn and John Rhodes. "Algebraic Theory of Machines. I. Prime Decomposition Theorem for Finite Semigroups and Machines". In: *Transactions of the American Mathematical Society* 116 (1965), pp. 450–464. ISSN: 0002-9947.
- [80] David J. Chalmers. "Does a Rock Implement Every Finite-State Automaton?" In: *Synthese* 108.3 (Sept. 1996), pp. 309–333. ISSN: 0039-7857, 1573-0964. DOI: [10.1007/BF00413692](#).
- [81] Derek Shiller. "Functionalism, Integrity, and Digital Consciousness". In: *Synthese* 203.2 (Jan. 2024), p. 47. ISSN: 1573-0964. DOI: [10.1007/s11229-023-04473-z](#).
- [82] Ned Block. "Troubles with Functionalism". In: *Minnesota Studies in the Philosophy of Science* 9 (1978), pp. 261–325.
- [83] G. Tononi and C. Koch. "Consciousness: Here, There and Everywhere?" In: *Philosophical Transactions of the Royal Society B: Biological Sciences* 370.1668 (Mar. 2015), pp. 20140167–20140167. ISSN: 0962-8436, 1471-2970. DOI: [10.1098/rstb.2014.0167](#).
- [84] Alireza Zaeemzadeh and Giulio Tononi. *Shannon Information and Integrated Information: Message and Meaning*. Dec. 2024. DOI: [10.48550/arXiv.2412.10626](#). arXiv: [2412.10626 \[q-bio\]](#).
- [85] Giulio Tononi et al. *Only What Exists Can Cause: An Intrinsic View of Free Will*. Mar. 2023. DOI: [10.48550/arXiv.2206.02069](#). arXiv: [2206.02069 \[q-bio\]](#).
- [86] Masafumi Oizumi, Larissa Albantakis, and Giulio Tononi. "From the Phenomenology to the Mechanisms of Consciousness: Integrated Information Theory 3.0". In: *PLoS Computational Biology* 10.5 (May 2014). Ed. by Olaf Sporns, e1003588. ISSN: 1553-7358. DOI: [10.1371/journal.pcbi.1003588](#).
- [87] Larissa Albantakis and Giulio Tononi. "Causal Composition: Structural Differences among Dynamically Equivalent Systems". In: *Entropy* 21.10 (Oct. 2019), p. 989. ISSN: 1099-4300. DOI: [10.3390/e21100989](#).
- [88] Matteo Grasso et al. "Causal Reductionism and Causal Structures". In: *Nature Neuroscience* 24.10 (Oct. 2021), pp. 1348–1355. ISSN: 1546-1726. DOI: [10.1038/s41593-021-00911-8](#).
- [89] Giulio Tononi. "Integrated Information Theory". In: *Scholarpedia* 10.1 (2015), p. 4164. ISSN: 1941-6016. DOI: [10.4249/scholarpedia.4164](#).
- [90] Marcello Massimini et al. "Breakdown of Cortical Effective Connectivity During Sleep". In: *Science* 309.5744 (Sept. 2005), pp. 2228–2232. DOI: [10.1126/science.1117256](#).
- [91] Matteo Grasso, Andrew M Haun, and Giulio Tononi. "Of Maps and Grids". In: *Neuroscience of Consciousness* 2021.2 (Dec. 2021), niab022. ISSN: 2057-2107. DOI: [10.1093/nc/niab022](#).
- [92] Andrew Haun and Giulio Tononi. *The Unfathomable Richness of Seeing*. Apr. 2024. DOI: [10.31234/osf.io/jmg35](#).
- [93] David Balduzzi and Giulio Tononi. "Qualia: The Geometry of Integrated Information". In: *PLoS Computational Biology* 5.8 (Aug. 2009), e1000462. ISSN: 1553-7358. DOI: [10.1371/journal.pcbi.1000462](#).
- [94] Larissa Albantakis et al. "Evolution of Integrated Causal Structures in Animals Exposed to Environments of Increasing Complexity". In: *PLoS Computational Biology* 10.12 (Dec. 2014). Ed. by Daniel Polani, e1003966. ISSN: 1553-7358. DOI: [10.1371/journal.pcbi.1003966](#).
- [95] Jeffrey A. Edlund et al. "Integrated Information Increases with Fitness in the Evolution of Animals". In: *PLoS Computational Biology* 7.10 (Oct. 2011), e1002236. ISSN: 1553-7358. DOI: [10.1371/journal.pcbi.1002236](#).
- [96] William G. P. Mayner, Bjørn Erik Juel, and Giulio Tononi. *Intrinsic Meaning, Perception, and Matching*. Dec. 2024. DOI: [10.48550/arXiv.2412.21111](#). arXiv: [2412.21111 \[q-bio\]](#).
- [97] Christof Koch. *The Feeling of Life Itself: Why Consciousness Is Widespread but Can't Be Computed*. Cambridge, MA: MIT Press, 2019. ISBN: 978-0-262-04281-9 978-0-262-53955-5.
- [98] Justin T. Sampson. "Integrated Information Theory of Consciousness in Conventional Computing". PhD thesis. San Francisco State University, 2024. DOI: [10.46569/hq37vw99k](#).
- [99] Donhee Ham et al. "Neuromorphic Electronics Based on Copying and Pasting the Brain". In: *Nature Electronics* 4.9 (Sept. 2021), pp. 635–644. ISSN: 2520-1131. DOI: [10.1038/s41928-021-00646-1](#).
- [100] Alexantrou Serb et al. "Memristive Synapses Connect Brain and Silicon Spiking Neurons". In: *Scientific Reports* 10.1 (Feb. 2020), p. 2590. ISSN: 2045-2322. DOI: [10.1038/s41598-020-58831-9](#).
- [101] J. Schemmel et al. "A Wafer-Scale Neuromorphic Hardware System for Large-Scale Neural Modeling". In: *Proceedings of 2010 IEEE International Symposium on Circuits and Systems*. May 2010, pp. 1947–1950. DOI: [10.1109/ISCAS.2010.5536970](#).
- [102] C. Mead. "Neuromorphic Electronic Systems". In: *Proceedings of the IEEE* 78.10 (Oct. 1990), pp. 1629–1636. ISSN: 1558-2256. DOI: [10.1109/5.58356](#).
- [103] Aaron R. Young et al. "A Review of Spiking Neuromorphic Hardware Communication Systems". In: *IEEE Access* 7 (2019), pp. 135606–135620. ISSN: 2169-3536. DOI: [10.1109/ACCESS.2019.2941772](#).

- [104] Hilary Putnam. "The Nature of Mental States". In: *Art, Mind, and Religion*. Ed. by W. H. Capitan and D. D. Merrill. Pittsburgh University Press, 1967, pp. 1–223.
- [105] Larissa Albantakis, Robert Prentner, and Ian Durham. "Computing the Integrated Information of a Quantum Mechanism". In: *Entropy* 25.3 (Mar. 2023), p. 449. ISSN: 1099-4300. DOI: [10.3390/e25030449](https://doi.org/10.3390/e25030449).
- [106] Johannes Kleiner and Sean Tull. "The Mathematical Structure of Integrated Information Theory". In: *Frontiers in Applied Mathematics and Statistics* 6 (June 2021). ISSN: 2297-4687. DOI: [10.3389/fams.2020.602973](https://doi.org/10.3389/fams.2020.602973).
- [107] Paolo Zanardi, Michael Tomka, and Lorenzo Campos Venuti. *Towards Quantum Integrated Information Theory*. Dec. 2018. DOI: [10.48550/arXiv.1806.01421](https://doi.org/10.48550/arXiv.1806.01421). arXiv: [1806.01421](https://arxiv.org/abs/1806.01421).
- [108] Christof Koch and Giulio Tononi. "Can We Quantify Machine Consciousness?" In: *IEEE Spectrum* 54.6 (June 2017), pp. 64–69. DOI: [10.1109/MSPEC.2017.7934235](https://doi.org/10.1109/MSPEC.2017.7934235).
- [109] Herbert A. Simon. *The Sciences of the Artificial, Third Edition*. MIT Press, Sept. 1996. ISBN: 978-0-262-26449-5.
- [110] Fikri Birey et al. "Assembly of Functionally Integrated Human Forebrain Spheroids". In: *Nature* 545.7652 (May 2017), pp. 54–59. ISSN: 1476-4687. DOI: [10.1038/nature22330](https://doi.org/10.1038/nature22330).
- [111] Nita A. Farahany et al. "The Ethics of Experimenting with Human Brain Tissue". In: *Nature* 556.7702 (Apr. 2018), p. 429. DOI: [10.1038/d41586-018-04813-x](https://doi.org/10.1038/d41586-018-04813-x).
- [112] Judea Pearl. *Causality: Models, Reasoning, and Inference*. Cambridge, U.K. ; New York: Cambridge University Press, 2000. ISBN: 978-0-521-89560-6 978-0-521-77362-1.
- [113] Wesley C. Salmon. *Statistical Explanation and Statistical Relevance*. University of Pittsburgh Press, Sept. 1971. ISBN: 978-0-8229-7411-6 978-0-8229-5225-1. DOI: [10.2307/j.ctt6wr9p](https://doi.org/10.2307/j.ctt6wr9p). JSTOR: [10.2307/j.ctt6wr9p](https://www.jstor.org/stable/10.2307/j.ctt6wr9p).
- [114] Robert Brandon. "The Levels of Selection". In: *PSA: Proceedings of the Biennial Meeting of the Philosophy of Science Association 1982* (1982), pp. 315–323.
- [115] Hans Reichenbach and Maria Reichenbach. *The Direction of Time*. Dover Books on Physics. Mineola, N.Y.: Dover, 1999. ISBN: 978-0-486-40926-9.
- [116] Graham Findlay et al. "Dissociating Intelligence from Consciousness in Artificial Systems – Implications of Integrated Information Theory". In: *Towards Conscious AI Systems*. Vol. 2287. ISSN 1613-0073. Stanford, CA: CEUR Workshop Proceedings, Mar. 2019.
- [117] David J. Chalmers. "Absent Qualia, Fading Qualia, Dancing Qualia". In: *Conscious Experience*. Ed. by Thomas Metzinger. Ferdinand Schoningh, 1995, pp. 309–328.

Supplementary Materials

Step-by-step guide to the computer's operation.

Let U be the four-bit computer, which simulates any system of four Boolean units. The computer is composed of synchronously updating discrete units, each of which implements an elementary logic function. These units are organized into four main modules: the clock and frequency dividers, the program and instruction registers, the data registers, and the multiplexer/processing unit (Figure 12). The data registers store the state of the system until instructed to update; the program memory stores the function of the units that the computer is simulating; the processing unit combines this information about state and function to compute the next state of the data registers; the clock synchronizes these processes and tells the registers when to update their state. When initialized appropriately, the computer simulates the behavior of any four-unit Boolean network indefinitely. The system being simulated is referred to as the *target system*.

A reference implementation with precise details of every unit and connection can be found at https://github.com/CSC-UW/Findlay_et_al_2024a. A diagram of the computer with all units labeled is shown in Fig. 12. Briefly, the four-bit computer has the following components:

- Four data registers $\mathbf{R} = \{R_1, R_2, R_3, R_4\}$, ordered from left to right. Each register R_i contains 3 units: $R_i = \{R_i^{\text{SIM}}, R_i^{\text{AND}}, R_i^{\text{XOR}}\}$. In the main text, R_1^{SIM} , R_2^{SIM} , R_3^{SIM} , and R_4^{SIM} are referred to as P', Q', R', and S', respectively.
- Sixteen program lines $\mathbf{P} = \{P_1, P_2, \dots, P_{16}\}$, ordered from top to bottom. Each program line is a ring of 4 COPY units: $P_i = \{P_i^1, P_i^2, P_i^3, P_i^4\}$.
- An instruction register $\mathbf{IR} = \{IR_1, IR_2, \dots, IR_{16}\}$, consisting of sixteen AND units, ordered from top to bottom.
- A multiplexer $\mathbf{M} = \{M_1, M_2, \dots, M_{16}, MO\}$, consisting of sixteen AND units, ordered from top to bottom, and a generalized OR unit (MO).
- A timekeeping chain $\mathbf{T} = \{C_0, X_1, A_1, X_2, A_2\}$, consisting of the core oscillator C_0 and two frequency dividers.
- A buffer $\mathbf{B} = \{B_1, B_2, B_3\}$, consisting of three COPY units between the multiplexer's output (MO) and the leftmost register R_1 .

Once initialized (see below), the computer takes eight updates (nine transitions) to simulate a single state transition of the target system. After these eight updates, which we refer to as a single *simulation iteration*, the states of units R_1^{SIM} , R_2^{SIM} , R_3^{SIM} , and R_4^{SIM} (also referred to as P'Q'R'S', see main text Figure 2) update synchronously to reflect the state of the target system. The target and the computer are functionally equivalent in the sense that there exists a constant factor C (here, $C = 8$) such that the state of the target at update i (in its own reference frame) is equal to the state of $R_1^{\text{SIM}}R_2^{\text{SIM}}R_3^{\text{SIM}}R_4^{\text{SIM}}$ at all updates $t \in (iC, (i+1)C]$ (in the computer's reference frame).

We now walk through the initialization procedure and an entire simulation iteration, as the computer simulates PQRS's transition from state 0101 to state 1110.

Update 0: Initialization. Figure 13 shows the initialized computer. First, the initial states of units R_1^{SIM} , R_2^{SIM} , R_3^{SIM} , and R_4^{SIM} are set to reflect the initial state of the target (0101). We set the initial state of R_1^{SIM} to OFF (0/white), the initial state of R_2^{SIM} to ON (1/black), the initial state of R_3^{SIM} to OFF, and the initial state of R_4^{SIM} to ON (main text Figure 2C). The other units in the data registers are set to OFF.

Next, the state of the clock is set so that the COPY unit (C_0) is ON, and all other units (X_1 , A_1 , X_2 , and A_2) are OFF. C_0 with its negated self-loop forms the oscillating core of the clock; its state will flip every update. X_1 and A_1 form a frequency divider. After the first update X_1 will be ON for two updates, then OFF for two updates, and will repeat this pattern indefinitely (i.e., X_1 has a 50% duty cycle and a period of four updates). A_1 will turn ON for the last one of every four updates (i.e., a 25% duty cycle and a period of four updates). X_2 and A_2 form a second frequency divider, and have the same duty cycles as their upstream counterparts, and the same initial phase, but double their periods. X_2 is OFF for four updates, then ON for four updates. A_2 is OFF for seven updates, then ON for one update. For example, the state of each unit over the first 16 updates is shown in Table 1.

Next, the states of the computer's program units are set in such a way that they encode the target's state transition rules (main text Figure 2B). The program has 16 lines, each implemented by a ring of four COPY units. The top ring encodes the state that PQRS would transition to from state 0000. The second ring from the top encodes the state that PQRS would transition to from state 1000. The third from 0100, the fourth from 1100, the fifth from 0010, the sixth from 1010, the seventh from 0110, the eighth from 1110, and so on (in little-endian like fashion, with leftmost bits changing faster than rightmost bits). The leftmost unit in each ring is set to the state that P would transition to, the second from the left is set

Update	C ₀	X ₁	A ₁	X ₂	A ₂
0	1	0	0	0	0
1	0	1	0	0	0
2	1	1	0	0	0
3	0	0	1	0	0
4	1	0	0	1	0
5	0	1	0	1	0
6	1	1	0	1	0
7	0	0	1	1	1
8	1	0	0	0	0
9	0	1	0	0	0
10	1	1	0	0	0
11	0	0	1	0	0
12	1	0	0	1	0
13	0	1	0	1	0
14	1	1	0	1	0
15	0	0	1	1	1

Table 1: The state of each timekeeping unit over the course of the first 16 updates

to the state that Q would transition to, the third from the left is set to the state that R would transition to, and the fourth from the left is set to the state that S would transition to. Finally, the instruction register units (**IR**) are initialized OFF.

Because we want to program the computer with the information that PQRS in state 0101 transitions to state 1110, we need to set the units in the eleventh ring from the top to ON, ON, ON, OFF, and ON when read from left to right. If we would like the computer’s simulation to continue indefinitely, we should also initialize the other fifteen program instructions (main text Figure 2B).

Transition from update 0 to update 1. Figure 14 shows the first of nine transitions involved in a single simulation iteration. During this step, the computer fetches the data that it needs to compute the future state of P (equivalently, R_1^{SIM}). The two important elements of this calculation are the simulated current state of PQRS and, given that state, the rule describing what P would do.

Recall that the states of the computer’s program units were set in such a way that they encoded the target’s state transition rules. The leftmost unit in each program line was initially set to represent a post-transition state of P, with each line of the program representing a different pre-transition state of PQRS. At update 1, the program and instruction registers have updated so that the initial states of these leftmost units are now the states of the instruction register units. Thus, each instruction register unit is set to a post-transition state of P, with each unit representing a different pre-transition state of PQRS.

The states of R_1^{SIM} , R_2^{SIM} , R_3^{SIM} , and R_4^{SIM} —which reflect the simulated current state of PQRS—have not changed (by design, no R_i^{SIM} will change unless its respective R_i^{AND} is ON). The clock updates (Table 1).

Transition from update 1 to update 2. Figure 15 shows the second of nine transitions involved in a single simulation iteration. During this step, the next state of P is computed, and the computer becomes ready to compute the next state of Q.

Each AND unit in the multiplexer receives four inputs from the data registers and one input from the instruction register. We say that one of these AND units is enabled at update t when all four of its inputs (after any negation) from the data registers were ON at update $t - 1$. Because no two multiplexer units receive the same four inputs from the data registers, at most one AND unit in the multiplexer can be enabled at any given update. In fact, the state of the data register outputs at any update t enables exactly one multiplexer unit at update $t + 1$.

Invariably, the topmost multiplexer unit will be enabled when the data register outputs are in state 0000. The second multiplexer unit from the top will be enabled when the data register outputs are in state 1000. The third in 0100, the fourth in 1100, the fifth in 0011, the sixth in 1010, the seventh in 0110, and the eighth in 1110, and so on. Notice how this mirrors the way in which the program memory is structured. When a particular AND unit in the multiplexer is enabled, its next state is determined by the content of the corresponding instruction register unit. This is the means by which we combine the current state of the target (represented by which multiplexer unit is enabled) with the desired behavior of the target in that state (represented by the corresponding instruction register unit).

At update 1, the eleventh multiplexer unit from the top (M_{11}) was enabled. Whether it turns ON is determined by the eleventh line of the program ($P_{11} = \{P_{11}^0, P_{11}^1, P_{11}^2, P_{11}^3\}$) and instruction register (IR_{11}), which says what PQRS would do in state 0101. More precisely, the eleventh line’s instruction register unit (IR_{11}) at update 1 was in the same state that P

would transition to when PQRS is in state 0101. All together, this means that at update 2, the eleventh multiplexer unit (M_{11}) will be ON if and only if the next state of P is ON. All other AND units in the multiplexer will be OFF.

To reiterate, there are two possibilities at update 2: either exactly one AND unit in the multiplexer is ON, in which case P should transition to ON in our simulation, or exactly no AND units in the multiplexer are ON, in which case P should transition to OFF in our simulation. At this point we say that the next state of P is computed, since an extrinsic observer of the system can tell what the next state of P should be without looking ahead into the computer's future (i.e., at updates $t > 2$).

As all of this is happening, the computer is also becomes ready to compute the next state of Q. The state of the data register outputs has not changed since they were initialized. This means that the same multiplexer unit which was enabled at update 1 (M_{11}) will still be enabled at update 2, and the same program line (P_{11}) and instruction register (IR_{11} , though its output will of course change) will determine that multiplexer unit's state. Finally, the instruction register updates so that each unit is set to represent a post-transition state of Q, again with each line representing a different pre-transition state of PQRS.

The clock updates (Table 1).

Transition from update 2 to update 3. Figure 16 shows the third of nine transitions involved in a single simulation iteration. During this step, the next state of P propagates towards the registers, the next state of Q is computed, and the computer becomes ready to compute R.

At update 2, either exactly one AND unit in the multiplexer was ON, in which case P would transition to ON in our simulation, or exactly no AND units in the multiplexer were ON, in which case P would transition to OFF in our simulation. The multiplexer's generalized OR unit (MO, which turns ON whenever one of its 16 inputs is ON and is OFF otherwise) at update 3 is therefore ON if the next state of P is ON, and OFF if the next state of P is OFF. In our example, the OR unit is ON.

The next state of Q is now computed exactly as the next state of P was. Because Q transitions to ON when PQRS is 0101, exactly one AND unit in the multiplexer is ON.

The state of the data register outputs has not changed. This means that the same AND unit (M_{11}) which was enabled at update 2 will still be enabled at update 3, and the same program line (P_{11}) and instruction register (IR_{11} , though its output will of course change) will determine that multiplexer unit's state. Finally, the instruction register updates so that each unit is set to represent a post-transition state of R, again with each line representing a different pre-transition state of PQRS.

The clock updates (Table 1).

Transition from update 3 to update 4. Figure 17 shows the fourth of nine transitions involved in a simulation iteration. During this step, P and Q's next state propagate towards the registers, the next state of R is computed, and the computer becomes ready to compute S.

At update 3, the multiplexer's OR unit (MO) represented the next state of P, and was therefore ON. At update 4 the buffer unit which the OR outputs to (B_3) will therefore be ON.

The multiplexer's OR now updates to reflect the next state of Q, just as it did for P at update 3. In our case, the OR is ON.

The next state of R is now computed exactly as the next state of P and Q were. Because R transitions to ON when PQRS is 0101, exactly one AND unit in the multiplexer (M_{11}) is ON.

The state of the data register outputs has not changed. This means that the same AND unit which was enabled at update 3 (M_{11}) will still be enabled at update 4, and the same program line (P_{11}) and instruction register (IR_{11} , though its output will of course change) will determine that multiplexer unit's state. Finally, the instruction register updates so that each unit is set to represent a post-transition state of S, again with each line representing a different pre-transition state of PQRS.

The clock updates (Table 1). Unit X_2 , which has been OFF since initialization, is now ON. This disables the instruction register.

Transition from update 4 to update 5. Figure 18 shows the fifth of nine transitions involved in a simulation iteration. During this step, P, Q, and R's next state propagate towards the registers, and the next state of S is computed.

At update 4, the COPY unit immediately downstream of the multiplexer (B_3) represented the next state of P, and was therefore ON. At update 5 this state will propagate to the next unit in the buffer chain (B_2). Similarly, at update 4, MO represented the next state of Q, and was therefore ON. At update 5, this state will propagate to B_3 .

MO now updates to reflect the next state of R, just as it did for Q at update 4. In our case, MO is ON.

The next state of S is now computed exactly as the next state of P, Q, and R were. Because S transitions to OFF when PQRS is 0101, exactly zero AND units in the multiplexer will be ON at update 5.

The clock updates (Table 1).

Transition from update 5 to update 6. Figure 19 shows the sixth of nine transitions involved in a simulation iteration.

At update 6, the inputs to each data register's R^{XOR} unit (i.e., B_1 to R_1^{XOR} , B_2 to R_2^{XOR} , B_3 to R_3^{XOR} , and MO to R_4^{XOR}) will hold that register's future state. In our case, this means that B_1 , B_2 , B_3 , and MO will be ON, ON, ON, and OFF, respectively.

The clock updates (Table 1).

Transition from update 6 to update 7. Figure 20 shows the seventh of nine transitions involved in a simulation iteration. During this step, the registers determine whether their outputs need to change state.

Each register's lower XOR unit (R_1^{XOR} , R_2^{XOR} , R_3^{XOR} , R_4^{XOR}) receives two inputs: one from its upper XOR (R_1^{SIM} , R_2^{SIM} , R_3^{SIM} , R_4^{SIM} respectively), which holds the simulated current state of P, Q, R, or S, and one from another unit which holds the simulated next state of P, Q, R, or S (B_1 , B_2 , B_3 , MO respectively). The XOR performs a "same-or-different" operation on these two inputs, effectively deciding whether a register's currently stored value is the same as its desired future value, or different. This is the same as deciding whether or not a register's current value needs to change.

In our case, we are simulating a transition from state 0101 to state 1110. The state of each register's lower XOR is therefore ON ("change"), OFF ("no change"), and ON ("change"), and OFF ("change") respectively.

The clock updates (Table 1). A_2 is now ON, which enables the AND units in each data register (R_1^{AND} , R_2^{AND} , R_3^{AND} , R_4^{AND}).

Transition from update 7 to update 8. Figure 21 shows the eighth of nine transitions involved in a simulation iteration. During the step, each register's "change" signal is allowed to influence its stored value.

Normally, A_2 is OFF. As a consequence, the data registers' AND units (R_1^{AND} , R_2^{AND} , R_3^{AND} , R_4^{AND}) are usually disabled. At update 8, however, the A_2 was ON. This means that each register's AND unit was enabled, and is now ON if and only if its register's lower XOR was ON at update 7. In our case, these AND units are ON, OFF, ON, and ON respectively. The "change" signals from update 7 have thus propagated to these AND units at update 8, where they are able to influence the what the states of the data register outputs will be at update 9.

The clock updates (Table 1). A_2 returns to being OFF. X_2 also switches OFF, which enables the instruction register to preemptively begin loading values for the next simulation iteration.

Transition from update 8 to update 9. Figure 22 shows the last transition involved in a simulation iteration. During this transition, the states of R_1^{SIM} , R_2^{SIM} , R_3^{SIM} , and R_4^{SIM} update synchronously to become 1110, and the next simulation iteration begins.

R_1^{SIM} , R_2^{SIM} , R_3^{SIM} , and R_4^{SIM} each receive two inputs: one from itself, which carries its own current state, and one from the AND unit which determines whether that state needs to change. If the XOR's current state is ON and it gets a "change" signal (ON), then it turns OFF. If the XOR's current state is ON and it gets a "no change" signal (OFF), then it remains ON. If the XOR's current state is OFF and it gets a "change" signal (ON), then it turns ON. If the XOR's current state is OFF and it gets a "no change" signal (OFF), then it remains OFF. Because the AND units are normally all OFF ("no change"), the XORs normally hold their state. At update 8, however, some of these AND units may have been ON, and therefore the data register outputs at update 9 have changed their state, if necessary.

In our case, the data register outputs have become 1110. Our first simulation iteration is complete, and the next one has begun. The process continues indefinitely.

Results

All notation and terminology below follows [18] and [34], which should be consulted for details of IIT’s framework and formalism. In keeping with [18] and [34], log is always the base 2 logarithm. Results are organized as follows:

General notation, definitions, and results	19
Analysis of the ring motif	20
Analysis of the monad motif	24
Analysis of the computer	24
Micro grain analysis of the weakly connected computer	25
Micro-grain analysis of the strongly connected computer	26
Macro grain analysis of the weakly and strongly connected computers	29

The definition and results below build up to three key claims concerning (1) the potential complexes and bounds on Φ in the weakly connected computer at the micro grain (see Claim 17); (2) the potential complexes and bounds on Φ in the strongly connected computer at the micro grain (see Claim 23); and (3) the inability of function-relevant macroings to replicate the cause–effect structure of the target system (see Claim 24).

General notation, definitions, and results.

Definition 1 (Micro connectivity). For a model $U = (U_1, U_2, \dots, U_n)$ with transition probabilities $p(u \mid \bar{u})$ for $u, \bar{u} \in \Omega_U$, the micro connectivity matrix, denoted $\mathcal{A}^\mu(U)$, is an $n \times n$ matrix where the (i, j) cell (a_{ij}^μ) indicates whether U_i has an effect on U_j . Specifically, $a_{ij}^\mu = 1$ if there exists $u, \bar{u}$ and $\tilde{u}$ with $\bar{u}_k = \tilde{u}_k$ for $k \neq i$ and $\bar{u}_i \neq \tilde{u}_i$ such that $p(u_j \mid \bar{u}) \neq p(u_j \mid \tilde{u})$. Essentially, there is a directed connection from U_i to U_j if the current state of U_i has an effect on U_j , for any fixed state of the rest of the units.

Definition 2 (Paths). For a model $U = (U_1, U_2, \dots, U_n)$ with micro connectivity matrix $\mathcal{A}^\mu(U)$ a path from U_a to U_b is a sequence of units $\wp(U_a, U_b) = (U_{c_1}, U_{c_2}, \dots, U_{c_m})$ such that:

$$U_{c_1} = U_a, U_{c_m} = U_b, a_{c_i c_{i+1}}^\mu = 1 \quad \forall i = 1, \dots, m-1.$$

Definition 3 (Generalized connectivity). A unit J connects to a unit K (denoted $J \rightarrow K$) if there exists a micro path between units J and K , consisting entirely of micro units that are either constituents or apportionments of J . In other words

$$J \rightarrow K \iff \exists \wp^\mu(U_a, U_b) : \wp(U_a, U_b) \setminus U_b \subseteq U^J \cup W^J \wedge U_a \in U^J \wedge U_b \in U^K.$$

Definition 4 (Generalized connectivity matrix). For a system $S = (J_1, J_2, \dots, J_{|S|})$ the generalized connectivity matrix of S , denoted $\mathcal{A}(S)$, is an $|S| \times |S|$ matrix where the (i, j) cell (a_{ik}) indicates whether J_i connects to J_k ,

$$a_{ik} = \begin{cases} 1 & \text{if } J_i \rightarrow J_k \\ 0 & \text{otherwise.} \end{cases}$$

Definition 5 (Cut matrix). For a system $S = (J_1, J_2, \dots, J_{|S|})$ and partition $\theta \in \Theta(S)$, the cut matrix $\mathcal{X}(\theta, S)$, is an $|S| \times |S|$ matrix where the (i, k) cell (x_{ik}) indicates whether θ cuts the potential connection from J_i to J_k . Let $S^{(i)}, S^{(k)} \in \theta$ be the parts containing J_i and J_k respectively, then

$$x_{ik} = \begin{cases} 0 & \text{if } S_{\leftarrow}^{(i)} \in \theta \text{ or } S_{\rightarrow}^{(k)} \in \theta \text{ or } S^{(i)} = S^{(k)} \\ 1 & \text{otherwise.} \end{cases}$$

Definition 6 (Γ -notation). Given a system $S = (S_1, S_2, \dots, S_n)$ with generalized connectivity matrix $\mathcal{A}(S)$, let $\Gamma_S^{\leftarrow}(S_i) \subseteq S$ be unit S_i ’s *system predecessors*—the set of units that act upon S_i , with all background units removed:

$$\Gamma_S^{\leftarrow}(S_i) = \{S_j : S_j \in S, a_{ji} = 1\}.$$

Similarly, let $\Gamma_S^{\rightarrow}(S_i)$ denote S_i ’s *system successors*:

$$\Gamma_S^{\rightarrow}(S_i) = \{S_j : S_j \in S, a_{ij} = 1\}.$$

Finally, we can overload these operators to accept as arguments a set of units V . For example, let $\Gamma_S^{\leftarrow}(V) \subseteq S$ be the *system predecessor union* of V within S :

$$\Gamma_S^{\leftarrow}(V) = \bigcup_{V_i \in V} \Gamma_S^{\leftarrow}(V_i).$$

Claim 1. *Systems that are not strongly connected are reducible.*

Proof. Let S be a system in any state $s \in \Omega_S$. If S is not strongly connected, then there exists $F \subset S$ such that $\Gamma_S^{\leftarrow}(F) \cap (S \setminus F) = \emptyset$. Consider a partition $\theta = \{F^{\leftarrow}, (S \setminus F)^{\rightarrow}\} \in \Theta(S)$, where we noise all incoming connections from $S \setminus F$ to F . Since there are no incoming connections to F from $S \setminus F$, the system's transition probabilities remain the same before and after partitioning (i.e., $\mathcal{T}_e = \mathcal{T}_e^\theta$ and $\mathcal{T}_c = \mathcal{T}_c^\theta$; see Eqns. 16-18 in [18]). Therefore, $\theta' = \{F^{\leftarrow}, S \setminus F^{\rightarrow}\}$ is a minimum partition, and the system is reducible: $\varphi_s(s, \theta') = 0$. ■

Definition 7 (Externally determined units). Let $S = (S_1, S_2, \dots, S_m)$ be a system in state s , and $W = U \setminus U^S$ be background conditions in state w . A unit $s_i \in S$ is *externally determined* if:

$$p_e(\bar{s}_i | s) = p_e(\bar{s}_i), \forall \bar{s}, s \in \Omega_S \quad \vee \quad p_c(s_i | \bar{s}) = p_c(s_i), \forall \bar{s}, s \in \Omega_S.$$

Essentially, a unit is externally determined if either its current or future state are fully determined by the background conditions of the system. Note that whether a unit is externally determined depends on the state of the background units w , but not on the system state s .

An example is a deterministic AND unit S_i receiving input from a background unit which is 0. It does not matter what s is: the causal marginalization of w , conditional on u , guarantees that $\bar{s}_i = 0$.

Another example is a deterministic OR unit S_i receiving input from a background unit which is 1. It does not matter what s is: the causal marginalization of w , conditional on u , guarantees that $\bar{s}_i = 1$.

These examples illustrate that it is not the actual value (0 or 1) of the next state $\bar{s}_i$ that defines whether or not a unit is externally determined—it is about whether anything within the system can affect that state.

Claim 2. *Systems with externally determined units are reducible.*

Proof. Let S be a system in any state $s \in \Omega_S$ with background conditions w . Let $S_i \in S$ be externally determined, then consider a partition of S where the first part consists of the externally determined unit with its inputs cut, and the second part is its complement:

$$\theta = \{\{S_i\}^{\leftarrow}, S \setminus \{S_i\}^{\rightarrow}\}$$

The partitioned probability matrix is unaffected by this partition since the probability distribution of S_i does not depend on the state of S (either $\mathcal{T}_c = \mathcal{T}_c^\theta$ or $\mathcal{T}_e = \mathcal{T}_e^\theta$, depending on whether the current or future state is determined; see Eqns. 16-18 in [18]) and thus

$$\min\{\varphi_c(s, \theta), \varphi_e(s, \theta)\} = 0.$$

Therefore, $\theta' = \{\{S_i\}^{\leftarrow}, S \setminus \{S_i\}^{\rightarrow}\}$ is a minimum partition, and the system is reducible: $\varphi_s(s, \theta') = 0$. ■

Claim 3. *For any system $S = (S_1, S_2, \dots, S_n)$, if unit S_i has a single system successor S_j , and unit S_j has a single system predecessor S_i , then neither S_i nor S_j can contribute to higher-order mechanisms.*

Proof. Assume S_i has a single system successor S_j (i.e., $\Gamma_S^{\rightarrow}(S_i) = S_j$) and S_j has a single system predecessor S_i (i.e., $\Gamma_S^{\leftarrow}(S_j) = S_i$). Consider a higher-order mechanism $M \subseteq S$ in any state and a potential purview $Z \subseteq S$. If $S_i \in M$, we can define a disintegrating partition $\theta \in \Theta(M, Z)$ that cuts no connections from M to Z . If $S_j \in Z$, then we can define

$$\theta = \{(S_i, S_j), (M \setminus S_i, Z \setminus S_j)\}$$

or if $S_j \notin Z$, then we can define

$$\theta = \{(S_i, \emptyset), (M \setminus S_i, Z)\}.$$

In either case, we have a partition θ that does not cut any connections from M to Z , and so $\pi_e(z | m) = \pi_e(z)$ for all $z \in \Omega_Z$ and $m \in \Omega_M$, and thus

$$\varphi_e(m) = \varphi_e(m, Z, \theta) = \pi_e(z | m) \left| \log \frac{\pi_e(z | m)}{\pi_e(z)} \right|_+ = 0,$$

and M is reducible. Note that the above partition is only valid for $|M| > 1$, and thus it is still possible that $M = S_i$ forms a first-order mechanism.

Similarly, if $S_j \in M$, we can define a partition $\theta \in \Theta(M, Z)$ that cuts no connections from Z to M . If $S_i \in Z$, then we can define

$$\theta = \{(S_j, S_i), (M \setminus S_j, Z \setminus S_i)\},$$

or if $S_i \notin Z$, then we can define

$$\theta = \{(S_j, \emptyset), (M \setminus S_j, Z)\}.$$

In either case, we have a partition θ that does not cut any connections from Z to M , and so $\pi_c(z | m) = \pi_c(z)$ for all $z \in \Omega_Z$ and $m \in \Omega_M$, and thus

$$\varphi_c(m) = \varphi_c(m, Z, \theta) = \pi_c^{\leftarrow}(z | m) \left| \log \frac{\pi_c(z | m)}{\pi_c(z)} \right|_+ = 0,$$

and M is reducible. ■

Analysis of the ring motif.

Definition 8 (Rings). A system $S = (S_1, S_2, \dots, S_n)$ is a ring if and only if all of the following conditions hold:

1. S is strongly connected
2. For each $S_k \in S$, there exists a unique $i \neq k$ such that $\Gamma_S^{\leftarrow}(S_k) = S_i$

Claim 4. Let S be a ring in state s . The cause-effect structure of s contains at most $|S|$ distinctions, each of which is a first order distinction with $\varphi_d \leq 1$.

Proof. Let $m = s_j \in s$ be a potential first order mechanism. Because S is a ring, S_j has a single input $\Gamma_S^{\leftarrow}(S_j)$ and a single output $\Gamma_S^{\rightarrow}(S_j)$. If s_j does not specify the cause state of its input, then the ring can be cut $\theta = \{\{S_j\}_{\leftarrow}, (S \setminus S_j)_{\rightarrow}\}$ and $\varphi_s(s, \theta) = 0$. A similar argument applies to the effect state of its output. Therefore, if $\varphi_s(s) > 0$, then s_j specifies the cause state of $\Gamma_S^{\leftarrow}(S_j)$ and the effect state of $\Gamma_S^{\rightarrow}(S_j)$. Since S_j has no other inputs or outputs, the only possible purviews are $Z_c^* = \Gamma_S^{\leftarrow}(S_j)$ and $Z_e^* = \Gamma_S^{\rightarrow}(S_j)$. Thus, the distinction is

$$d(m) = (m = s_j, z_c^* = \Gamma_S^{\leftarrow}(s_j), z_e^* = \Gamma_S^{\rightarrow}(s_j), \varphi_d(m)).$$

As demonstrated in [49], since $|\Gamma_S^{\leftarrow}(S_j)| = |\Gamma_S^{\rightarrow}(S_j)| = 1$,

$$\varphi_d(m) \leq 1.$$

Also note that the information specified by the system as a whole about each unit's cause and effect is congruent with the information specified by each unit about its maximal cause-effect state. Therefore, all $|S|$ first-order distinctions exist in the cause-effect structure.

Finally, no higher-order distinctions exist. This is because no units in the ring have both shared inputs and shared outputs (see Claim 3). Therefore, the set of distinctions is exactly the set of first order distinctions just described. ■

Claim 5. Let S be a ring in state s with $\varphi_s(s) > 0$, and let $|S| \geq 4$. The cause-effect structure of s contains at most $|S|$ unique sets of distinctions that are related and

$$\sum_d \varphi_r(d) \leq \frac{|S|}{2}.$$

Proof. Let $D(s)$ be the set of distinctions in the cause-effect structure (see Claim 4), $\mathbf{d} \subseteq D(s)$ be a set of distinctions that may be related, $d \in D(s)$ be a single distinction, and $M(d)$ the mechanism of $d \in D(s)$.

First, consider $|S| = 4$ as a unique case. By direct inspection of the potential distinctions, there are at most two possible relations, each with up to two faces and

$$\begin{aligned} \varphi_r(\mathbf{d}) &= \min_{\mathbf{d} \in \mathbf{d}} \left\| \bigcup_{f \in \mathbf{f}(\mathbf{d})} o_f^* \right\| \frac{\varphi_d(d)}{|z_c^*(d) \cup z_e^*(d)|} \\ &= \min_{\mathbf{d} \in \mathbf{d}} (2) \frac{\varphi_d(d)}{2} \\ &= \min_{\mathbf{d} \in \mathbf{d}} \varphi_d(d) \leq 1, \end{aligned}$$

and thus $\sum_d \varphi_r(\mathbf{d}) \leq 2 = |S|/2$.

Next, consider $|S| > 4$. Because of the ring connectivity of S , there are guaranteed to be exactly $|S|$ pairs of units that are “nearly neighbors” (i.e., separated from each other in the ring by a single, intervening unit). Call the set of the corresponding distinction pairs NN:

$$\text{NN} = \{d_i, d_j \in D(s) : \Gamma_S^{\rightarrow}(M(d_i)) = \Gamma_S^{\leftarrow}(M(d_j))\}$$

For each of these pairs, the cause purview of d_j will overlap with the effect purview of d_i . If $z_c^*(d_j) = z_e^*(d_i)$, the cause and effect are congruent, then $\mathbf{d} = \{d_i, d_j\}$ constitutes a 2nd-degree relation (because 2 distinctions, d_i and d_j , are bound) with a 2nd-degree face (because 2 purviews, $z_c^*(d_j)$ and $z_e^*(d_i)$, are involved).

Whether or not all $|S|$ of the possible relations $r(\mathbf{d})$, $\mathbf{d} \in \text{NN}$ exist depends on whether all pairs of distinctions $\{d_i, d_j\} \in \text{NN}$ agree on the state of their overlapping purview, i.e., on whether $z_c^*(d_j) = z_e^*(d_i)$. This will generally not be the case, so $|S|$ is an upper bound on the possible number of relations. For each relation $r(\mathbf{d})$ that exists, $\varphi_r \leq 0.5$,

$$\begin{aligned} \varphi_r(\mathbf{d}) &= \min_{\mathbf{d} \in \mathbf{d}} \left\| \bigcup_{f \in \mathbf{f}(\mathbf{d})} o_f^* \right\| \frac{\varphi_d(d)}{|z_c^*(d) \cup z_e^*(d)|} \\ &= \min_{\mathbf{d} \in \mathbf{d}} (1) \frac{\varphi_d(d)}{2} \\ &\leq \frac{1}{2} \end{aligned}$$

No other relations are possible (e.g., 3rd-degree relations, 4th-degree relations, etc.), because there are no higher-order purviews. A candidate relation involving 3 or more distinctions will have non-overlapping purviews. Thus

$$\sum \varphi_r \leq \sum \frac{1}{2} \leq \frac{|S|}{2}.$$

■

Claim 6. *Let S be a ring in state s with $\varphi_s(s) > 0$, and let $|S| \geq 4$. The structure integrated information $\Phi(s)$ is bounded above by $\frac{3}{2}|S|$ ibits.*

Proof. $\Phi(s)$ is the sum of the values of integrated information of a substrate's distinctions and relations (see Eqn. 59 in [18]).

Per Claim 4, distinctions will always contribute a maximum of $|S|$ ibits to $\Phi(s)$:

$$\sum_{d \in D(s)} \varphi_d \leq \sum_{d \in D(s)} 1 = |S| \text{ ibits.}$$

Per Claim 5, the contribution of relations to $\Phi(s)$ will be maximized when s is homogenous (i.e., all 0 or all 1), so that all maximal cause-effect states are congruent with each other. In this case, relations will contribute at most $\frac{|S|}{2}$ ibits to $\Phi(s)$:

$$\sum_{d \in \text{NN}} \varphi_r(d) \leq \sum_{d \in \text{NN}} \frac{1}{2} = \frac{1}{2}|S| \text{ ibits.}$$

for a total of:

$$\Phi(s) = \sum_{d \in D(s)} \varphi_d + \sum_{d \in \text{NN}} \varphi_r(d) \leq |S| + \frac{1}{2}|S| = \frac{3}{2}|S| \text{ ibits.}$$

■

Definition 9 (Imperfect Ring). An imperfect ring $S = S' \cup S_n$ is defined by starting from a ring $S' = (S_1, S_2, \dots, S_{n-1})$ and modifying it in a specific way. Start with a ring S' with $|S'| = n - 1$ and add an additional unit S_n such that S_n outputs to S_1 and receives input from S_{n-2} :

$$\Gamma_S^{\rightarrow}(S_n) = S_1, \Gamma_S^{\leftarrow}(S_n) = S_{n-2}.$$

The modification has the following impact on the existing ring units:

$$\Gamma_S^{\leftarrow}(S_1) = \{S_{n-1}, S_n\}, \Gamma_S^{\rightarrow}(S_{n-2}) = \{S_{n-1}, S_n\}.$$

Claim 7. *Let S' be a ring with $|S'| = n - 1$, and $S = S' \cup S_n$ an imperfect ring in state s . The cause-effect structure of s contains at most $|S| + 1$ distinctions, $|S|$ potential first-order distinctions, and one potential second-order distinction, each with $\varphi_d \leq 1$.*

Proof. The first-order distinctions specified by S are those potentially specified by S' , with two possible differences: The distinction specified by s_1 now has the potential for a larger cause purview, $Z_c^*(m) \subseteq \{s_{n-1}, s_n\}$, and the distinction specified by s_{n-2} now has the potential for a larger effect purview, $Z_e^*(s_{n-2}) \subseteq \{s_{n-1}, s_n\}$. However, regardless of whether this actually occurs, and regardless of whether $m = s_1$ or $m = s_{n-2}$:

$$\min(|Z_c^*(m)|, |Z_e^*(m)|) = 1.$$

This means that, for both $m = s_1$ and $m = s_{n-2}$, $\varphi_d(m) \leq 1$.

It remains to check for potential distinctions whose mechanism includes S_n . For the first order $m = s_n$, the potential cause purview is $Z_c^* = s_{n-2}$ and the potential effect purview is $Z_e^* = s_1$, and thus $\varphi_d(m) \leq 1$.

Next, S_{n-1} and S_n have common input (S_{n-2}) and common output (S_1), so they can potential specify a higher order mechanism. If $m = \{s_{n-1}, s_n\}$ then the potential cause purview is $Z_c^* = s_{n-2}$ and the potential effect purview is $Z_e^* = s_1$, and thus $\varphi_d(m) \leq 1$.

Finally, no other higher-order distinctions exist. This is because no sets of units in the imperfect ring have both shared inputs and shared outputs (see Claim 3). ■

Claim 8. *Let S' be a ring with $|S'| = n - 1 \geq 4$ and $S = S' \cup S_n$ an imperfect ring in state s . The cause-effect structure $C(s)$ contains at most $|S| + 15$ unique sets of distinctions relating distinctions in $D(s)$ and*

$$\sum_d \varphi_r(d) \leq \frac{|S| - 2}{2} + \frac{64}{6}.$$

Proof. Let $D(s)$ be the set of distinctions in the cause-effect structure (see Claim 7), $\mathbf{d} \subseteq D(s)$ be a set of distinctions that may be related, $d \in D(s)$ be a single distinction, and $M(d)$ the mechanism of $d \in D(s)$.

First, consider $|S'| = 4$ as a unique case. By direct inspection of the potential distinctions, there are at most 11 possible relations, each with up to two faces and $\varphi_r \leq 1$. Thus,

$$\sum_{\mathbf{d}} \varphi_r(\mathbf{d}) \leq 11 < \frac{|S| - 2}{2} + \frac{64}{6} = 12.17.$$

Next, consider $|S'| > 4$. Consider the distinctions in $D(s')$. Those distinctions are also in $D(s)$, with the exception being a potentially larger cause purview for $m = s_1$ and a potentially larger effect purview for $m = s_{n-2}$. Thus, all relations specified in Claim 4 are also relations among distinctions in $D(s)$, except for the relation among $\mathbf{d} = \{d(m = s_1), d(m = s_{n-2})\}$. That is, there are potentially $n - 2$ 2nd-degree relations with a single 2nd-degree face and $\varphi_r \leq 0.5$.

Any further relations must include either the cause of $d(m = s_1)$, the effect of $d(m = s_{n-2})$, or one of the new distinctions $d(m = s_n)$, $d(m = \{s_{n-1}, s_n\})$. The possible relations for such a small set of distinctions can be computed explicitly, and are summarized in Table 2. The faces and overlap can be extracted from the necessary features of the distinctions, and then the maximum possible value of φ_r is obtained based on the maximum possible φ_d of the corresponding distinctions.

Combined, there are at most $|S| - 2$ relations contributing at most $\varphi_r = \frac{1}{2}$ each, plus the φ_r of the relations listed in Table 2:

$$\sum_{\mathbf{d}} \varphi_r(\mathbf{d}) \leq \frac{|S| - 2}{2} + \frac{64}{6} \text{ ibits.}$$

■

$\mathbf{d}$	faces	overlap	max φ_r
$d(s_1), d(s_{n-2})$	cause-effect	$\{s_{n-1}, s_n\}$	2/3
$d(s_2), d(s_n)$	cause-effect	s_1	1/2
$d(s_n), d(s_{n-3})$	cause-effect	s_{n-2}	1/2
$d(s_2), d(\{s_{n-1}, s_n\})$	cause-effect	s_1	1/2
$d(\{s_{n-1}, s_n\}), d(s_{n-3})$	cause-effect	s_{n-2}	1/2
$d(s_{n-1}), d(s_n)$	cause-cause effect-effect	s_{n-2} s_1	1
$d(\{s_{n-1}, s_n\}), d(s_{n-1})$	cause-cause effect-effect	s_{n-2} s_1	1
$d(s_n), d(\{s_{n-1}, s_n\})$	cause-cause effect-effect	s_{n-2} s_1	1
$d(s_2), d(s_n), d(\{s_{n-1}, s_n\})$	cause-effect-effect	s_1	1/2
$d(s_2), d(s_n), d(s_{n-1})$	cause-effect-effect	s_1	1/2
$d(s_2), d(\{s_{n-1}, s_n\}), d(s_{n-1})$	cause-effect-effect	s_1	1/2
$d(s_n), d(\{s_{n-1}, s_n\}), d(s_{n-3})$	cause-cause-effect	s_{n-2}	1/2
$d(s_n), d(s_{n-1}), d(s_{n-3})$	cause-cause-effect	s_{n-2}	1/2
$d(s_{n-1}), d(\{s_{n-1}, s_n\}), d(s_{n-3})$	cause-cause-effect	s_{n-2}	1/2
$d(s_{n-1}), d(s_n), d(\{s_{n-1}, s_n\})$	cause-cause-cause effect-effect-effect	s_{n-2} s_1	1
$d(s_2), d(s_{n-1}), d(s_n), d(\{s_{n-1}, s_n\})$	cause-effect-effect-effect	s_1	1/2
$d(s_{n-1}), d(s_n), d(\{s_{n-1}, s_n\}), d(s_{n-3})$	cause-cause-cause-effect	s_{n-2}	1/2

Table 2: Potential relations involving S_n for an imperfect ring of at least five units.

Claim 9. Let S' be a ring with $|S'| = n - 1$ and $S = S' \cup S_n$ be an imperfect ring in state s , and let $|S| \geq 5$. The structure integrated information $\Phi(s)$ is bounded above by

$$\frac{9n + 64}{6} \text{ ibits.}$$

Proof. $\Phi(s)$ is the sum of the values of integrated information of a substrate's distinctions and relations (see Eqn. 59 in [18]).

Per Claim 7, distinctions will always contribute a maximum of $|S|$ ibits to $\Phi(s)$:

$$\sum_{d \in D(s)} \varphi_d \leq \sum_{d \in D(s)} 1 \leq |S| + 1 \text{ ibits.}$$

Per Claim 8, there are at most $n - 2$ relations contributing at most $\varphi_r = \frac{1}{2}$ each, plus the φ_r of the relations listed in Table 2:

$$\sum_{\mathbf{d}} \varphi_r(\mathbf{d}) \leq \frac{n-2}{2} + \frac{64}{6} \text{ ibits.}$$

for a total of:

$$\Phi(s) = \sum_{d \in D(s)} \varphi_d + \sum_{\mathbf{d}} \varphi_r(\mathbf{d}) \leq (n+1) + \frac{n-2}{2} + \frac{64}{6} = \frac{9n+64}{6} \text{ ibits.}$$

■

Analysis of the monad motif.

Definition 10 (Monad). A system S is a monad if and only if all of the following conditions hold:

1. $|S| = 1$ (S is a single unit).
2. $S = \Gamma_S^{\leftarrow}(S) = \Gamma_S^{\rightarrow}(S)$.

Claim 10. For a monad S in state s with $\varphi_s(s) > 0$, the cause-effect structure of s has the following properties:

- Exactly one distinction, of 1st order, with $\varphi_d \leq 1$ ibit.
- At most one relation, of 1st-degree with a 2nd-degree face, and $\varphi_r = \varphi_d$ ibits.
- $\Phi(s) \leq 2$ ibits.

Proof. Because S is a monad it can have at most a single, first order distinction d , with mechanism m . As demonstrated in [49], since $|\Gamma_S^{\leftarrow}(m)| = |\Gamma_S^{\rightarrow}(m)| = 1$,

$$\varphi_d(m) \leq 1.$$

Because S is a monad, $m = S$, and the information specified by the system as a whole about m 's cause and effect is necessarily congruent with the information specified by m about its maximal cause-effect state. Therefore, m must exist in the cause-effect structure.

The cause-effect structure of S may contain no relations, for example if d specifies a cause state that is incongruent with its effect state. If relations are present, then there can be at most a single relation r between $\mathbf{d} = \{d\}$ (r must be of 1st-degree, with a 2nd-degree face). Furthermore, it is trivially true that the union of unique purview units (see Eqn. 53 of [18]) and the relation purview (see Eqn. 54 of [18]) of r cannot be more than a single unit. By Eqn. 55 in [18],

$$\max \varphi_r(\mathbf{d}) = \varphi_d(m)$$

So, if r exists at all, $\Phi(s) = \varphi_d(m) + \varphi_r(\mathbf{d}) = 2\varphi_d(m)$. It follows from the upper bound on $\varphi_d(m)$ that $\Phi(s) \leq 2$ ibits. ■

Analysis of the computer.

Definition 11 (The weakly connected computer). Let $\text{WCC}(k)$, $k \in \{2, 3, 4, \dots\}$ be the k^{th} weakly connected computer, which simulates any system of $n = 2^k$ boolean units. A reference implementation with precise details of every unit and connection in $\text{WCC}(k)$ can be found at https://github.com/CSC-UW/Findlay_et_al_2024a. The 2nd (i.e., four-bit) weakly connected computer is shown in Fig. 12. Briefly, $\text{WCC}(k)$ has the following components:

- $n = 2^k$ data registers $\mathbf{R} = \{R_1, R_2, \dots, R_n\}$, ordered from left to right. Each register R_i contains 3 units: $R_i = \{R_i^{\text{SIM}}, R_i^{\text{AND}}, R_i^{\text{XOR}}\}$. As useful shorthand, let $\mathbf{R}^{\text{SIM}} = \{R_i^{\text{SIM}}\}_{i=1}^n$, since we often need to refer to this set of units.
- 2^n program lines $\mathbf{P} = \{P_1, P_2, \dots, P_{2^n}\}$, ordered from top to bottom. Each program line is a ring of n COPY units: $P_i = \{P_i^1, P_i^2, \dots, P_i^n\}$.
- An instruction register $\mathbf{IR} = \{IR_1, IR_2, \dots, IR_{2^n}\}$, consisting of 2^n AND units, ordered from top to bottom.
- A multiplexer $\mathbf{M} = \{M_1, M_2, \dots, M_{2^n}, \text{MO}\}$, consisting of 2^n AND units, ordered from top to bottom, and a generalized OR unit (MO).
- A timekeeping chain $\mathbf{T} = \{C_0, X_1, A_1, X_2, A_2, \dots, X_k, A_k\}$, consisting of the core oscillator C_0 and k frequency dividers $\{X_i, A_i\}$.
- A buffer $\mathbf{B} = \{B_1, B_2, \dots, B_{2n-5}\}$, consisting of $2n - 5$ COPY units between the multiplexer's output (MO) and the leftmost register R_1 .

Altogether, $\text{WCC}(k) = \mathbf{R} \cup \mathbf{P} \cup \mathbf{IR} \cup \mathbf{M} \cup \mathbf{T} \cup \mathbf{B}$.

Definition 12 (The strongly connected computer). Let $\text{SCC}(k)$, $k \in \{2, 3, 4, \dots\}$ be the k^{th} *strongly connected computer*, which simulates any system of $n = 2^k$ boolean units. A reference implementation with precise details of every unit and connection in $\text{SCC}(k)$ can be found at https://github.com/CSC-UW/Findlay_et_al_2024a. Main text Fig. 3E illustrates how the 2^{nd} (i.e., four-bit) strongly connected computer is constructed from its weakly connected counterpart. Briefly, $\text{SCC}(k)$ is constructed by making the following modifications to $\text{WCC}(k)$:

- The second-to-last unit in each program line, $\mathbf{P}_i^{n-1}$, $i \in \{1, 2, \dots, 2^n\}$ becomes an OR unit, and a new input to this unit from $\mathbf{IR}_i$ is introduced.
- The core oscillator $\mathbf{C}_0$ becomes an OR unit, and a new negated input is added to this unit from every data register's AND unit, $\mathbf{R}_j^{\text{AND}}$, $j \in \{1, 2, \dots, n\}$.

Definition 13 (State sequences, feasibility, and reachability). For a model U with transition probabilities $p(u \mid \bar{u})$ for $u, \bar{u} \in \Omega_U$. Following [34], we define a sequence of micro updates, from $t = a$ to $t = b$ as

$$u_{[a,b]} = (u_a, u_{a+1}, \dots, u_{b-1}, u_b) \quad a, b \in \mathbb{Z}, \quad a < b.$$

The sequence's length is defined as:

$$|u_{[a,b]}| = b - a + 1 = \tau \quad \tau \in \mathbb{N}^+.$$

A sequence of updates is *feasible* if

$$p(u_{i+1} \mid u_i) > 0 \quad \text{for all } u_i \in u_{[a,b]}, \quad a \leq i < b.$$

A state u_b is reachable from u_a in exactly τ updates if there exists a feasible $u_{[a,b]}$ with $|u_{[a,b]}| = \tau$. The set of all states reachable from u_a in exactly τ updates, and the set of all states generally reachable from u_a , are therefore given by

$$R(u_a, \tau) = \{u_b \in \Omega_U : \text{there exists a feasible sequence } u_{[a,b]}, \quad |u_{[a,b]}| = \tau\} \quad (1)$$

$$R(u_a) = \bigcup_{\tau} R(u_a, \tau) \quad \tau \in \mathbb{N}^+. \quad (2)$$

We can also overload this operator to accept as its argument a set of states $\Omega'_U \subseteq \Omega_U$,

$$R(\Omega'_U, \tau) = \{u_b \in \Omega_U : \text{there exists a feasible sequence } u_{[a,b]}, \quad |u_{[a,b]}| = \tau, \quad u_a \in \Omega'_U\} \quad (3)$$

$$R(\Omega'_U) = \bigcup_{\tau} R(\Omega'_U, \tau) \quad \tau \in \mathbb{N}^+. \quad (4)$$

In order for $\text{WCC}(k)$ or $\text{SCC}(k)$ to simulate a target system, they must be initialized properly. A short description of this procedure is given in [Step-by-step guide to the computer's operation: Initialization](#), and a complete reference implementation is found at the GitHub link above. Let $U \in \{\text{WCC}(k), \text{SCC}(k)\}$ be a computer of order k , and let Ω_0 be that computer's set of valid initial states. For every conceivable target system T of $n = 2^k$ boolean units, the proper initial state of U to simulate T is in Ω_0 . Because there are 2^{n2^n} possible systems T (equivalently, there are this many possible programs), 2^n possible initial states of T , and one valid initial state of U for each combination of these, $|\Omega_0| = 2^{n2^n+n}$. Following Definition 13, $R(\Omega_0)$ is the set of all states that U could possibly visit in the course of any conceivable simulation. The set of all valid initial states, Ω_0 , and the set of all reachable states, $R(\Omega_0)$, are the same for both the strongly and weakly connected computers.

Micro-grain analysis of the weakly connected computer.

Claim 11. Let $U = \text{WCC}(k)$ be the k^{th} weakly connected computer with valid initial states Ω_0 . For every reachable state $u \in R(\Omega_0)$, $\varphi_s(u) = 0$.

Proof. U is not strongly connected, because the clock and program are feedforward to the rest of U , so is reducible per Claim 1. ■

Claim 12. Let $U = \text{WCC}(k)$ be the k^{th} weakly connected computer with valid initial states Ω_0 , and $n = 2^k$. For every reachable state $u \in R(\Omega_0)$, $\mathbf{P}$ forms 2^n separate n -unit complexes, each with $\Phi \leq \frac{3n}{2}$.

Proof. Per Claim 11, the computer as a whole has $\varphi_s(u) = 0$. However, this does not mean that all subsystems $S \subset U$ must have $\varphi_s(s) = 0$. Any strongly connected subset of the computer could be a complex. The only strongly connected subsets of U that include units from the program are the 2^n program lines in $\mathbf{P}$, each of which is an n -unit COPY ring. It follows from Claims 1 and 6 that these form 2^n separate complexes, each with $\Phi < \frac{3n}{2}$ (e.g., main text Fig. 3C). ■

Claim 13. Let $U = \text{WCC}(k)$ be the k^{th} weakly connected computer with valid initial states Ω_0 . For every reachable state $u \in R(\Omega_0)$, $\mathbf{C}_0$, $\{\mathbf{X}_i\}_{i=1}^k$, and $\{\mathbf{A}_i\}_{i=1}^k$ form at most $2k + 1$ single-unit complexes.

Proof. Per Claim 11, the computer as a whole has $\varphi_s(u) = 0$. However, this does not mean that all subsystems $S \subset U$ must have $\varphi_s(s) = 0$. Any strongly connected subset of the computer could be a complex. The only strongly connected subsets that include units from the timekeeping chain are the $2k + 1$ monads. ■

Remark. Although not proven (or necessary) here, we could be more precise about the complexes that form: C_0 always, X_1 through X_{k-1} always, X_k only if A_{k-1} is ON, and A_i , $i \in \{1, 2, \dots, k\}$ only if all its inputs are ON. In other words, we are being charitable to the computer here and elsewhere, listing all the complexes that could form in any reachable state, even if they do not actually exist most of the time.

Claim 14. *Let $U \in \{\text{WCC}(k), \text{SCC}(k)\}$ be an $2^k = n$ -bit computer in a reachable state $u \in R(\Omega_0)$. Let $S \subseteq U$ be a system within U , in a reachable state $s \subseteq u$. If S includes some data register's AND unit, then either it includes the output of the k^{th} frequency divider or it is reducible:*

$$\exists i : R_i^{\text{AND}} \in S \implies (A_k \in S \vee \varphi_s(s) = 0)$$

Proof. Let $S \subset U$ be a candidate system and without loss of generality, $R_1^{\text{AND}} \in S$. Assume $A_k \notin S$. There are two cases:

1. Case 1: A_k is 0. Then R_1^{AND} is externally determined. It follows from Claim 2 that S is reducible with $\varphi_s(s) = 0$.
2. Case 2: A_k is 1. When evaluating causes, knowledge of the current state of u is used to compute the probability distribution over potential prior states of the background units (Eqn. 5 in [18]). The past state of A_k was 0 with probability one. Thus the current state of R_1^{AND} is externally determined. It follows from Claim 2 that S is reducible with $\varphi_s(s) = 0$.

Both cases lead to $\varphi_s(s) = 0$, therefore either $A_k \in S$ or $\varphi_s(s) = 0$. ■

Claim 15. *Let $U = \text{WCC}(k)$ be the k^{th} weakly connected computer in a reachable state $u \in R(\Omega_0)$. Let $S \subseteq U$ be a system within U , in a reachable state $s \subseteq u$. If S includes some data register's R^{AND} unit then $\varphi_s(s) = 0$:*

$$\nexists i : R_i^{\text{AND}} \in S \implies \varphi_s(s) = 0.$$

Proof. Assume $\exists i : R_i^{\text{AND}} \in S$. It follows from Claim 14 that either $A_k \in S$ or $\varphi_s(s) = 0$. But if $A_k \in S$, S is weakly connected and reducible per Claim 1 with $\varphi_s(s) = 0$. ■

Claim 16. *Let $U = \text{WCC}(k)$ be the k^{th} weakly connected computer in a reachable state $u \in R(\Omega_0)$. The only candidate complexes that include a unit outside of the program or timekeeping chain are monads in R^{SIM} .*

Proof. Let $\mathbb{S}^*$ be the set of complexes that exist within U . Let $S \in \mathbb{S}^*$ in a reachable state $s \subseteq u$ be a complex that includes at least one unit outside of the program or timekeeping chain. It follows from Claim 15 that $\nexists i : R_i^{\text{AND}} \in S$. From Claim 1, S must be strongly connected. The only strongly connected S are monads $S = \{R_i^{\text{SIM}}\}$ for $i = 1, \dots, n$. ■

Claim 17 (Complexes of the weakly connected computer). *Let $U = \text{WCC}(k)$ be the k^{th} weakly connected computer with valid initial states Ω_0 , and $n = 2^k$. For every reachable state $u \in R(\Omega_0)$, the following complexes may form:*

1. C_0 forms at most one single-unit complex, with $\Phi \leq 2$.
2. $\{X_i\}_{i=1}^k$ form at most k separate single-unit complexes, each with $\Phi \leq 2$.
3. $\{A_i\}_{i=1}^k$ form at most k separate single-unit complexes, each with $\Phi \leq 2$.
4. R^{SIM} form at most n separate single-unit complexes, each with $\Phi \leq 2$.
5. P form 2^n separate n -unit complexes, each a ring with $\Phi \leq \frac{3n}{2}$.

Proof. Items 1-3 follow from Claim 13. Item 4 follows from Claim 16. Item 5 follows from Claim 12. The bounds on Φ follow from Claim 10 and Claim 6. ■

This suffices to show that the weakly connected computer at the micro grain—no matter its size or program—can only support cause-effect structures with first-order mechanisms and linearly increasing (or constant) Φ .

Micro-grain analysis of the strongly connected computer.

Claim 18. Let $U = \text{SCC}(k)$ be the k^{th} strongly connected computer in a reachable state $u \in R(\Omega_0)$. Let $S \subseteq U$ be a system within U , in a reachable state $s \subseteq u$. If $|S| > 1$, $A_k \in S$ and $\varphi_s(s) > 0$, then $C_0 \in S$ and $A_k = 0$.

Proof. Suppose $S \subseteq U$ is a candidate system with $\varphi_s(s) > 0$ and $A_k \in S$. It follows from Claim 1 that S is a strongly connected, which implies that $C_0 \in S$.

Assume $A_k = 1$. Since $s \subseteq u$, $u \in R(\Omega_0)$, $A_k = 1$ implies that $C_0 = 0$ and $X_1 = 0$.

- Case 1: $X_1 \notin S$. Then $\{A_i\}_{i=1}^k$ are externally determined by X_1 . By Claim 2, S is reducible.
- Case 2: $X_1 \in S$. The future state of $\{A_i\}_{i=1}^k$ are determined by C_0 . We cut the outputs of X_1 without loss,

$$\theta = \{X_1 \rightarrow, (S \setminus X_1) \leftarrow\},$$

yielding $\varphi_s(s, \theta) = 0$ (i.e., S is reducible).

The reducibility of S contradicts that $\varphi_s(s) > 0$, and thus $A_k = 0$. ■

Claim 19. Let $U = \text{SCC}(k)$ be the k^{th} strongly connected computer in a reachable state $u \in R(\Omega_0)$. There are no complexes $S \subseteq U$ in a reachable state $s \subseteq u$ that include some data register's R^{AND} unit, as well as the R^{XOR} unit that inputs to it:

$$\exists i : \{R_i^{\text{AND}}, R_i^{\text{XOR}}\} \subset S \implies \varphi_s(s) = 0$$

Proof. Consider $S \subseteq U$ such that $\exists i : \{R_i^{\text{AND}}, R_i^{\text{XOR}}\} \subset S$. From Claim 14, $A_k \in S$. From Claim 18, $C_0 \in S$ and $A_k = 0$. We can cut the outputs of R_i^{XOR} without loss,

$$\theta = \{R_i^{\text{XOR}} \rightarrow, (S \setminus R_i^{\text{XOR}}) \leftarrow\},$$

yielding $\varphi_s(s, \theta) = 0$ (i.e., S is reducible). ■

Claim 20. Let $U = \text{SCC}(k)$ be the k^{th} strongly connected computer in a reachable state $u \in R(\Omega_0)$. Any candidate system $s \subseteq u$ that includes a unit from the multiplexer or buffer is reducible:

$$S \cap (\mathbf{M} \cup \mathbf{B}) \neq \emptyset \implies \varphi_s(s) = 0.$$

Proof. For S to have $\varphi_s(s) > 0$, it follows from Claim 1 that S must be strongly connected. If S includes a unit from $\mathbf{M}$ or $\mathbf{B}$, then S must include some register's R^{AND} unit, as well as the R^{XOR} unit that inputs to it, in order to be strongly connected. That is, $\exists i : \{R_i^{\text{AND}}, R_i^{\text{XOR}}\} \subset S$. By Claim 19, S is reducible with $\varphi_s(s) = 0$. ■

Definition 14 ($L_i^\diamond$, $L_i^\blacktriangle$, and $L_i^\blacktriangledown$). Let $U = \text{SCC}(k)$ be the k^{th} strongly connected computer. Let $L_i = P_i \cup \{IR_i\}$ be the i^{th} line of the program, plus its corresponding line of the instruction register. We define the three strongly connected subsets of $L_i \subset U$ as follows:

$$\begin{aligned} L_i^\diamond &= L_i = \{P_i^1, P_i^2, \dots, P_i^n, IR_i\} \\ L_i^\blacktriangle &= L_i \setminus \{IR_i\} = P_i = \{P_i^1, P_i^2, \dots, P_i^n\} \\ L_i^\blacktriangledown &= L_i \setminus \{P_i^n\} = \{P_i^1, P_i^2, \dots, P_i^{n-2}, IR_i\} \end{aligned}$$

Claim 21. Let $U = \text{SCC}(k)$ be the k^{th} strongly connected computer in a reachable state $u \in R(\Omega_0)$. Let $S \subseteq U$ be a system within U , in a reachable state $s \subseteq u$. If S is irreducible and includes some unit in the program or instruction register, then S is a strongly connected subset of that unit's line.

$$(\varphi_s(s) > 0 \quad \wedge \quad S \cap L_i = \emptyset) \implies S \in \{L_i^\diamond, L_i^\blacktriangle, L_i^\blacktriangledown\}$$

Proof. Since S is irreducible, it follows from Claim 1 that S is a strongly connected. Claim 19 and Claim 20 eliminate all strongly connected systems that intersect with L_i , except for $\{L_i^\diamond, L_i^\blacktriangle, L_i^\blacktriangledown\}$. Therefore, S must be one of these candidates. ■

Claim 22. Let $U = \text{SCC}(k)$ be the k^{th} strongly connected computer in a reachable state $u \in R(\Omega_0)$, and $n = 2^k$. Let $S \subseteq U$ be a system within U , in a reachable state $s \subseteq u$.

If all of the following are true:

- S is irreducible ($\varphi_s(s) > 0$)
- S includes any unit outside of the program and instruction register ($S \cap \{\mathbf{R}, \mathbf{M}, \mathbf{T}, \mathbf{B}\} \neq \emptyset$)

- S is not a monad ($|S| > 1$).

then the following are implied:

- $u \in R(\Omega_0, 2nj - 3)$ for some $j \in \mathbb{N}^+$.
- $S = \{C_0, A_k, R_i^{\text{AND}}\}$ for some $i \in \{1, 2, \dots, n\}$.

Proof. Assume all of the premises. Claims 1, 19, 20, and 21 eliminate all candidates for S except those where $\{C_0, A_k, R_i^{\text{AND}}\} \subseteq S$ for some $i \in \{1, 2, \dots, n\}$. The following must also be true:

1. $A_k = 0$. This follows from Claim 18.
2. $C_0 = 0$. If $C_0 = 1$, then it determines its own future state, and we can cut the outputs of any $R_i^{\text{AND}} \in S$ without loss, and S is reducible.
3. $S \cap \{X_i\}_{i=1}^k = \emptyset$. Otherwise we could cut the outputs of $S \cap \{X_i\}_{i=1}^k$ without loss, since C_0 determines the future state of A_k .
4. $S \cap \{A_i\}_{i=1}^{k-1} = \emptyset$. Otherwise S would be weakly connected and reducible.
5. $\{X_i\}_{i=1}^k = 1$. Otherwise, A_k is externally determined, and S is reducible.
6. $u \in R(\Omega_0, 2nj - 3)$ for some $j \in \mathbb{N}^+$. This follows from the reachability of $s \subseteq u$ and the constraints on the states of units $C_0 = 0$ and $A_k = 0$ (both in S), and $\{X_i\}_{i=1}^k = 1$ (not in S).
7. Finally, $|S \cap \{R_i^{\text{AND}}\}_{i=1}^n| = 1$. Assume otherwise, i.e., $\exists i, j, i \neq j : \{R_i^{\text{AND}}, R_j^{\text{AND}}\} \subset S$.

For any $u \in R(\Omega_0, 2nj - 3)$, the current state of S is $s = (0, 0, \dots, 0)$ (all OFF). What is the maximal cause state s'_c (see Eqn. 12 in [18])? Given the current state, $A_k = 0$ in the cause state, because this is sufficient to bring about $R_i^{\text{AND}} = 0$ and $R_j^{\text{AND}} = 0$ in the current state, regardless of the state of background units $W = U \setminus S$, whereas $A_k = 1$ would have no power to bring about the current state s . Moreover, the only candidate cause states $\bar{s}$ that have any power to bring about s are those for which at least one of $C_0, R_i^{\text{AND}}, R_j^{\text{AND}}$ is 1. These cause states are sufficient to bring about $C_0 = 0$, whereas other cause states have no power to bring about s .

Let $\bar{s}_1$ be a candidate cause state such that C_0 is 1 and A_k is 0. Let $\bar{s}_2$ be any candidate cause state such that C_0 and A_k are 0 and at least one of $R_i^{\text{AND}}, R_j^{\text{AND}}$ is 1. Observe that $\bar{s}_2$ is sufficient to bring about s , including $A_k = 0$, regardless of the background state w . In contrast, $\bar{s}_1$ can only bring about s for certain background conditions (e.g., when $\{X_i\}_{i=1}^k = 1$, $\bar{s}_1$ does not bring about s .) Thus, some degradation of the cause information is guaranteed for $\bar{s}_1$ relative to $\bar{s}_2$. More precisely, because $p_c(s \mid \bar{s}_2) > p_c(s \mid \bar{s}_1)$ and $p_c^\leftarrow(\bar{s}_2 \mid s) = p_c^\leftarrow(\bar{s}_1 \mid s)$, this implies $\text{ii}_c(s, \bar{s}_2) > \text{ii}_c(s, \bar{s}_1)$ (see Eqn. 7 in [18]), and therefore $s'_c = \bar{s}_2$.

Without loss of generality, assume that $R_i^{\text{AND}} = 1$ in $s'_c = \bar{s}_2$. Given this cause state, we can cut the outputs of R_j^{AND} without loss (because C_0 's current state was fully determined by R_i^{AND} 's cause state). The reducibility of S contradicts that S is irreducible, and the claim follows. ■

Claim 23 (Complexes of the strongly connected computer). *Let $U = \text{SCC}(k)$ be the k^{th} strongly connected computer with valid initial states Ω_0 , and $n = 2^k$. For every reachable state $u \in R(\Omega_0)$, the following complexes may form:*

1. For each line $\{L_i\}_{i=1}^{2^n}$ of the program and instruction register, at most one of its strongly connected subsets $\{L_i^\blacklozenge, L_i^\blacktriangle, L_i^\blacktriangledown\}$ may form a complex, for a total of 2^n separate complexes. Each complex is either a ring of size n with $\Phi \leq \frac{3n}{2}$ ibits, or an imperfect ring of size $n + 1$ with $\Phi \leq \frac{9(n+1)+64}{6}$.
2. R^{SIM} form at most n separate single-unit complexes, each with $\Phi \leq 2$.
3. $\{X_i\}_{i=1}^k$ form at most k separate single-unit complexes, each with $\Phi \leq 2$.
4. $\{A_i\}_{i=1}^{k-1}$ form at most $k - 1$ separate single-unit complexes, each with $\Phi \leq 2$.
5. $\{C_0\}$ forms at most one single-unit complex, with $\Phi \leq 2$.
6. $\{A_k\}$ forms at most one single-unit complex, with $\Phi \leq 2$.

For states $u \in R(\Omega_0, 2nj - 3)$ for some $j \in \mathbb{N}^+$, the following complex may form:

8. $\{C_0, A_k, R_i^{\text{AND}}\}$ for some $i \in \{1, 2, \dots, n\}$, each with $\Phi \leq \frac{9}{2}$.

Proof. Item 1 follows from Claims 21, 6, and 9.

Per Claim 1, any complex needs to be strongly connected, and Claims 20 and 19 rule out most strongly connected subsets of U . Item 2 follows from the fact that the monads R^{SIM} are the only strongly connected subsets of U that do not involve units in the timekeeping chain and have not already been eliminated by Claims 20 and 19, or covered by Claim 21.

The only remaining complexes to consider are those that might form from units in the timekeeping chain and the AND units ($\{R_i^{\text{AND}}\}_{i=1}^n$) of each data register. Items 3-6 follow from Claim 22, which details the only complex larger than a monad that might form (item 8), and implies that any other complex we have yet to consider must be a monad (items 3, 4, 5, and 6). ■

This suffices to show that the strongly connected computer at the micro grain—no matter its size or program—can only support cause-effect structures with first-order mechanisms (and potentially a single second-order mechanism) and linearly increasing (or constant) Φ .

Macro grain analysis of the weakly and strongly connected computers. In what follows, $U = \text{SCC}(k)$ is always the k^{th} strongly connected computer. Note that the results below also hold for the weakly connect computer. Following Marshall et al. [34], $\mathbb{P}(u)$ is the set of all valid systems (both micro and macro) that can be defined from U in state u . System $S = \{J_1, \dots, J_{|S|}\}$ in state $s \in \mathbb{P}(u)$, with micro constituents U^S in (micro) state $u^S \subset u$, is a valid *macro system* if there exists at least one unit $J \in S$, such that J 's update grain constitutes more than one micro update (i.e., $\tau_J > 1$), or J contains more than one micro constituent (i.e., $|U^J| > 1$). S has background units $W = U \setminus U^S$ in state $w \subset u$. The background is always treated at the micro grain (every $W_i \in W$ is a micro unit). The background apportionments for the system, $W^S \subseteq W$, are simply the (non-overlapping) apportionments of its constituents: $W^S = \bigcup_{i=1}^{|S|} W^{J_i}$.

Here, we relax the restrictions of IIT [34] in only one respect: we do not assume that U is a system of micro units, and instead consider them to be meso units that implicitly satisfy the postulates. This counters an objection that our micro units are stand-ins for what are actually meso units (e.g., our AND unit could be considered a stand-in for an AND gate built from smaller components/transistors). The practical consequence of this perspective is that the individual units of U can be considered as constituents of a macro unit with $\tau > 1$, even if they do not have a “self-loop”. This gives the computer the best possible chance of forming complexes whose cause-effect structures replicate those of a simulated substrate, which strengthens the result. Note that all results below would hold (often trivially), if we treated the U as a set of micro units.

Let $\mathbb{P}^{\text{SIM}}(u)$ be the set of all valid macro systems that include the outputs of all the data registers as constituents (i.e., $R^{\text{SIM}} \subseteq U^S$, $\forall S \in \mathbb{P}^{\text{SIM}}$). It follows from Claim 23 and the requirement that macro units be maximally irreducible within [34] that R^{SIM} are not constituents of other units in S , but are themselves units of S (i.e., $R^{\text{SIM}} \subseteq S$), possibly at a macro update grain. We aim to show that there is no system S in state $s \in \mathbb{P}^{\text{SIM}}(u)$ that replicates the cause-effect structure $C(t)$ of the size- n simulated target system T in state t . The reason we do not show this for any $s \in \mathbb{P}(u)$ is that we are interested in knowing if the macro system replicates $C(t)$ *in virtue of its function as a simulator*. After all, there are many “pathological” choices of t that could result in spurious cause-effect structure replication. For example, t could be designed to mimic precisely a subset of the computer that supports a complex, or to fragment into single-unit complexes whose cause-effect structures are trivial to reproduce. In fact, even the restriction that $s \in \mathbb{P}^{\text{SIM}}(u)$ leaves the door open to some spurious replication of simple cause-effect structures, but it will suffice for us to ultimately demonstrate that simulation does not imply cause-effect structure replication.

We first introduce two conditions related to the potential equivalence of cause-effect structure.

Definition 15 (NON-T). Let $d = (m, z^* = \{z_c^*, z_e^*\}, \varphi_d) \in C(s)$ be a distinction in the cause-effect structure of s , with mechanism M in state m , cause purview Z_c^* in state z_c^* , effect purview Z_e^* in state z_e^* , and irreducibility φ_d . NON-T(s) is the following proposition:

$$\exists d \in C(s) : z^* \not\subseteq R^{\text{SIM}}$$

The NON-T property indicates that a unit outside of R^{SIM} shows up as a purview in the cause-effect structure of s , thus precluding cause-effect structure equivalence with the target system.

Definition 16 (ISOLATED). Let $d = (m, z^* = \{z_c^*, z_e^*\}, \varphi_d) \in C(s)$ be a distinction in the cause-effect structure of s , with mechanism M in state m , cause purview Z_c^* in state z_c^* , effect purview Z_e^* in state z_e^* , and irreducibility φ_d .

ISOLATED^c(i) is the following proposition:

$$R_i^{\text{SIM}} \in z_c^* \implies |z_c^*| = 1, \forall d \in C(s).$$

Similarly for ISOLATED^e(i). Finally, let $\text{ISOLATED}(i) = \text{ISOLATED}^c(i) \vee \text{ISOLATED}^e(i)$.

The ISOLATED(i) property indicates that a unit in R^{SIM} is isolated within a cause-effect structure, meaning that it does not appear alongside other target units in a purview, and thus cannot form relations that bind it with other target units. If a unit is isolated, this precludes the possibility that the cause-effect structure of s can be equivalent to the cause-effect structure of any target system that has relations to bind together units (including $PQRS$).

In what follows, we outline a sequence of claims that together demonstrate the desired result. Specifically, we aim to show:

Claim 24. For every system $s \in \mathbb{P}^{\text{SIM}}(u)$, at least one of the following must be true:

1. $\varphi_s(s) = 0$
2. $\text{NON-T}(s)$
3. $\exists i \in \{1, \dots, n\}$ such that $\text{ISOLATED}(i)$

Proof. The claim follows from Claims 25 - 31, proven below. ■

This result suffices to show that the computer cannot replicate the cause-effect structure of an arbitrary target system (including $PQRS$) in virtue of its function as a simulator.

Claim 25. Let S in state $s \in \mathbb{P}^{\text{SIM}}(u)$ be a macro system. If any register's AND gate is neither a constituent nor apportionment of the system, or is an apportionment of that register's SIM gate, then the system is reducible:

$$R_i^{\text{AND}} \in W \setminus W^S \vee R_i^{\text{AND}} \in W^{R_i^{\text{SIM}}} \implies \varphi_s(s) = 0$$

Proof. By definition, $R^{\text{SIM}} \subseteq S$, $\forall S \in \mathbb{P}^{\text{SIM}}(u)$. Assume $\exists i : R_i^{\text{AND}} \in W \setminus W^S \vee R_i^{\text{AND}} \in W^{R_i^{\text{SIM}}}$. Then we can cut the inputs to R_i^{SIM} without loss, and $\varphi_s(s) = 0$. ■

Claim 26. Let S in state $s \in \mathbb{P}^{\text{SIM}}(u)$ be a macro system. If the output of the multiplexer is neither a constituent of S nor part of some multiplexer unit's apportionment, then Claim 24 holds for s :

$$\text{MO} \notin S \cup W^M \implies \varphi(s) = 0 \vee \text{NON-T}(s) \vee \text{ISOLATED}(i) \forall i.$$

Proof. Assume $\text{MO} \notin S \cup W^M$. It is possible that $\text{MO} \in W^{R_i^{\text{SIM}}}$ for exactly one i , and that $C_0 \in W^{R_i}$ for exactly one i . Thus, there exists at least one W^{R_i} such that $(\text{MO} \cup C_0) \cap W^{R_i} = \emptyset$. Assume without loss of generality that it is $i = 0$ (neither MO nor C_0 are in W^{R_0}).

Consider the possible assignments of R_0^{AND} (it could be a constituent of the system, an apportionment of some other unit, etc.). By Claim 25, if $R_0^{\text{AND}} \in W \setminus W^S$ or $R_0^{\text{AND}} \in W^{R_0^{\text{SIM}}}$, then $\varphi_s(s) = 0$. If $R_0^{\text{AND}} \in W^J$ for some $J \in S$ with $U^J \cap R_0 = \emptyset$ (a mediator for a unit outside R_0), then:

- Case 1: $\exists k$ such that $M_k \in S$. Then the outputs of M_k can be cut without loss (since $\text{MO} \notin S$ and $\text{MO} \notin W^{M_k}$) and $\varphi_s(s) = 0$.
- Case 2: $\nexists k$ such that $M_k \in S$. Then we can cut the outputs of R_0^{SIM} (if $R_0^{\text{XOR}} \notin S$) or the outputs of R_0^{XOR} (if $R_0^{\text{XOR}} \in S$) without loss, and $\varphi_s(s) = 0$.

All of the possible assignments for R_0^{AND} considered so far result in $\varphi_s(s) = 0$. The remaining possibilities are $R_0^{\text{AND}} \in S$, or $(R_0^{\text{XOR}} \in S \wedge R_0^{\text{AND}} \in W^{R_0^{\text{XOR}}})$. Consider a potential distinction that includes R_0^{SIM} in its effect purview. Such a distinction must include units that output to R_0^{SIM} (possibilities are $R_0^{\text{SIM}}, R_0^{\text{AND}}, R_0^{\text{XOR}}$). Now consider whether such a distinction could also include another register output $R_{i'}^{\text{SIM}}$ ($i' \neq 0$) in its effect purview. Units R_0^{SIM} and $R_{i'}^{\text{SIM}}$ share no common inputs, so the only way for such a distinction with R_0^{SIM} and $R_{i'}^{\text{SIM}}$ in its effect purview to be irreducible is to include an additional unit $V \notin R^{\text{SIM}}$ in the effect purview that integrates the effect. If such a V is in the effect purview, then $\text{NON-T}(s)$, otherwise such a distinction does not appear in the cause-effect structure and $\text{ISOLATED}(0)^e$. ■

Thus, Claim 24 holds for any system for which MO is not a unit or a mediator of some M_k . The next set of claims considers the potential role of R_i^{AND} within a system where $\text{MO} \in S \cup W^M$.

Claim 27. Let S in state $s \in \mathbb{P}^{\text{SIM}}(u)$ be a macro system. If the output of the multiplexer is a constituent of S or part of some multiplexer unit's apportionment, and if some register's AND gate is a constituent of the system, then Claim 24 holds for s :

$$\text{MO} \in S \cup W^{M_k} \wedge R_i^{\text{AND}} \in S \implies \varphi_s(s) = 0 \vee \text{NON-T}(s) \vee \text{ISOLATED}^e(i).$$

Proof. Suppose $\text{MO} \in S$ or $\exists k : M_k \in S \wedge \text{MO} \in W^{M_k}$. Also suppose $\exists i : R_i^{\text{AND}} \in S$. Assume without loss of generality that $R_1^{\text{AND}} \in S$.

The only inputs to $R_1^{\text{SIM}} \in S$ are R_1^{SIM} and R_1^{AND} . Therefore any distinction with R_1^{SIM} in its effect purview must have R_1^{SIM} and/or R_1^{AND} in its mechanism. Consider whether some $R_{i'}^{\text{SIM}}$ could also be in the effect purview ($i' \neq i$). Then the mechanism must include some unit J that outputs to $R_{i'}^{\text{SIM}}$ (otherwise it is reducible).

- Case 1: $J = R_1^{\text{AND}}$. Then $A_k \in W^{R_1^{\text{AND}}}$, in order to create the necessary output pathway to $R_{i'}^{\text{SIM}}$. Also, $R_1^{\text{XOR}} \in W^{R_1^{\text{SIM}}}$, providing an output pathway from R_1^{SIM} to R_1^{AND} , otherwise the distinction's cause would be reducible. However, this means that we can cut the inputs to R_i without loss when partitioning s to test for system integration, so $\varphi_s(s) = 0$.

- Case 2: $J \neq R_1^{\text{AND}}$. Then $J \notin R_1$, and J does not share any outputs with R_1^{SIM} or R_1^{AND} . Therefore the distinction is reducible (i.e., $\text{ISOLATED}^e(1)$), or includes some unit outside R^{SIM} in its effect purview (i.e., $\text{NON-T}(s)$). ■

Based on Claim 26, Claim 24 holds for systems where MO is neither a unit nor a mediator of M_k . Based on Claims 25 and 27, Claim 24 also holds in systems where MO is a unit itself, or mediator of some M_k , and any R_i^{AND} are units, background, or mediators of R_i^{SIM} . It remains to be shown for systems where MO is a unit itself, or mediator of some M_k , and each R_i^{AND} is a mediator for some $J \neq R_i^{\text{SIM}}$. We now focus on the potential distinctions that such a system could support, and whether or not they entail $\text{NON-T}(s)$ or $\text{ISOLATED}(i)$ for some i .

Claim 28. *Let S in state $s \in \mathbb{P}^{\text{SIM}}(u)$ be a macro system. Let $d = (m, z^* = \{z_c^*, z_e^*\}, \varphi_d) \in C(s)$ be a distinction in the cause-effect structure of s , with mechanism M in state m , cause purview Z_c^* in state z_c^* , effect purview Z_e^* in state z_e^* , and irreducibility φ_d . Let $\mathbf{X} = \mathbf{T} \cup \mathbf{B}$ be the set of all meso units in the buffer and timekeeping chain.*

If (1) MO is a unit itself, or a mediator of some multiplexer unit, and (2) each R_i^{AND} is a mediator for some $J \neq R_i^{\text{SIM}}$, and (3) any mechanism $m \in d \in C(s)$ includes a member of $\mathbf{X}$, then (4) Claim 24 holds for s :

$$(\text{MO} \in S \cup W^M) \wedge (\exists J \neq R_i^{\text{SIM}} : R_i^{\text{AND}} \in W^J, \forall i) \wedge (U^m \cap \mathbf{X} \neq \emptyset) \implies \text{NON-T}(s)$$

Proof. Assume all the premises, and consider any $i \in \{1, \dots, n\}$. Since $\text{MO} \in S$ or $\exists k : M_k \in S \wedge \text{MO} \in W^{M_k}$, and $R_i^{\text{AND}} \notin W^{R_i^{\text{SIM}}}$, $\forall i$, there is no path consisting only of units in $W^{R_i^{\text{SIM}}}$ from R_i^{SIM} to any unit in $\mathbf{X}$, including $U^m \cap \mathbf{X}$. Without such a path, it is possible to cut $U^m \cap \mathbf{X}$ away from Z_c^* without loss ($\varphi_d = 0$) unless there is another unit $J \notin R^{\text{SIM}}$ that integrates the cause (i.e., $\text{NON-T}(s)$). ■

Claim 29. *Let S in state $s \in \mathbb{P}^{\text{SIM}}(u)$ be a macro system. Let $d = (m, z^* = \{z_c^*, z_e^*\}, \varphi_d) \in C(s)$ be a distinction in the cause-effect structure of s , with mechanism M in state m , cause purview Z_c^* in state z_c^* , effect purview Z_e^* in state z_e^* , and irreducibility φ_d . Let $\mathbf{X} = \mathbf{P}$ be the set of all meso units in the program.*

If (1) MO is a unit itself, or a mediator of some multiplexer unit, and (2) each R_i^{AND} is a mediator for some $J \neq R_i^{\text{SIM}}$, and (3) any mechanism $m \in d \in C(s)$ includes a member of $\mathbf{X}$, then (4) Claim 24 holds for s :

$$(\text{MO} \in S \cup W^M) \wedge (\exists J \neq R_i^{\text{SIM}} : R_i^{\text{AND}} \in W^J, \forall i) \wedge (U^m \cap \mathbf{X} \neq \emptyset) \implies \text{NON-T}(s)$$

Proof. Assume all the premises, and consider any $i \in \{1, \dots, n\}$. Since $\text{MO} \in S$ or $\exists k : M_k \in S \wedge \text{MO} \in W^{M_k}$, there is no path consisting only of units in $W^{\mathbf{X}}$ from $\mathbf{X}$ to any R_i^{SIM} . Without such a path, it is possible to cut $U^m \cap \mathbf{X}$ away from Z_e^* without loss ($\varphi_d = 0$) unless there is another unit $J \notin R^{\text{SIM}}$ that integrates the effect (i.e., $\text{NON-T}(s)$). ■

Based on Claim 26, Claim 24 holds for systems where MO is neither a unit itself, nor a mediator of some M_k . Based on Claims 25 and 27, Claim 24 also holds in systems where MO is a unit or mediator of some M_k , and any R_i^{AND} are units, background, or mediators of R_i^{SIM} . Moreover, based on Claims 28-29, for systems where MO is a unit or mediator of some M_k , and each R_i^{AND} is a mediator for some $J \neq R_i^{\text{SIM}}$, no unit in $\mathbf{B} \cup \mathbf{T} \cup \mathbf{P}$ can contribute to a mechanism. We continue to focus on the potential distinctions that such a system could support, and whether or not they entail $\text{NON-T}(s)$ or $\text{ISOLATED}(i)$ for some i .

Claim 30. *Let S in state $s \in \mathbb{P}^{\text{SIM}}(u)$ be a macro system. If any multiplexer unit connects to every register's SIM gate, then Claim 24 holds for s :*

$$\exists J \in \mathbf{M} \cap S : J \rightarrow R_i^{\text{SIM}}, \forall i \implies \text{NON-T}(s) \vee \varphi_s(s) = 0.$$

Proof. Assume $\text{MO} \rightarrow R_i^{\text{SIM}}, \forall i$ (a similar argument holds for the case where $\exists k : \text{MO} \in W^{M_k}$). It follows from the irreducibility of s that (1) $\exists d = (m, z^*, \dots) \in C(s) : m = R^{\text{SIM}}$, and (2) $z_e^* = \text{MO}$, and thus (3) $\text{NON-T}(s)$. If this distinction did not exist (i.e., $\varphi_d(d) = 0$), it would mean that the mechanism could be partitioned along some $\theta \in \Theta(M, Z)$ (i.e., $\theta \in \Theta(R^{\text{SIM}}, \{\text{MO}\})$) without loss. However, this cut would also be available at the system level (i.e., $\theta \in \Theta(S)$, though this is not necessarily true in general), and thus $\varphi_s(s) = 0$. Said another way, a cut that makes d reducible cuts one or more connections from R^{SIM} without loss. The particular connectivity structure of s is such that the same cut is in $\Theta(S)$, meaning that the same connections that can be cut without loss are available assessing system integrated information, and thus $\varphi_s(s) = 0$. ■

Claim 31. *Let S in state $s \in \mathbb{P}^{\text{SIM}}(u)$ be a macro system. If the multiplexer's output is either a constituent of S or is part of some multiplexer unit's apportionment, then Claim 24 holds for s :*

$$\text{MO} \in S \cup W^M \implies \varphi_s(s) > 0 \vee \text{NON-T}(s) \vee \exists i : \text{ISOLATED}(i)$$

Proof. Assume the premises.

It follows from Claims 25-27 that either Claim 24 holds, or $\exists J \neq R_i^{\text{SIM}} : R_i^{\text{AND}} \in W^J, \forall i$.

It follows from Claims 28 and 29 that no units in $\mathbf{T} \cup \mathbf{P} \cup \mathbf{B}$ contribute to any mechanism in $C(s)$.

It follows from Claim 30 that there exists a unit $R_i^{\text{SIM}} \in S : \neg \wp(\text{MO}, R_i^{\text{SIM}}) \vee \neg \wp(\text{M}_k, R_i^{\text{SIM}})$. Assume, without loss of generality, that this unit is R_1^{SIM} .

The only possible mechanism units that output to R_1^{SIM} are itself and R_1^{XOR} :

$$R_i^{\text{SIM}} \in z_e^* \implies \{R_i^{\text{SIM}}, R_i^{\text{XOR}}\} \subseteq m.$$

These units do not output to any $R_{i'}^{\text{SIM}}, i' \neq 0$. For $R_{i'}^{\text{SIM}}$ to be in the effect purview ($R_{i'}^{\text{SIM}} \in Z_e^*$), there must be some additional $J \in m$ such that $\wp(J, R_{i'}^{\text{SIM}})$. However, $\wp(J, R_{i'}^{\text{SIM}}) \implies \neg \wp(J, R_1^{\text{SIM}})$. So either there is an additional unit $V \in Z_E^*$ that integrates the distinction (i.e., NON-T(s)) or the effect is reducible by $\theta = \{(\{R_1^{\text{SIM}}, R_1^{\text{XOR}}\}, \{R_1^{\text{SIM}}\}), (\{J\}, \{R_{i'}^{\text{SIM}}\})\}$ and ISOLATED(1). ■

Applying IIT’s postulates to candidate macro units within the computer

For a more complete and general discussion of IIT’s postulates as they apply to macro systems, see [34]. Here, we present a simplified summary of some key concepts, as they apply to the computer in Fig. 5.

A complex and its intrinsic units must comply with IIT’s postulates of physical existence [18]. Thus, like complexes, macro units can only qualify as units if their cause–effect power is irreducible to that of independent subsets (satisfying integration). Otherwise, one could literally build something (a macro unit) out of nothing (non-interacting micro units; Fig. 4C). And, just as complexes must be definite, which translates into maximally irreducible cause–effect power (satisfying exclusion), so must macro units. Otherwise, one could build something (a macro unit) out of nearly nothing (weakly interacting sets of micro units; Fig. 4D).

Exclusion has the further consequence that complexes cannot overlap in their constituent units, implying that their cause–effect power should not be counted multiple times. For the same reason, intrinsic units cannot overlap, and their cause–effect power should not be counted multiple times (Fig. 10A). Thus, the multiplexer micro units cannot simultaneously belong to σ , ς , or v (though an alternative mapping could reassign them, say from ω to ρ). In order not to count intrinsic cause–effect power multiple times, a further constraint for intrinsic units is that they cannot serve as mediators for other intrinsic units within the same complex. For example, one might include the multiplexer units into a macro unit (here, ω) separate from ρ , σ , ς , v . From an extrinsic perspective, one might observe that a change in the state of ρ can affect the states of ρ , σ , ς , and v through a pathway that includes the multiplexer. However, this causal power already “belongs” to the multiplexer’s macro unit ω (Fig. 10B). (This is somewhat akin to “screening-off” in other causal inference frameworks) [112–115]. By a similar logic, if ω were not treated as a macro unit and the multiplexer micro units were reassigned to ρ under an alternative mapping (not shown in Fig. 5), then the other macro units σ , ς , and v would not have direct causal power among themselves, because that would require the contribution of micro units that now belong to ρ .

Similar logic dictates that although micro units constituting the background conditions of a complex can mediate interactions among its intrinsic units, their mediation must be partitioned into causally independent subsets, each associated with a different intrinsic unit. Otherwise, overlaps in the mediating background conditions would allow for counting intrinsic cause–effect power multiple times. For example, consider the subsystem constituted of the four units ρ , σ , ς , and v —the read-out units of the computer. In this case, the multiplexer units that belonged to ω in Fig. 5 do not belong to any unit within the system, but instead are left out of the system entirely, and treated as background units. From the extrinsic perspective of an observer, a change in the state of ρ , σ , ς , and v can still affect each other’s future state through a pathway that includes the multiplexer. However, this extrinsic integration assumes that multiplexer units in the background mediate multiple interactions, doing multiple duty in terms of cause–effect power. This causal overlap can be discounted by apportioning the background conditions among the intrinsic units [34].

Why the computer is unlikely to support a complex with high structure integrated information (Φ)

In “Analysis of the computer” we analyzed all possible micro-grain systems, in all states that might be visited during simulation. The resulting cause–effect structures were stereotyped, lacking higher-order distinctions or complex relations, and their Φ grew either linearly with the number of units being simulated, or was a fixed constant. In “Macro grain analysis of the weakly and strongly connected computers”, the cause–effect structures of macro systems were characterized for all possible grains. The resulting cause–effect structures could not replicate the cause–effect structure of an arbitrary simulated system. This was sufficient to establish that functional equivalence between two systems does not imply phenomenal equivalence, contradicting a central tenet of computational-functionalism.

An outstanding question is, can the computer still support a macro complex with high Φ (i.e., can the computer be “highly conscious,” with rich content of *any* sort)? Perhaps there is “something it is like” to be a subset of the computer, albeit quite different from the simulated system, and not in virtue of the simulation *per se*. While we do not yet have a formal proof against this possibility under IIT 4.0, such a proof was possible under IIT 3.0 [116], and the advancements incorporated into the latest version of the theory only offer further reason to doubt that the computer can support a highly integrated macro complex, though they make obtaining a formal proof more difficult (for several reasons, one of which is described below). However, even if we completely set aside these intuitions and the previous result from IIT 3.0, there are several other reasons to conjecture that the computer cannot support a macro complex with high Φ , outlined in the following.

It is possible that macro grain systems can have more integrated information than their corresponding micro-grain systems [34, 38, 39]. Generally, there are two ways that a macro grain system can accomplish this: (1) increasing its intrinsic information by enhancing determinism (or reducing degeneracy) and thus improving effect (or cause) selectivity; or (2) enhancing integration by increasing interconnectivity within the system at the macro grain. The first scenario arises when groups of homogeneous micro units are averaged together to form a “coarse-grain” macro unit, while the latter scenario arises when specialized and heterogeneous units (and the fault lines between them) are concealed within a “blackbox” macro unit.

The model computer (whether weakly or strongly connected) at the micro grain does not have the right unit functions or connectivity for macro groupings to increase integrated information. For a macro unit to improve effect selectivity, it

should aggregate functionally homogeneous micro units. However, the micro-grain computer is already deterministic, so further improvements in selectivity are not achievable through coarse graining. Similarly, macro units improve integration when they combine sparsely interconnected micro units into larger, more interconnected macro systems. Due to the modular architecture of our system, especially bottlenecks in the time-keeping chain and multiplexer-based CPU, macro grain systems will struggle to achieve the necessary interconnectivity. This limitation is reflected in the macro grain results, which show that register units remain isolated in either the cause or effect purviews, regardless of how micro units are grouped into macro units. Isolated units have low fan-in and fan-out, representing fault lines within the system, and leading to low φ_s .

As mentioned above, all possible micro-grain systems were exhaustively evaluated to identify potential complexes. Such an analysis was possible because, at the micro grain, the computer is sparsely connected and contains externally determined units, which both decreases φ_s and simplifies its computation by eliminating the need to evaluate all possible partitions to identify the minimum partition (MIP) [18]. A similar analysis is not feasible at macro grains, due to the way mappings between micro and macro units, and background apportionments, can inject uncertainty into the TPM. This uncertainty makes analytically identifying the MIP difficult, even if φ_s is vanishingly small. Thus, it is difficult to rule out potential macro systems as having $\varphi_s = 0$ (and, by extension, $\Phi = 0$). However, this same uncertainty that makes finding an analytical solution difficult generally decreases cause-effect power at the macro grain. Specifically, uncertainty in the macro TPM reduces the selectivity of the macro system, and thus both its intrinsic information and integrated information [43].

How the computer compares to modern computer architectures.

An important question is to what extent the results for our simple computer can be extended to other computers. Of course, there is a tremendous diversity of computing machines, which include cell phones, tablets, mainframes, supercomputers, microcontrollers, video game consoles, enterprise servers, and more. All serve different purposes and face different design constraints. It should be no surprise, then, that there is no such thing as a canonical computer architecture.

Nevertheless, most modern computers have some important things in common. For example, common high-level components include: a clock, a data memory, an instruction memory (often the two memories are shared), and a processor containing both an arithmetic logic unit (ALU) and an instruction register. The clock generates a periodic signal that is used to coordinate and synchronize different parts of the computer. The instruction memory stores code which determines the specific actions—chosen from a larger repertoire of possible actions—that a computer will take. The processor repeatedly fetches one of these instructions from memory, determines (“decodes”) the corresponding action, and then executes that action. Executing the action often entails using the ALU to perform some simple operation (e.g., addition, subtraction) on one or more pieces of data fetched from the data memory. In some cases, the result of this operation can influence which instruction is fetched from memory next. This process repeats indefinitely in what is called the “fetch-decode-execute” loop. All of the sophisticated things that a computer does are achieved by cleverly exploiting this loop.

Many implementations of the fetch-decode-execute loop result in a computer that operates sequentially (the order of instructions matters), suffers some sort of computational bottleneck (e.g., the processor can only execute instructions as fast as the ALU can operate), and is organized in a highly modular fashion. This is perhaps partially due to the nature of the computations being performed, but is also for practical reasons. Computers are designed and built by engineers, and sequential, modular systems are easier to analyze than parallel, integrated systems; by contrast, as neuroscientists are well aware, it is difficult to reason about vast, stochastic systems that perform complex functions in parallel. The brain did not evolve to be understood.

So how much is our computer like a “real” computer, given that the latter is somewhat ill-defined? Ours is certainly a toy system, small and unsuitable for practical use. It is missing some common features of modern computers, such as an addressable memory. The instruction it fetches during each fetch-decode-execute loop is never a function of a previous “execute” step. Some of the authors have designed conventional computers made from transistors, and we would be the first to admit that what is presented here is only analogous to those machines.

It is important to point out that our computer is presumed to exist at the “micro grain,” where units are assumed to be indivisible and update their states synchronously at discrete intervals. Real computers are designed and fabricated at a level far above the smallest physical units, are subject to thermal noise, and receive inputs that are temporally jittered and passed on to outputs after variable propagation delays. In most systems, this desynchronization of units due to internal variability is one of the primary reasons for having a clock. We, on the other hand, only need a clock to coordinate steps of the computation. Our target system, PQRS, does not need a clock at all.

There are certain aspects of our toy computer that are not fundamental aspects of all computers, but only apply to a subset of computers. As examples: the program in our computer is not malleable at runtime, as with ROMs (c.f. RAMs) and FPGAs; and the data and program in our computer are separate modules, as with Harvard architectures (c.f. Princeton architectures).

That said, our computer captures many of a computer’s essential aspects. It has a clock, a program memory, a data memory, and a processing unit. It is a clocked sequential circuit with a convergent datapath and a modular architecture. It uses design motifs found in modern architectures, such as T-latches and a multiplexer. The processing unit even resembles an ALU whose opcode (a signal that indicates which operation to perform) doubles as one of its operands. As far as its

functional properties are concerned, we demonstrate that our computer can easily be made generally programmable and Turing-complete without affecting our results.

Ultimately, the reasons that our computer cannot support complexes with high Φ or rich cause-effect structures—much less replicate the cause-effect structure of a richly conscious simulandum—are sufficiently general and high-level as to be mostly abstracted from the ways in which our computer differs from other common architectures. Hence, our results are likely to generalize to most current computer architectures, insofar as they rely on a fetch-decode-execute loop, a central processing unit, a high degree of datapath convergence, and modules which are themselves largely unintegrated.

It is an open question whether certain parallel computing architectures that are very different from more traditional sequential (e.g., Harvard and Princeton) architectures, such as systolic array variants, may have substrates more favorable for supporting complexes of high Φ . It is important to note that many classes of parallel architecture may have relatively little interdependence between the parallel streams of computation, resembling the way in which the human cerebellum is thought to implement segregated parallel streams of computation that appear to be entirely unconscious. On the other hand, a parallel architecture that more closely resembled a grid-like lattice of internally integrated compute nodes, resembling the way in which the human visual cortex is thought to be organized, might stand a good chance of supporting one or more high- Φ complexes.

An interesting direction for future research would be to apply the tools of IIT to analyze computing machines with radically different architectures than those described above. For example, an intriguing question is whether truly neuromorphic computers that mimic the physical organization of our brain and do not rely on a CPU could be designed to achieve both functional *and* phenomenal equivalence to human brains. It is also unclear whether quantum computers, with entangled qubits, might be capable of supporting complexes with high integrated information and rich cause-effect structures [105].

Finally, we note that our most fundamental and important result, the dissociation in principle between sophisticated functions and consciousness, holds regardless of whether or not a high- Φ architecture ever becomes used in digital computing machines: what we have demonstrated is that such a complex would *not* have high Φ in virtue of the function being performed, and that the content of any experience—what the unfolded substrate is—may be dissociated from what the substrate does.

Replication vs. Simulation.

To better understand our results, it may be useful to consider a thought experiment in which various parts of a brain are replaced with functionally-equivalent, silicon substitutes. This experiment is much like the gradual neural replacement scenario often used to discuss organization invariance and the question of “fading qualia” [117]. Consider the following functionally equivalent brains:

1. A normal, organic, biological brain in a particular state, with a particular set of neurons and a particular set of directed connections between them. For the sake of the thought experiment, we assume that this system can be described by a causal model as in [18]. In principle, units of the causal model can be complete neurons, dendritic compartments, individual synapses, etc. For simplicity, let us say that each unit is a neuron, and that it is interactions at the level of neurons that are directly responsible for the quantity and quality of consciousness. The existence of a directed “connection” between neurons simply implies that the state of one neuron can directly affect the state of the other, without needing a third neuron to mediate this effect.
2. A brain just like (1), but where each neuron has been replaced by a silicon chip and wiring, such that there is a 1:1 mapping between the biological neurons in (1) and silicon chips in (2), and such that there is a connection between two chips in (2) if-and-only-if there is a connection between the two corresponding neurons in (1). Each silicon chip is perfectly functionally equivalent to the neuron that it replaces. There are two subcases to consider:
 - (a) The silicon chips use a (possibly neuromorphic) physical architecture such that each chip is integrated (to be precise, “maximally irreducible within” [34]).
 - (b) The silicon chips are unintegrated.
3. A brain like in (2), but where each silicon neuron is physically separated from its neighbors, and instead communicates with other neurons via transceivers. Each input to a given neuron is replaced by a single transceiver dedicated to relaying that input, and the neuron’s output is also replaced by a single dedicated transceiver. Each output transceiver is hardcoded to wirelessly pass its signal along to the appropriate input transceivers. For the sake of argument, assume that each transceiver is a perfect drop-in replacement for whatever physical connection it replaced, and operates entirely independently of all other transceivers (i.e., their signals are not multiplexed in the same frequency bands, etc.). The resulting system is perfectly functionally equivalent to (1) and (2).
 - (a) The silicon chips use a (possibly neuromorphic) physical architecture such that each chip is integrated (to be precise, “maximally irreducible within” [34]).
 - (b) The silicon chips are unintegrated.

4. A brain like in (3), but where transceivers no longer communicate directly with each other. Instead, each output transceiver sends its signal to a central router, and each input transceiver receives its signal from this router. The implementation of the routing circuitry matters, of course, but for our purposes we should only need to assume that it is non-trivial; That is, outputs are not merely connected directly to inputs, but rather there is some kind of computation (checking a lookup table, say) that needs to be done by the router in order to determine which outputs need to be routed to which inputs. There is only one central router servicing all transceivers in the brain (i.e., simulating all of that brain's connections), but it runs fast and its I/O is coordinated so that the resulting system is perfectly functionally equivalent to (1), (2), and (3).
5. A brain like in (2), but where we now replace silicon neurons, rather than the connections between them, with transceivers. Each silicon neuron is replaced by a transceiver, such that there is a 1:1 mapping between the silicon neurons in (2) and transceivers in (5), and such that there is connection between two transceivers in (5) if-and-only-if there is a connection between the two corresponding silicon neurons in (2). Each transceiver is a dumb relay, forwarding the inputs that it receives to a central processing unit (CPU) that computes the next state of the replaced silicon neuron. The CPU sends this state back to the same transceiver, which relays this output to any connected transceivers at exactly the moment when the replaced neuron would have. There is only one CPU servicing all transceivers in the brain (i.e., simulating all of that brain's neurons), but it runs fast and its I/O is coordinated so that the resulting system is perfectly functionally equivalent to (1), (2), (3), (4), and (5). There are many schemes according to which this central processing unit might schedule its workload. For instance, the CPU might compute the next state of each transceiver and the neuron that it represents one-at-a-time, sequentially, and always in the same order. This would provide a deterministic way for the computer to "know" exactly, at any given moment, which neuron it needs to be simulating the function of. Or it may be that each transceiver stamps its transmissions to the CPU with a barcode that identifies what neuron the CPU should simulate when processing that data. The exact scheme does not matter.
6. A brain in which both the local function of each neuron and the connections between neurons are simulated entirely on a single CPU, as was done in this paper. The system is functionally equivalent to (1), (2), (3), (4), and (5).

Our claim, based on the results obtained in this paper, is that systems (1), (2a), and (3a) are similarly conscious, while systems (2b), (3b), (4), (5), and (6) are unconscious as a whole, possibly supporting several small complexes with low Φ and simple cause-effect structures dissociated from the larger system's function.

Supplementary Figures

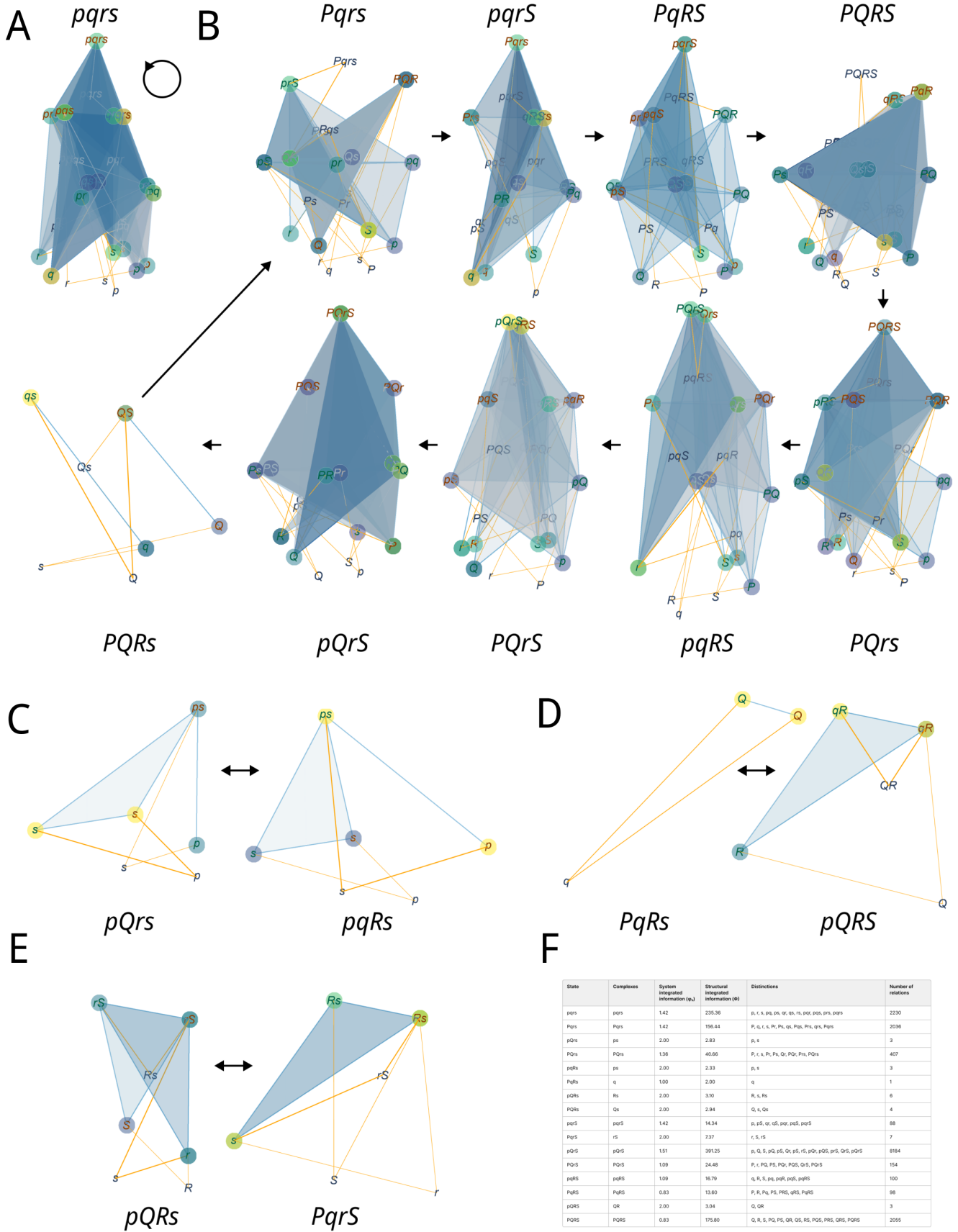


Figure 9: cause-effect structure of PQRS at every state. Dynamical evolution of the unfolded cause-effect structure of the target system PQRS as it transitions through the 16 possible states in its state space. Arrows between cause-effect structures indicate the trajectory of the system from one state to another. Depending on its initial state, PQRS may exhibit any of five cycles. **(A)** In the first cycle, PQRS is trapped in the fixed-point state *pqrs*, with a single maximally irreducible complex of 4 units. **(B)** In the second cycle, PQRS will transit through 9 states: *Pqrs*, *pqrS*, *PqRS*, *PQRS*, *PQRs*, *pQrs*, *pQrS*, *pQRs*, and *PQRs*. In the final eight states, PQRS will be a single, maximally-irreducible complex of four units. In the final state (*PQRs*), only *Qs* will form a complex. **(C-E)** The final three cycles are simple back-and-forth transitions between states where there is never more than a single, two-unit complex. **(F)** Summary of all the complexes and associated cause-effect structures that appear in every possible state of PQRS.

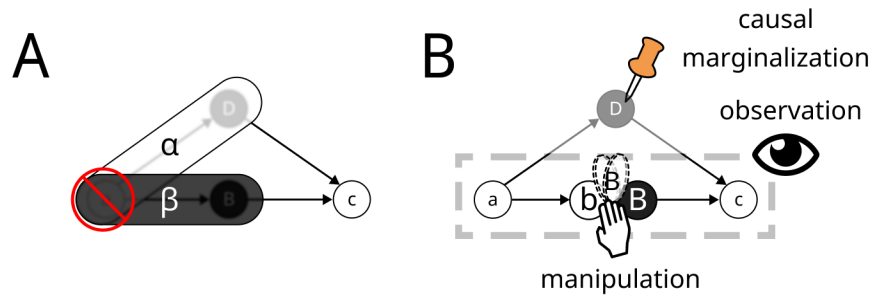


Figure 10: Constraints on intrinsic units. A complex and its intrinsic units must comply with IIT's postulates of physical existence [18]. (A) Units (here, α and β) cannot overlap in their micro constituents, because one should not count cause-effect power multiple times. This also means that it must be possible to manipulate each macro unit (change its state) independently and observe the result. This prohibits ρ , σ , ς , and ν from sharing any of the units that constitute ω in Fig. 5. (B) A complex's units cannot serve as mediators for other units. From an extrinsic perspective, one might observe that a change in the state of a can affect the state of c through a pathway that includes b and is entirely intrinsic to the system. However, from the intrinsic perspective, this causal power "belongs" to b and not a . This is somewhat akin to "screening-off" in other causal inference frameworks [112–115]. Similarly, the causal power of ω must be accounted for separately from the causal power of ρ , σ , ς , and ν in Fig. 5.

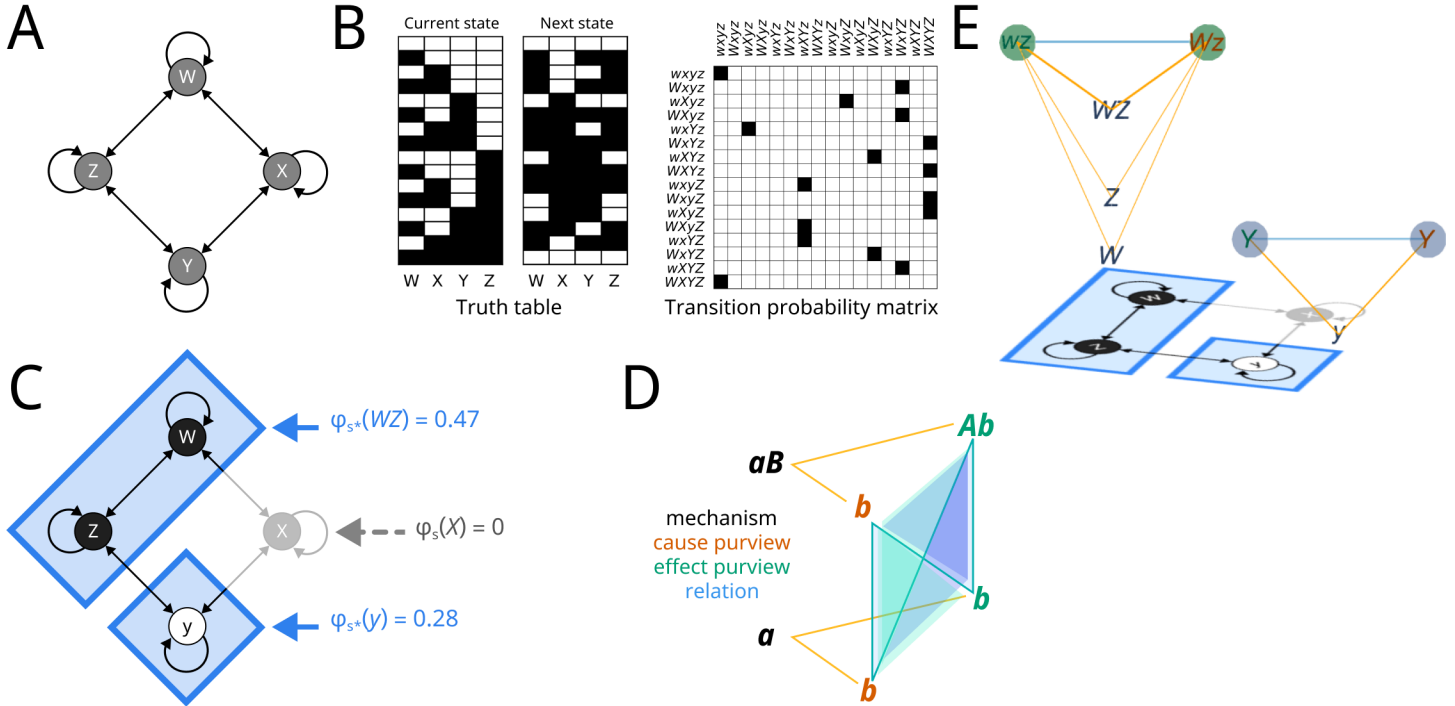


Figure 11: A four-cell elementary cellular automaton implementing Rule 110. The computer is generally programmable, thus can simulate any four-unit system. (A) WXYZ is a substrate of four units with binary states that update synchronously at discrete intervals. Here, the system is shown in state 1101, also written as $WXyZ$. (B) The units W, X, Y, and Z follow the rule 110 of elementary cellular automata, as defined in the truth table. The left table contains all 16 possible states of the system at update t . The right contains the state of the system at $t + 1$ for every state at t . Since all state transitions in WXYZ's dynamics are deterministic, this truth table is also equivalent to WXYZ's Transition Probability Matrix (TPM). (C) $WXyZ$ does meet IIT's postulates for consciousness. By applying IIT's postulates to $WXyZ$ and its TPM, the system is revealed to split into two irreducible complexes ($\varphi_s(WZ) = 0.47$, $\varphi_s(y) = 0.28$). (D) The first unfolded cause-effect structure, WX , consists of 3 distinctions (W , Z , WZ) and 7 relations among distinctions. The second unfolded cause-effect structure, y , consists of 1 distinction (y) and 1 self-relation.

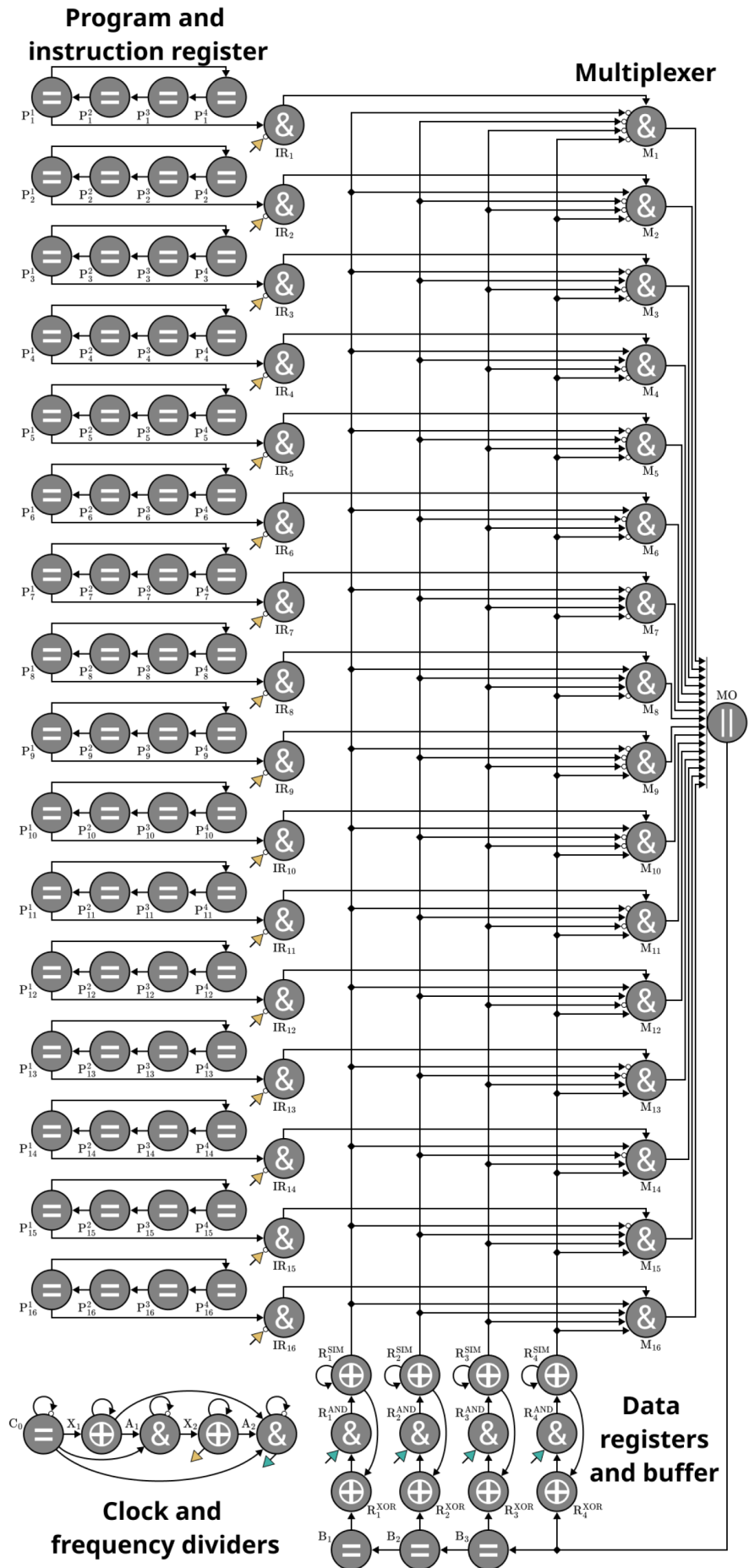


Figure 12: The four-bit computer with labeled units. Unit labels match those used in the supplementary code and proofs.

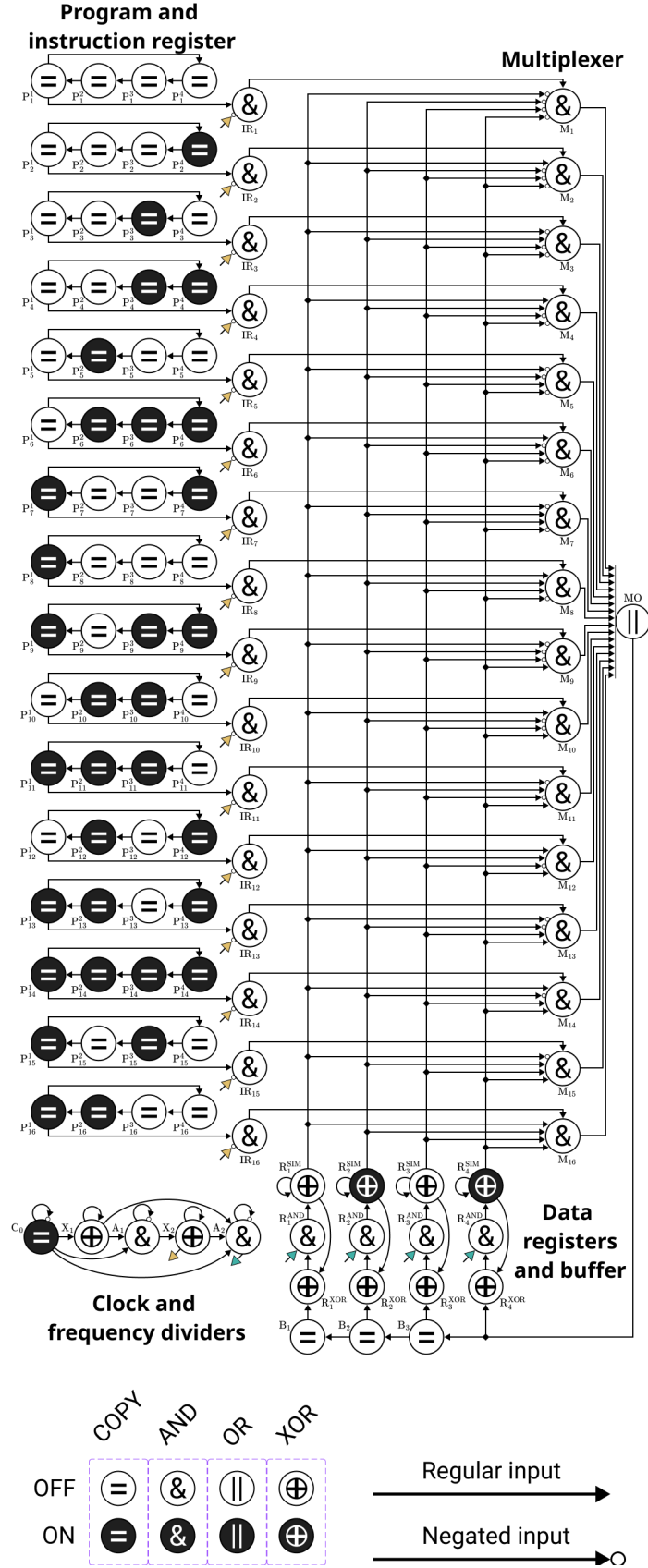


Figure 13: Update 0: Initialization. First, the initial states of units R_1^{SIM} , R_2^{SIM} , R_3^{SIM} , and R_4^{SIM} are set to reflect the state of the target PQRS (here, 0101). Next, the state of the clock is set as shown. Note that X_2 is 0, which will enable (because of the negated input) the instruction register to load values at the next update. Similarly, A_2 is 0, which will disable the data registers from loading new values. Finally, the states of the computer's program units are set in such a way that they encode the target's state transition rules (see main text Fig. 2B).

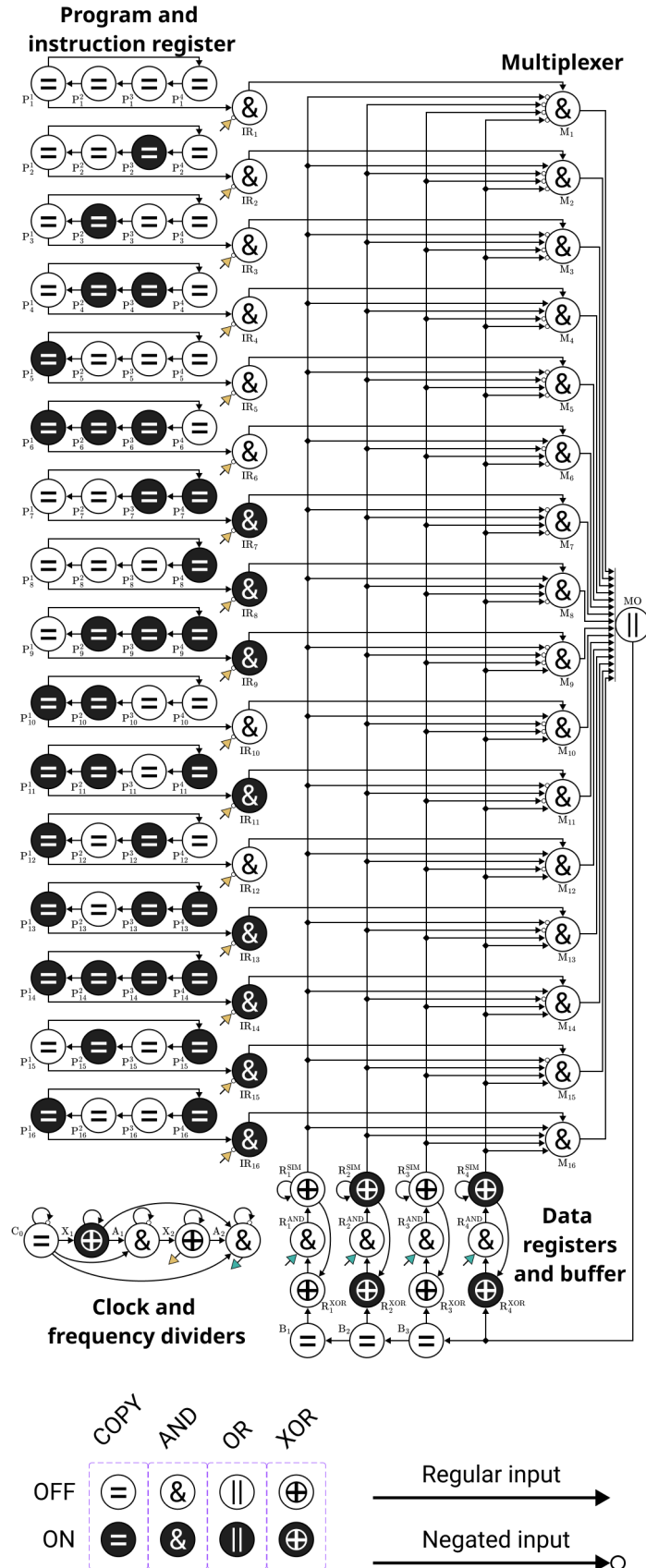


Figure 14: Update 1: The instruction register loads P's truth table, and current state selects a multiplexer input. At update 1, the computer is ready to compute the next state of P by using the current state of P to select a value from its truth table.

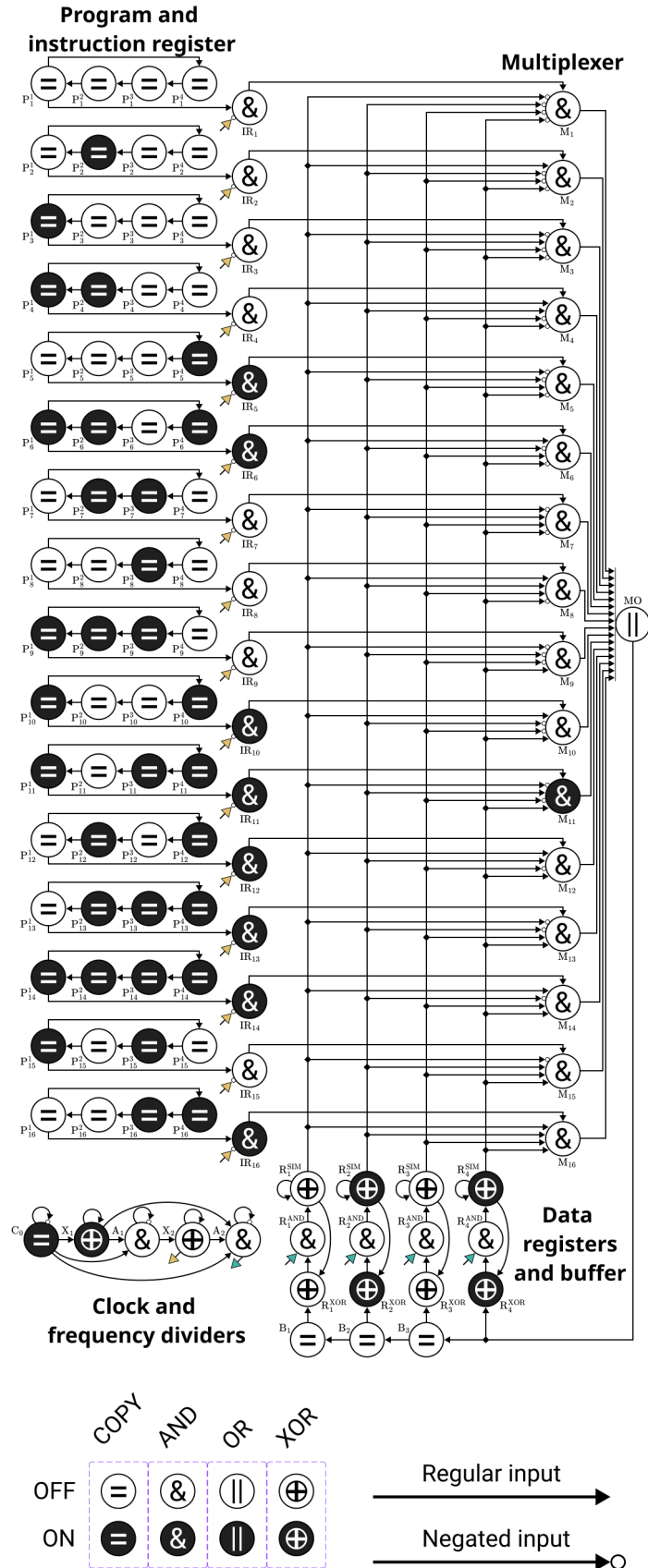


Figure 15: Update 2: The next state of P is computed. The next state of P is computed, while Q's truth table and current state arrive at the multiplexer's inputs.

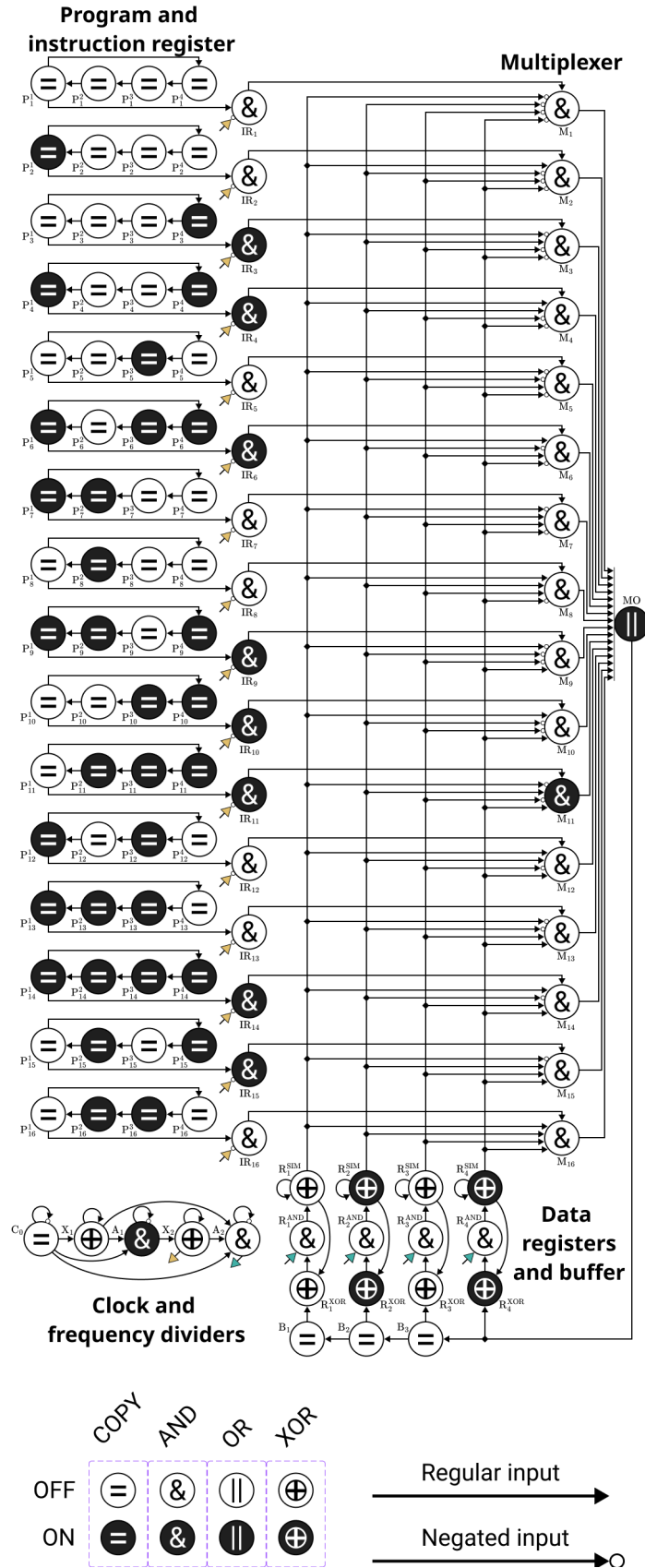


Figure 16: Update 3: The next state of Q is computed. The next state of P propagates towards the registers, the next state of Q is computed, while R's truth table and current state arrive at the multiplexer's inputs.

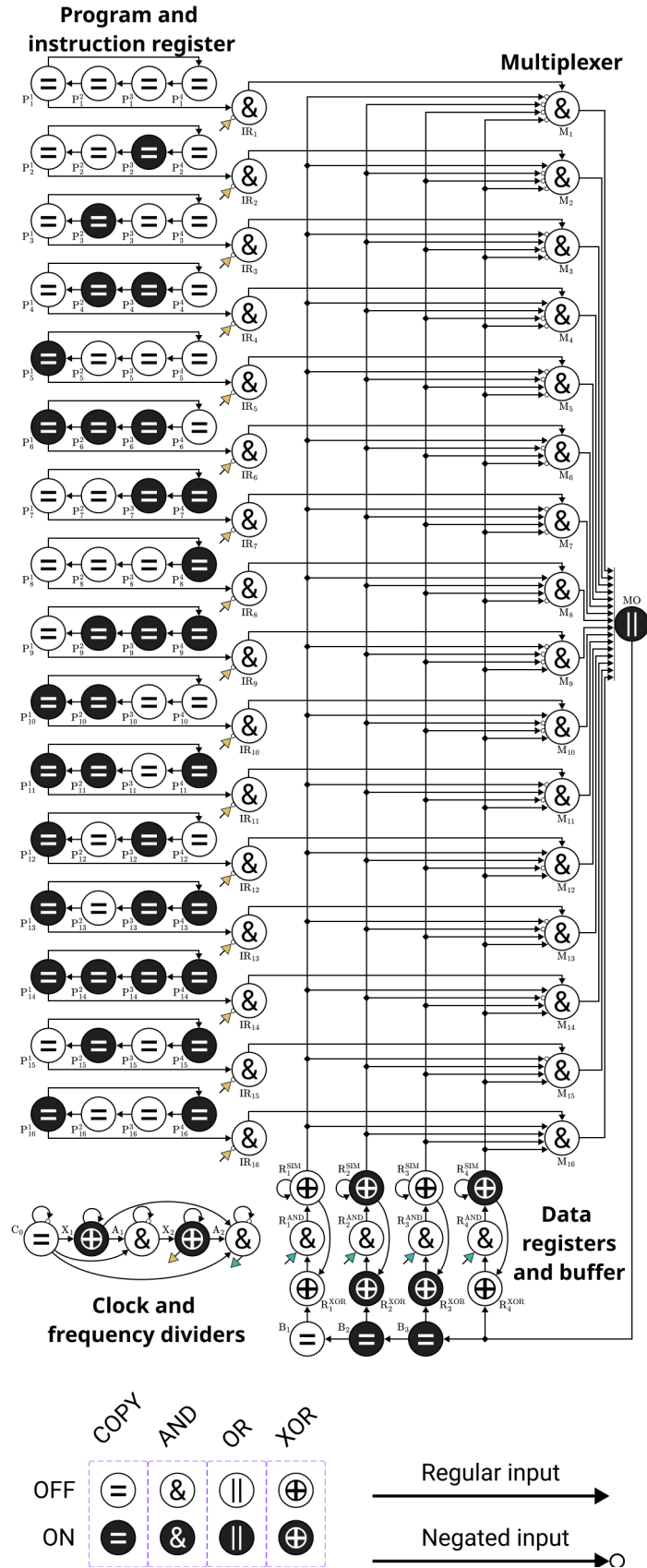


Figure 18: Update 5: The next state of S is computed. P, Q, and R's next state propagate towards the registers, and the next state of S is computed. X_2 is 1, which means that the instruction register will be cleared at the next update.

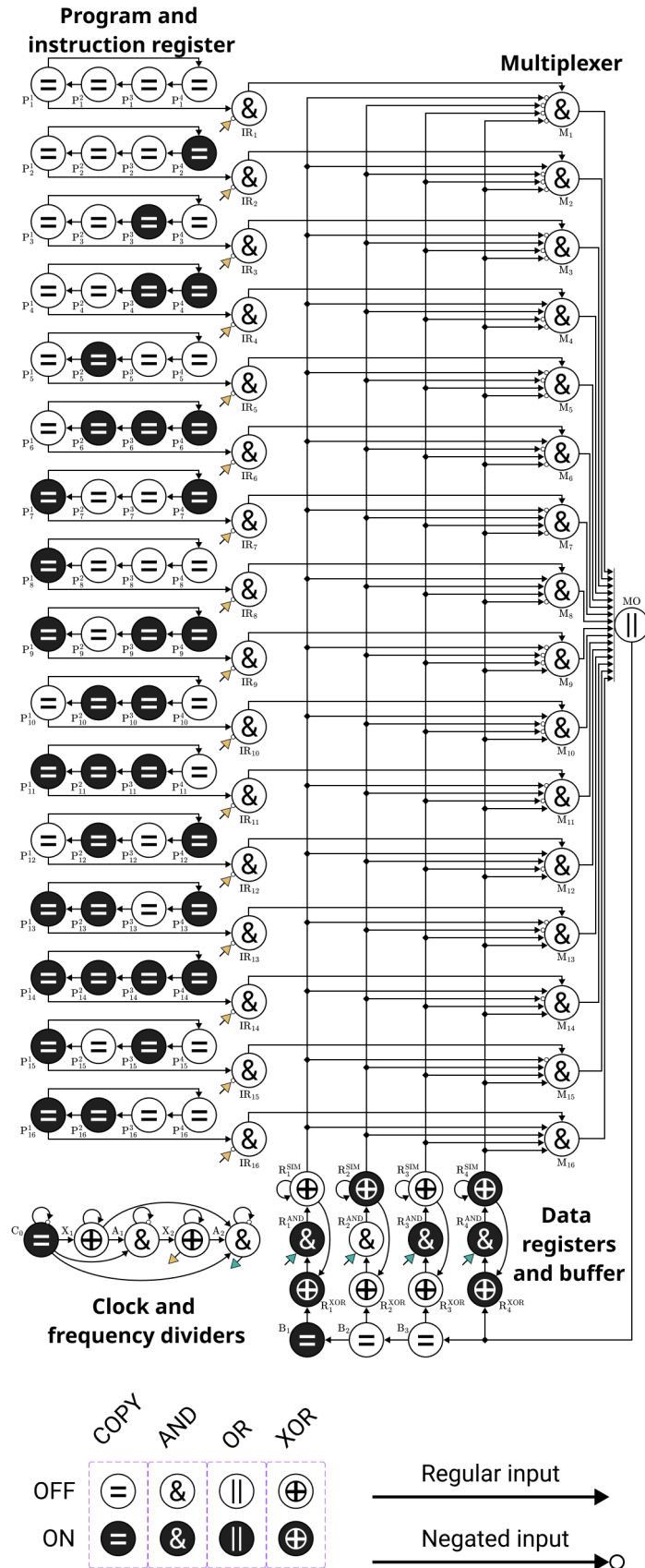


Figure 21: Update 8: Each register's toggle signal arrives at its output. Each register's AND unit reflects whether or not that register's output needs to toggle. This value is compared against the current value of the output, to determine what the next state of the output should be. The program has reset, and X_2 enables the instruction register to begin loading from the program in anticipation of the next simulation iteration.

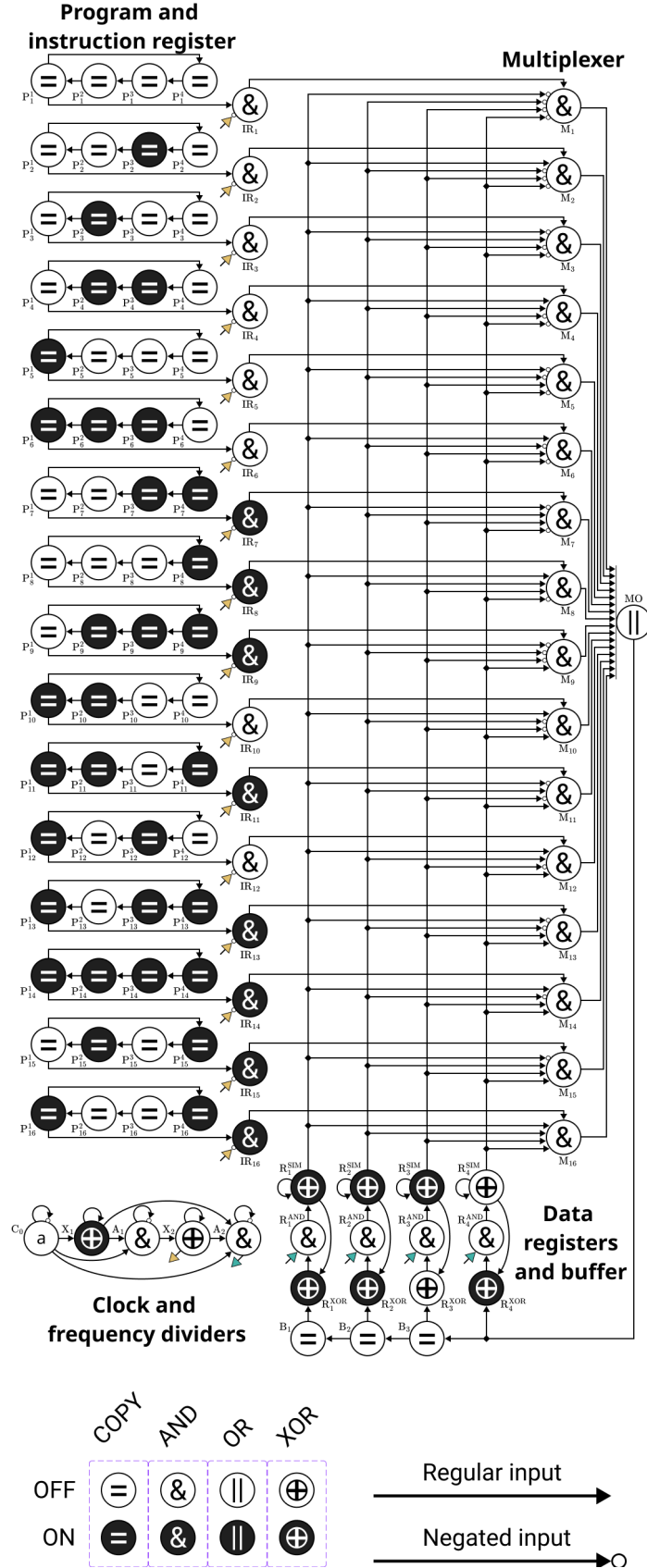


Figure 22: Update 9: The registers adopt the next state of PQRS, and the cycle repeats. R_1^{SIM} , R_2^{SIM} , R_3^{SIM} , and R_4^{SIM} update synchronously to state 1110, the instruction register load's P's truth table, and the next simulation iteration begins.